

微网与外部电网电力调控策略

摘要

本文针对微网在光伏本地消纳与外网购电成本之间的权衡问题，围绕**储能优化调度**，建立由确定性到随机、由单时点到多时点的递进式模型，给出四问的完整结果。全文统一以 10 分钟为时段（一天 144 段），储能容量 [1200, 10800] kWh、单向效率 0.9、功率上限 5000 kW，不售电且允许弃光，缺口按 5 倍电价紧急购电。

问题一为典型日确定性调度。以全天购电费最小为目标建立含 720 个决策变量的稀疏线性规划，用 HiGHS 求解。我们证明该 LP 的松弛是紧的：当往返效率 $0.9^2 = 0.81 < 1$ 时，若某时段同时充放电，可构造等 SOC 扰动使购电量严格下降，故最优解自动满足充放电互斥，**无需引入二元变量**。求解得全天购电费 **35 126.95 元**，较无储能基线 48 052.05 元**节省 26.90%**，其中光伏余电消纳贡献 4 037.75 元、峰谷价差套利 8 887.35 元。

问题二为全年滚动随机日前调度。关键发现是本题的**报童型结构**：计划量照付（缺货成本 $4p$ 、过量成本为电价 p ），缺额按 $5p$ 补救，故临界分位 $F^* = 0.80$ ，计划量应取净负载的 **80% 分位**而非均值。据此建立**两阶段随机规划**：以「计划净供给 $\geq$ 点预测」的约束叠加历史残差场景兜底，配合 48 小时前瞻窗口逐日滚动；预测严格只用历史数据。关键改进是**剔除 1 月 1–7 日的回退残差**（该段负载内核缺少同星期历史），以 56 天等权残差构造场景，2-12 月费用 **1 438.91 万元**。

问题三引入 0/6/12/18 点光伏预报与多时点调整。先将计费规则统一为 $J = \sum_t [p_t g_t^0 + 1.5 p_t u_t - 0.5 p_t v_t + 5 p_t e_t]$ ，并指出**调减项系数必须为 $-0.5p$** （净省 $0.5p$ ），否则最优解永不调整。其次提出**在线预报融合**（以因果最小二乘权重 α 融合历史预测与发布预报）并做收缩偏差校正；进而构建**经验场景树**，节点内共享购电/充放电/SOC 变量，在结构上严格保证非前视。2-12 月费用 **1 402.24 万元**，较同口径基线**节省 2.55%**，紧急购电量减少 28.74%。边际分析表明 6 点与 12 点调整各贡献约 8 万元，18 点几乎无额外收益。

问题四考虑实时波动电价。对 4-2，保留「当天 0 点电价未知」的主模型，以「最近 4 次同星期几均值」预测电价，并构造**电价—净负载联合场景**，理由是 $\mathbb{E}[p_t(N_t - z_t)_+] \neq \mathbb{E}[p_t]\mathbb{E}[(N_t - z_t)_+]$ ，高电价可能与缺电同时发生，不能分解，2-12 月费用 **1 516.30 万元**。对 4-3 叠加多时点调整与预报融合，降至 **1 506.17 万元**。经逐级同口径分解，多时点调整的价值由固定电价下的 15.90 万元提升至波动电价下的 23.34 万元（高约 47%），说明**电价波动显著放大了调整机制的边际价值**。

关键词： 储能优化调度；线性规划；报童模型；两阶段随机规划；滚动时域优化；预报融合；场景树；联合场景

1 问题重述与分析

1.1 问题背景

微网是为解决光伏等分布式能源本地消纳、减少并网问题而发展起来的小型发电系统。出于经济性考虑，**非孤岛式微网**可在外部电网（下称外网）电价较低、或分布式能源有剩余电量时为储能设备充电，并在外网电价较高时释放储存的电能，从而降低购电成本。因此，如何在光伏出力不确定、电价时变的条件下制定**购电与充放电策略**，是微网经济运行的核心问题。

本文研究的微网含光伏发电与储能设备，配置如下（题附录 1）：最大容量 12000 kWh，最大充放电功率 5000 kW，电量须保持在 [1200, 10800] kWh，充放电单向效率均为 90%，2025 年 1 月 1 日 0:00 初始电量为 6000 kWh。微网不得向外网售电；微网供电不足时须向外网**紧急购电**，电价为交易时刻电价的 5 倍。

1.2 数据说明

Table 1.1 附件数据汇总

附件	内容	用途	时间粒度
1	典型日电价、小区负载、光伏预测	问题一全部；二/三问电价与预测回退	10 min
2	全年实际负载与光伏	二/三/四问的预测与结算	10 min
3	每日 0/6/12/18 时的未来 24 h 整点光伏预报	三问、4-3 的调整依据	1 h（整点）
4	全年实际电价	四问的结算电价	10 min
5	结果文件模板	各问 <code>result*.xlsx</code> 格式	—

1.3 统一口径约定

为避免四问之间混淆，全文采用以下一致口径，后续各问不再重复声明。

- 时段划分**：一天划分为 $T = 144$ 个时段，每段时长 $\Delta t = \frac{1}{6}$ h。附件中负载、光伏功率的单位为 kW，须乘以 Δt 折算为每段电量（kWh）。
- 时间标签**：附件中的时刻为**区间末时刻**，即 00:10 表示 00:00–00:10，0:00+1 表示当天 24:00。
- 储能约束**：储电量 $E_t \in [1200, 10800]$ ；每段充、放电量 $c_t, d_t \leq 5000\Delta t = 833.33$ kWh；效率递推为 $E_t = E_{t-1} + 0.9c_t - d_t/0.9$ 。
- 交易规则**：不外送电；允许弃光与弃置多余的计划购电；缺口按 5 倍电价紧急购电。

1.4 四问的递进关系

四问在不确定性维度上层层递进，构成一条完整的建模主线：

问题	电价	光伏信息	决策时点
问题一	已知（典型日）	已知	单时点（日内一次）
问题二	每日固定	仅历史预测	仅 0:00
问题三	每日固定	历史预测 + 发布预报	0/6/12/18 时
问题四	实时波动	历史预测 + 发布预报	4-2: 仅 0:00; 4-3: 0/6/12/18 时

相应地，模型从确定性 LP（问题一）过渡到两阶段随机规划（问题二），再到多阶段场景树随机规划（问题三），最后引入电价随机性形成双重不确定下的联合场景模型（问题四）。

1.5 统计期与 2 月初储电量的确定

附件 5 要求年度结果自 2025 年 2 月 1 日起报，且“充放电量”表须填写该日 0 点的储电量，而题面只锚定了 2025 年 1 月 1 日 0 点的 6000 kWh。因此 2 月初状态须由模型自身确定，本文按“与谁衔接”采用两种约定，结果见表 1.2。

Table 1.2 五组结果的统计期与储电量条件

结果集	统计期及 1 月处理	2 月初/kWh	年末/kWh
问题一	典型日，日初固定	6000	6000
问题二、4-2	2—12 月；1 月纳入同一滚动调度	8550	6000
问题三、4-3	2—12 月；1 月电池不动作、仅积累历史	6000	6000

为何问题二、4-2 是 8550：把 1 月 1—31 日纳入同一套滚动调度，由题给 6000 kWh 连续执行 31 天后，2 月 1 日 0 点落在 8550.00 kWh。该值并非题面参数，而是由 1 月 1 日的窗口自行决定：该窗口覆盖 48 小时，终端约束只加在窗口末且等于窗口起点 6000 kWh，故 1 月 1 日 24 点的库存是自由变量。求解器在 20:00—20:30 高价段（1.340—1.395 元/kWh）放电至下限 1200 kWh，再于 21:50—23:30 低价段（0.386—0.425 元/kWh）充电 8166.67 kWh，库存回升 $0.9 \times 8166.67 = 7350.00$ kWh，于是 1 月 1 日 24 点为 $1200 + 7350 = 8550.00$ kWh。此后各日输入相同、窗口末约束等于当日 0 点库存，故 1 月月末值恒为 **8550.00 kWh**（从 [1200, 10800] 任意起点首日即收敛）。1 月 31 天调度费用 2 109 541.08 元不计入统计期。

8550 的两个边界：其一，它依赖效率口径：若把“90%”解释为往返效率（ $\eta_c = \eta_d = \sqrt{0.9}$ ），落点变为 8315.12 kWh，说明它是模型输出而非题面常量。其二，它只影响 2 月 1 日一天：若改以 6000 kWh 起算，2—12 月费用由 14 389 135.08 变为 14 390 345.38 元，差异仅 1210.30 元且全部落在 2 月 1 日，主要结论不变。

问题三、4-3 为何取 6000：第三问的 1 月仅用于积累同星期历史与预报误差，电池不动作，2 月 1 日沿用题给的 6000 kWh。两种约定都不违反题面（题面未规定 2 月初值），但初始化不同，故跨问的年度总费用不可直接相减；4-2 初值继承问题二，4-3 继承问题三，代码层面强制一致。

1.6 本文的主要特色

1. 报童结构的识别：指出问题二的本质是报童问题，临界分位 $F^* = 0.80$ ，为计划购电量提供了清晰的经济学解释，而非盲目调参。

2. **LP 松弛紧性的严格证明**: 证明问题一无需充放电互斥的二元约束, 大幅降低计算复杂度 (见 §3.4)。
3. **计费规则的统一化建模**: 把问题三的分段计费规则写成单一线性表达式, 并给出系数符号的必要性论证 (见 §5.2)。
4. **场景树保证非前视**: 节点内共享控制变量, 使模型在结构上不可能“偷看”未来信息。
5. **电价—净负载联合场景**: 识别出期望不可分解性 $\mathbb{E}[p \cdot (N-z)_+] \neq \mathbb{E}[p]\mathbb{E}[(N-z)_+]$, 据此构造配对场景。

2 模型假设与符号约定

2.1 模型假设

本文的建模基于以下假设。为便于评阅, 每条假设均给出理由。

- A1. 段内功率恒定**: 附件数据以 10 分钟为间隔, 本文视其为该区间内的平均功率, 故每段电量 = 功率 $\times \Delta t$ 。理由: 题面只给离散采样点, 平均功率是标准离散化处理。
- A2. 储能效率模型为单向恒定效率**: 充电、放电效率各 0.9 (往返 0.81), 不计功率相关性、温度与老化。理由: 题附录 1 只给“充放电效率 90%”, 未给效率曲线; 题面未区分单向/往返, 本文在 §7.2 给出两口径对照。
- A3. 不计储能折旧与运维费用**: 目标只含购电费用。理由: 题面要求最小化购电费用, 未提及设备成本。
- A4. 严格因果性 (非前视)**: 任何决策 (预测器构造、融合权重估计、场景提取与场景树分支) 只使用决策时刻之前已实现的数据。理由: 这是滚动决策可行性的前提; 本文在四问上逐问强化 (问题二只依赖历史, 问题三场景树分支也只依据届时新发布的预报 §5.4.1, 问题四再加电价非前视)。
- A5. 电价按供电时段计价**: 每段费用 = 该段电价 $\times$ 该段购电量。理由: 电量计量发生在用电时刻, 避免把未来时段折算到决策时刻价格。
- A6. 储能充放电互斥 (但不显式约束)**: 本文不加入二元变量, 而是证明在本文结构下最优解自动满足互斥 (§3.4), 并在输出前逐段检查。理由: 见 §3.4 的等价性证明。

题面已明确给定的计费与物理规则 (计划购电照付、取消部分退还原价并另付 50% 违约金、增购按 1.5 倍、紧急购电按 5 倍、不外送电且允许弃光、各问的终端电量要求) 不属于假设, 而是建模的已知条件, 其统一处理见 §1.3; 与各问模型直接相关的终端条件在对应问题的模型建立中陈述。

2.2 全局符号说明

四问共用同一套物理量, 故统一在模型假设后集中说明, 各问不再重复定义。表 2.1 给出全文使用的符号。

Table 2.1 全文符号说明（四问通用）

符号	含义	说明	单位
(一) 时段与数据			
T	时段数	$T = 144$ 个十分钟时段	—
Δt	每段时长	$\Delta t = 1/6$ h	h
t	时段索引	$t = 1, \dots, 144$	—
d	日期索引	年度模型中的日	—
p_t	第 t 段电价	一至三问用附件 1；四问用附件 4	元/kWh
L_t, V_t, N_t	负载、光伏、净负载电量	$N_t = L_t - V_t$ ；附件功率乘 Δt 折算	kWh
(二) 决策变量			
g_t	计划购电量	日前制定的购电量	kWh
g_t^0	0 点原计划购电量	问题三、4-3 的初始计划	kWh
q_t	最终调整后购电量	问题三、4-3 的执行量	kWh
c_t, d_t	充电量、放电量	作用于设备外端	kWh
w_t	弃光量	仅设非负约束	kWh
E_t	段末储能电量	内部库存, $E_t \in [1200, 10800]$	kWh
u_t, v_t	相对原计划的增购量、减购量	$u_t = (q_t - g_t^0)_+$, $v_t = (g_t^0 - q_t)_+$	kWh
e_t	紧急购电量	缺口补救, 按 $5p_t$ 计价	kWh
(三) 效率与储能参数			
η_c, η_d	充电效率、放电效率	主口径各 0.9, 往返 0.81	—
ρ	往返效率	$\rho = \eta_c \eta_d$	—
E_0	初始储电量	题给 2025-01-01 0:00 为 6000	kWh
(四) 随机规划量			
$\hat{N}_t$	净负载点预测	只用历史数据构造	kWh
$N_{s,t}$	场景净负载	场景 s 的第 t 段净负载	kWh
$p_{s,t}$	场景电价	仅问题四使用	元/kWh
s	场景索引	由历史日整日残差构成	—
w_s	场景权重	等权时 $w_s = 1/S$	—
n, π_n	场景树节点索引、节点概率	落入该节点的历史日数占比	—
α	融合权重	在线估计, 见 §5.3.1	—
(五) 评价指标			
J	总费用	分项构成见对应公式	元
λ_t	边际电价（对偶变量）	能量平衡约束的影子价格	元/kWh

记号说明： $(\cdot)_+ = \max\{\cdot, 0\}$ ； $\mathbb{E}[\cdot]$ 为数学期望；带下标 s 的量随场景变化、由对应历史日产生。

3 问题一：典型日确定性储能调度

3.1 问题分析

问题一给定某典型日的电价、小区负载与光伏发电预测功率，要求制定使购电费最小的调度策略。由于该日全部数据已知，这是一个**确定性优化问题**。

其经济本质是两条：

1. **光伏余电消纳**：光伏出力超过负载时，若不储能则被迫弃光；储能可把余电存起

来供后续使用，减少购电。

2. **峰谷价差套利：**在低价时段多购电存入电池，在高价时段放电使用，赚取价差。能否套利取决于价差是否超过往返效率损失。

3.2 符号说明

本问使用的全部符号已在 §2.2 表 2.1 统一说明，此处不再重复。下面直接给出模型。

3.3 模型建立

以全天购电费用最小为目标：

$$\min J_1 = \sum_{t=1}^{144} p_t g_t \quad (3.1)$$

约束条件包括五组：

- (1) **功率平衡**（供给侧 = 需求侧）：

$$g_t + V_t + d_t = L_t + c_t + w_t \quad (3.2)$$

其中 w_t 为弃光量，故外购电与光伏之和，加上电池放电，恰好满足负载与电池充电。

- (2) **弃光非负：**

$$w_t \geq 0 \quad (3.3)$$

- (3) **储能电量递推：**

$$E_t = E_{t-1} + 0.9 c_t - \frac{d_t}{0.9}, \quad t = 1, \dots, 144 \quad (3.4)$$

- (4) **储能容量与功率约束：**

$$1200 \leq E_t \leq 10800, \quad 0 \leq c_t \leq \frac{5000}{6}, \quad 0 \leq d_t \leq \frac{5000}{6} \quad (3.5)$$

- (5) **非负与循环条件：**

$$g_t \geq 0, \quad E_0 = E_{144} = 6000 \quad (3.6)$$

模型共含 $5 \times 144 = 720$ 个决策变量，全部约束为线性，构成一个稀疏线性规划 (LP)，调用 `scipy.optimize.linprog` (HiGHS 内点/单纯形) 求解。

3.4 关键性质：LP 松弛的紧性证明

物理上储能不能同时充电与放电，即要求 $c_t \cdot d_t = 0$ 。但引入这类互斥条件需要二元变量，问题变为混合整数规划 (MILP)，规模与计算量显著上升。我们证明：**在本模型结构下，最优解自动满足互斥，无需显式约束。**

设某时段同时有 $c_t > 0$ 与 $d_t > 0$ 。取

$$\varepsilon = \min \left\{ c_t, 0.81 d_t, \frac{g_t}{0.19} \right\} > 0,$$

并作替换 $c_t \leftarrow c_t - \varepsilon$ 、 $d_t \leftarrow d_t - 0.81\varepsilon$ ，其余变量不变。则

(i) 储电量轨迹与全部 SOC 约束完全不变，因为

$$0.9(c_t - \varepsilon) - \frac{d_t - 0.81\varepsilon}{0.9} = \left(0.9c_t - \frac{d_t}{0.9}\right) - 0.9\varepsilon + 0.9\varepsilon = \left(0.9c_t - \frac{d_t}{0.9}\right);$$

(ii) 充电、放电功率均不增加（故功率约束仍满足）；

(iii) 由功率平衡 (3.2)， g_t 变为 $g_t - 0.19\varepsilon < g_t$ ，即购电量严格下降，费用随之严格下降。

这与最优性矛盾。故最优解中必不存在“该时段有购电且同时充放电”的情形。

经济解释：往返效率 $0.81 < 1$ 意味着“买 1 度电只有 0.81 度能存进电池再取出”。当该时段仍在从外网购电时，把充放电的这部分对冲掉，等价于少买一点电、同时少损耗一点，而负载供应不受影响——因此必然更省钱。

验证：本实例中 LP 最优解的 144 个时段全部满足 $c_t d_t = 0$ ，无需任何后处理，故 LP 松弛是紧的。求解 MILP 作为对照，两者目标值完全相同（35 126.95 元，差 0.000000），从数值上证实了上述证明。

3.5 求解结果

Table 3.1 问题一求解结果对比

方案	购电费（元）	相对基线
无储能基线	48052.05	—
储能在位、优化调度	35126.95	-26.90%

经济性校验：充电加权均价 0.4930 元/kWh，放电加权均价 1.1376 元/kWh，比值 2.31，高于盈亏平衡点 $1/0.81 = 1.235$ ，故储能套利有正收益。

3.6 收益来源分解

把优化结果与无储能基线逐项对比。由逐时段功率平衡 $g_t = L_t + c_t - d_t - V_t + w_t$ （基线取 $c = d = 0$ 且 $w_t = \max(V_t - L_t, 0)$ ）两式相减得 $g_t^{\text{base}} - g_t = \max(V_t - L_t, 0) - w_t + d_t - c_t$ ，逐段乘以 p_t 并求和，可得严格恒等式

$$\sum_t p_t \max(L_t - V_t, 0) - \sum_t p_t g_t = \underbrace{\sum_t p_t [\max(V_t - L_t, 0) - w_t]}_{\text{弃光消除}} + \underbrace{\sum_t p_t d_t - \sum_t p_t c_t}_{\text{净套利}}, \quad (3.7)$$

即“总节省 = 弃光消除 + 净套利”，两项之和恒等于总节省，无重复、无遗漏。本实例优化解中 $w_t \equiv 0$ （全天弃光为零），故弃光消除项即 $\sum_t p_t \max(V_t - L_t, 0) = 4037.75$ 元；一般情形下若存在弃光，须按式 (3.7) 扣除 $\sum_t p_t w_t$ 后等式才严格成立。

Table 3.2 问题一收益来源分解（严格恒等式 (3.7)）

收益来源	金额（元）	占基线	说明
光伏余电消纳	4037.75	8.40%	基线中被弃掉的 6247.96 kWh 全部被电池吸收
峰谷价差套利	8887.35	18.50%	低价购电存入、高价放出，已扣除充电成本
合计	12925.10	26.90%	

套利是主要来源（约 69%）：本例光伏余电仅 6247.96 kWh，而峰谷价差最高 $1.40/0.37 \approx 3.8$ 倍，远超往返盈亏平衡点 $1/0.81 = 1.235$ ，故储能边际价值主要在跨时段搬移电量而非吸收余电。

3.7 电量平衡与调度解释

Table 3.3 问题一全天电量平衡

项目	电量（kWh）	项目	电量（kWh）
外网购电	59482.70	小区负载	111024.81
光伏发电	55482.84	电池充电	20740.67
电池放电	16799.94	弃光	0.00
		电池往返损失	3940.73

为何弃光为 0：光伏总量 55482.84 kWh 全部被利用（部分直接供负载、部分存入电池），说明本日电池容量足以吸收全部余电，储能边际价值主要在套利而非消纳。

为何充放电发生在这些时段：附件 1 电价呈三段特征：凌晨 0–6 时低谷（0.37–0.44 元/kWh，大量充电）、上午 6–9 时尖峰（最高 1.01，放电供负载）、中午回落（0.41–0.53，光伏余电 + 补充电）、傍晚 18–20 时全天最高（1.35–1.40，放电）。调度逻辑即在凌晨与中午低价窗口充电、在上午与傍晚高价窗口放电，最大化价差收益。

3.8 对偶分析

对功率平衡约束 (3.2) 取对偶变量（影子价格） λ_t ，其经济含义为“第 t 段若能多供 1 kWh，购电费能下降多少”。由最优性条件， $\lambda_t \leq p_t$ 时该段不购电（ $g_t = 0$ ）； $\lambda_t = p_t$ 时该段购电为正。对偶量可用来识别“哪些时段的容量瓶颈最值钱”，具体数值已保存于结果文件的 duals.npz 中。

储电量始终位于规定区间内，且日初、日末均为 6000 kWh，因此该收益并非通过消耗日初库存获得。

图 3.2 给出了节省的账面来源：储能使部分时段少购电，对应费用减少 19111.64 元；同时在另一些时段增购电，对应费用增加 6186.54 元。两者抵消后净节省 12925.10 元，降幅为 26.90%。这一分解是逐时段购电费差额的恒等核算，与表 3.2 按能量来源的 4037.75 元与 8887.35 元分解各自自洽，不互相替代，也不把同一节省重复归因。

4 问题二：滚动随机日前调度

4.1 问题分析：报童结构的识别

问题二在每天 0:00 制定当天的计划购电策略，而负载与光伏的实际值未知。题面给出两条关键计费规则：

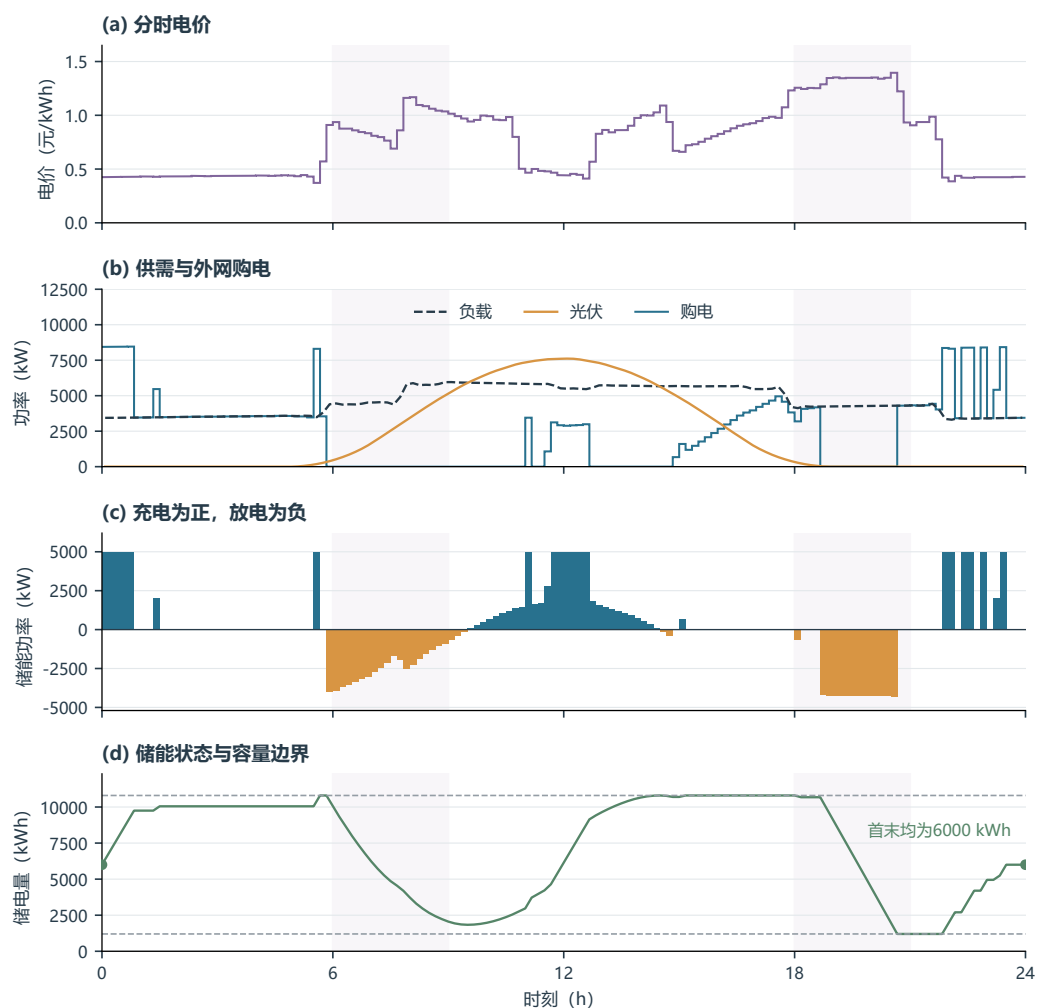


Figure 3.1 典型日确定性调度结果。四个面板共享时间轴，依次展示电价、供需与购电功率、储能充放电功率及储电量；水平虚线为 1200 与 10800 kWh 的储电量边界，浅色背景仅辅助定位早晚时段。

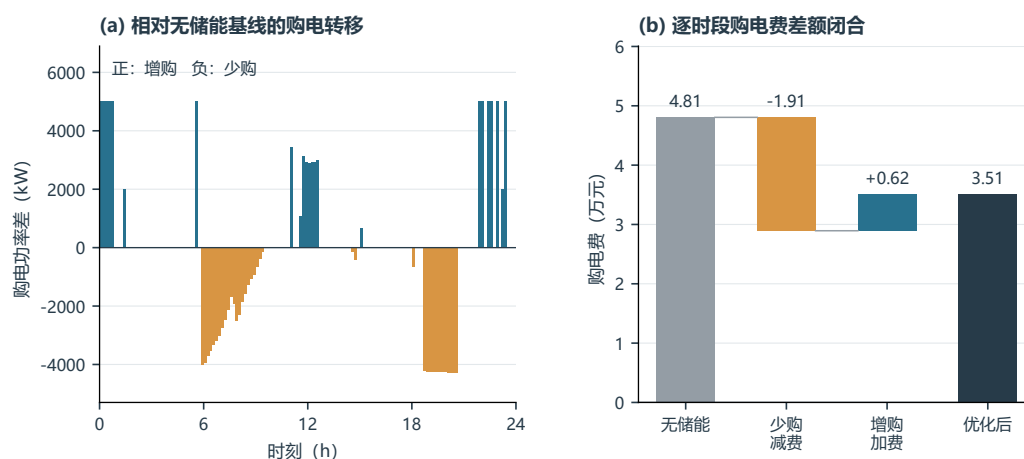


Figure 3.2 储能调度相对无储能基线的购电差异与费用桥接。左图为优化购电功率减去无储能购电功率；右图按同一时段电价分别累计少购所减少的费用和增购所增加的费用。

“微网提供的电能不可低于小区负载，如果低于负载，需向外网紧急购电，其电价是交易时刻电价的 5 倍。除紧急购电费用外，其他时间段的购电费用均按计划购电量计算。”

第二条规则意味着：**计划购电量照付，用不完不退**。于是单时段决策呈现出典型的**报童模型**结构：计划量过高的代价是“多付了钱”，边际成本为电价 p ；计划量过低的代价是“缺额须按 $5p$ 补救”，边际成本为 $4p$ 。最优计划量应满足临界分位数

$$F^* = \frac{c_u}{c_o + c_u} = \frac{4p}{p + 4p} = \frac{4}{5} = 0.80, \quad (4.1)$$

其中 $c_o = p$ 为过量成本， $c_u = 4p$ 为缺货成本。即计划购电量应取净负载分布的**80% 分位**，而非期望值（50% 分位）。这一结论解释了为何“按点预测制定计划”会系统性低估购电量、进而产生大量昂贵的紧急购电。

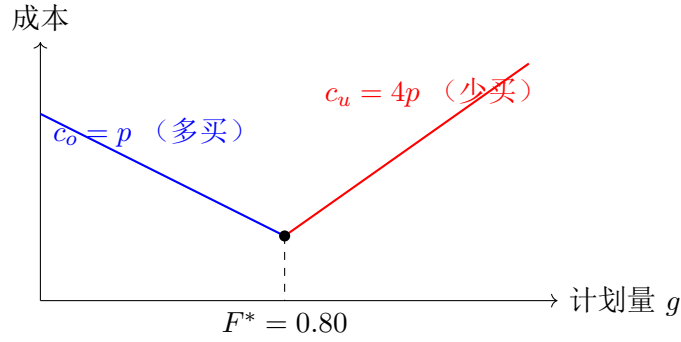


Figure 4.1 报童结构示意图：最优计划量位于 80% 分位，而非均值

4.2 两阶段随机规划模型

本节首先明确一个贯穿第二至四问的前提：**非前视约束**：问题二的特征是“制定当天计划时，当天的负载与光伏均未发生”，故预测与决策**只能使用历史数据**。这一要求不是事后附加的检查项，而是模型可解性的前提：若允许读取当天实际值，问题就退化为问题一的确定性调度。因此本节把它写入模型结构：预测器只用已实现数据；场景取自自己结束的历史日；第一阶段的 g, c, d 对所有场景共享。第三问在此基础上进一步强化为“场景树的节点划分也只能依据届时新发布的预报” (§5.4.1)，第四问再叠加电价的非前视。

设 $\hat{N}_t$ 为净负载点预测 (kWh/段)， $\{N_{s,t}\}_{s=1}^S$ 为历史残差构造的场景， w_s 为场景权重。每天在 48 h 窗口上求解如下两阶段随机规划：

$$\min J_2 = \underbrace{\sum_{t \in 48h} p_t g_t}_{\text{计划购电费}} + 5 \underbrace{\sum_s w_s \sum_{t \in \text{当天}} p_t e_{s,t}}_{\text{期望紧急购电费}} \quad (4.2)$$

约束为：

$$\text{(计划净供给下界)} \quad g_t + d_t - c_t \geq \hat{N}_t, \quad (4.3)$$

$$\text{(场景缺口兜底)} \quad g_t + d_t - c_t + e_{s,t} \geq N_{s,t}, \quad e_{s,t} \geq 0, \quad (4.4)$$

$$\text{(SOC 递推)} \quad E_t = E_{t-1} + 0.9c_t - \frac{d_t}{0.9}, \quad (4.5)$$

$$\text{(容量与功率)} \quad 1200 \leq E_t \leq 10800, \quad 0 \leq c_t, d_t \leq \frac{5000}{6}, \quad g_t \geq 0. \quad (4.6)$$

模型解读：

- 约束 (4.3) 是**不等式**而非等式：计划净供给不低于点预测即可，允许“多买”。这一点至关重要：它使计划量可以落在“80% 分位”而非均值上，与 (4.1) 的报童结论一致。
- 约束 (4.4) 用场景 s 的紧急购电 $e_{s,t}$ 兜底最坏情形，其代价系数为 $5p_t$ 。第一阶段的 g, c, d 对所有场景共享，因此模型天然满足**非前视**要求。

4.3 预测方法（严格因果）

所有预测只使用决策日 d 之前已实现的数据：

Table 4.1 问题二的预测内核

分量	方法	参数
负载	最近 k 次同星期几的均值 + 近 m 天残差漂移	$k = 4, m = 5$
光伏	近 m 天均值 + 近 m' 天残差漂移	$m = 7, m' = 28$

以负载为例，点预测为

$$\hat{L}_{d,t} = \underbrace{\frac{1}{|\mathcal{W}_d|} \sum_{i \in \mathcal{W}_d} L_{i,t}}_{\text{同星期几均值}} + \underbrace{\frac{1}{5} \sum_{i=d-5}^{d-1} (L_{i,t} - \hat{L}_{i,t})}_{\text{残差漂移校正}}, \quad \mathcal{W}_d = \{d-7j : j = 1, \dots, 4, d-7j \geq 0\}. \quad (4.7)$$

为何可以用“近期残差均值”捕捉系统性偏差：设负载的真实分解为 $L_{i,t} = a_t + b_i + \varepsilon_{i,t}$ ，其中 a_t 为日内形状（由同星期几均值估计）、 b_i 为缓慢变化的水平漂移（如天气趋势）、 ε 为平稳噪声。同星期几均值对 b 的估计基于 4 周前的数据，因此在 b 缓慢变化时会留下一个近似常值的偏差 $b_d - \bar{b}_{\text{历史}}$ 。若 b 在 m 天内近似不变，则近期残差 $L_{i,t} - \hat{L}_{i,t} \approx b_d - \bar{b}_{\text{历史}} + \varepsilon_{i,t}$ ，对其取 m 日均值即可消去零均值的 ε ，从而得到漂移项的无偏估计（标准误按 $1/\sqrt{m}$ 缩小）。取 $m = 5$ 是偏差方差与估计方差之间的平衡： m 过小则噪声未消、 m 过大则 b 已变化。这也解释了为何校正对“系统性”偏差有效、对随机噪声无效。

4.4 场景构造与“有效残差”修正

场景采用**整日联合残差**（而非逐时段独立抽样），以保留日内时间相关性：

$$N_{s,t} = \hat{N}_{d,t} + r_{s,t}, \quad r_{s,t} = N_{s,t}^{\text{hist}} - \hat{N}_{s,t}^{\text{hist}}, \quad (4.8)$$

其中 s 取最近 56 个已结束的历史日，等权 $w_s = 1/56$ 。

改进：剔除回退期残差：为说清这一改进的针对对象，先交代问题的来源。本节场景的历史样本来自预测器的历史残差，而预测器本身在运行的最初几天并不完整：负载内核依赖“同星期几”的历史，而 1 月 1–7 日不足一周，此时只能使用附件 1 的典型日曲线作回退预测。因此 $r_{s,t}$ 的构成在时间上分为两段：

阶段	负载预测内核	残差性质
1 月 1–7 日	附件 1 典型日回退	与后续口径不同，混入会污染残差分布
1 月 8 日起	同星期几均值 + 漂移校正	与 2–12 月运行口径一致

即：本节要修正的对象是“场景样本的残差口径”，不是预测方法、也不是窗口规则。回退期残差本身是合法的因果预测误差（并未使用未来数据），但它对应的预测器与后续 334 天的运行不同，故不适合作为场景样本。因此把场景历史的起点设为 $d \geq 7$ （即 1 月 8 日）——该边界由预测器可用性决定，而不是通过遍历全年费用挑选截断日：

Table 4.2 有效残差的样本量增长

日期	可用历史残差天数	说明
2 月 1 日	24	1 月 8–31 日
⋮	⋮	逐日增加
3 月初起	56（上限）	达到窗口上限后滚动

该修正与窗口方案的区分：残差口径与窗口规则是两个独立维度：前者决定“用哪些历史日作场景样本”，后者决定“取多少天、如何加权”。下表把两者交叉列出，以分离各自的作用：

Table 4.3 窗口与加权方案的对照实验（第二问，共两组口径）

方法	总费用（元）	较同组基线（元）	紧急电量（kWh）
第一组：含 1 月 1–7 日回退残差（原样口径）			
56 天等权（基线）	14401996.98	—	237775.35
56 天 / 半衰期 7 天	14444376.04	−42379.06	258228.71
56 天 / 半衰期 14 天	14404271.57	−2274.59	244460.92
56 天 / 半衰期 28 天	14391380.10	+10616.87	237997.14
28 天等权	14443538.21	−41541.23	250845.99
42 天等权	14407394.33	−5397.35	241795.11
第二组：剔除回退残差（有效残差口径，最终采用）			
56 天等权（基线）	14389135.08	—	246063.25
56 天 / 半衰期 28 天	14390031.13	+896.04	243038.84
在线四选一（每周重选窗口）	14399216.51	+10081.43	248532.83

结论：在两组口径下，短窗口与指数加权均无稳定收益（多数反而变差）。第一组中 28 天半衰期看似改善 10616.87 元，但该改善主要来自 2 月（−12638.73 元），

3-12 月合计反而增加 2021.86 元，说明它捕获的是启动阶段效应而非全年适应能力。剔除回退残差（第二组）后，该优势消失（+896.04 元），而剔除回退残差本身带来 12861.89 元改善（第二组基线 vs 第一组基线）。故最终采用“有效残差 + 56 天等权”。

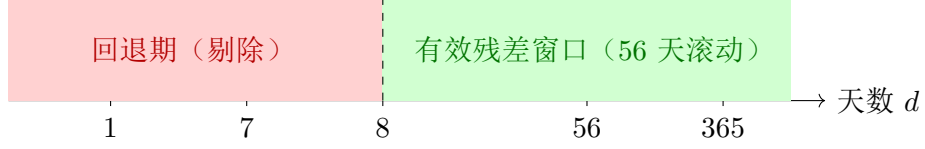


Figure 4.2 有效残差的时间口径：剔除 1 月 1-7 日的回退预测残差

4.5 滚动执行与终端条件

每天在 48 小时窗口上求解，但只执行前 24 小时（当天 144 段），次日重新求解。终端条件为窗口末回到窗口起始电量：

$$E_{288} = E_{\text{窗口起始}}, \quad (4.9)$$

而非每天日末回到日初。原因：滚动优化中终端条件代表“未来价值”，若要求每天回到日初，会人为禁止跨日套利，系统性低估储能价值。

实际执行的紧急购电量为

$$e_{d,t} = \max\{N_{d,t} - g_{d,t} - d_{d,t} + c_{d,t}, 0\}, \quad (4.10)$$

实际费用为

$$C = \sum_d \sum_t [p_t g_{d,t} + 5p_t e_{d,t}]. \quad (4.11)$$

4.6 结果

Table 4.4 问题二最终结果（2 月 1 日-12 月 31 日，334 天）

项目	金额（元）
计划购电费	13462061.30
紧急购电费	927073.78
总费用	14389135.08

全年计划购电量 22 098 240.82 kWh，紧急购电量 246 063.25 kWh。2 月初储电量为 8550 kWh（1 月纳入同一滚动调度后的落点，见 §1.5），年末回到 6000 kWh。

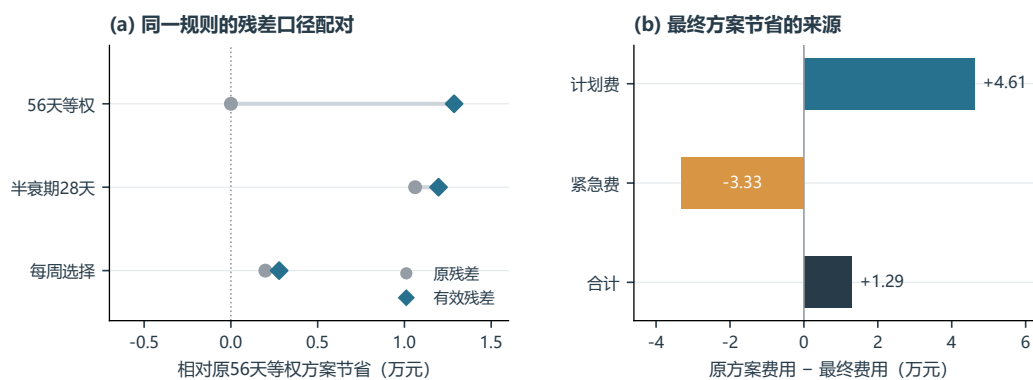


Figure 4.3 残差口径的配对比较（图 4.3 上半）。(a) 连线连接同一窗口规则在两种残差口径下的年度费用点，不是置信区间；(b) 分解计划费与紧急费的节省。

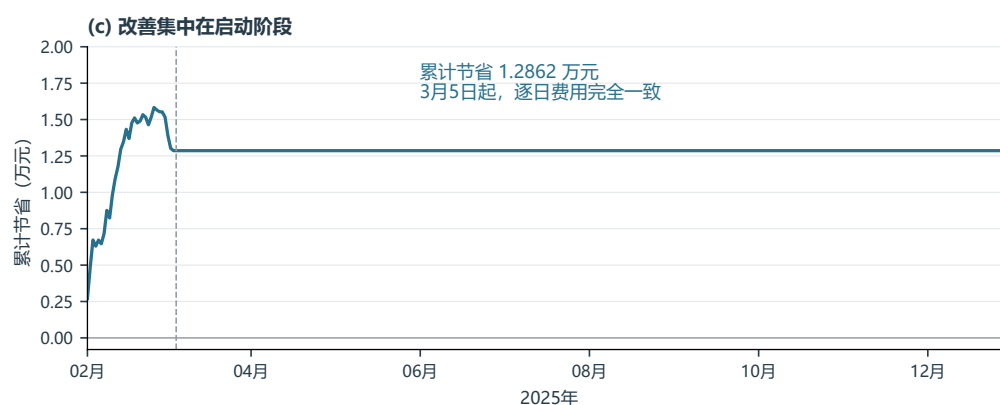


Figure 4.4 最终方案相对原方案的累计费用节省（图 4.3 下半）。(c) 展示全年累计节省，改善集中在启动阶段，3 月 5 日起逐日费用与原方案一致。

5 问题三：预报融合与场景树多时点调整

5.1 问题分析

问题三在问题二基础上引入两项新要素：

1. **多时点预报**：每天 0、6、12、18 时各发布一次未来 24 h 的光伏整点预报，后续时刻的预报可用于**调整购电策略**；
2. **调整计费**：调整不是免费的——计划量高于调整量的部分按 50% 违约电价、调整量高于计划量的部分按 1.5 倍电价。

因此模型需要回答两个层次的问题：(i) 如何在 0 点制定“预留调整余地”的计划；(ii) 如何在后续发布时刻利用新信息最优地修正计划：

5.2 计费规则的统一化建模

设第 t 段的 0 点原计划量为 g_t^0 ，最终调整后的购电量为 q_t 。定义调增量与调减量

$$u_t = (q_t - g_t^0)_+, \quad v_t = (g_t^0 - q_t)_+, \quad (5.1)$$

则 $q_t = g_t^0 + u_t - v_t$ 。题面规则可统一写为

$$J = \sum_t \left[\underbrace{p_t g_t^0}_{\text{原计划费}} + \underbrace{1.5 p_t u_t}_{\text{调增, 1.5 倍}} - \underbrace{p_t v_t}_{\text{退还原价}} + \underbrace{0.5 p_t v_t}_{\text{违约费, 50\%}} + \underbrace{5 p_t e_t}_{\text{紧急购电}} \right]. \quad (5.2)$$

即**净调整费**为 $1.5 p_t u_t - 0.5 p_t v_t$ ，它可以为负。合并同类项得另一便于独立核验的等价式：

$$J = \sum_t \left[p_t \min(g_t^0, q_t) + 0.5 p_t v_t + 1.5 p_t u_t + 5 p_t e_t \right]. \quad (5.3)$$

5.3 在线预报融合

5.3.1 融合权重

设历史预测为 H 、发布预报为 F 、实际值为 V 。只用最近 56 天内已完全实现的预报行，以最小二乘拟合权重：

$$\alpha = \text{clip}_{[0,1]} \frac{\sum (F - H)(V - H)}{\sum (F - H)^2}, \quad \text{融合值} = \alpha F + (1 - \alpha)H. \quad (5.4)$$

式 (5.4) 的推导：以融合值与实际值的平方误差为目标 $\min_{\alpha} \sum [V - (\alpha F + (1 - \alpha)H)]^2$ 。记 $z = F - H$ ，则中括号内为 $(V - H) - \alpha z$ ，目标化为 $f(\alpha) = \sum [(V - H) - \alpha z]^2$ 。对 α 求导并置零：

$$\frac{df}{d\alpha} = -2 \sum z [(V - H) - \alpha z] = 0 \implies \alpha = \frac{\sum z (V - H)}{\sum z^2}. \quad (5.5)$$

又因二阶导 $d^2 f / d\alpha^2 = 2 \sum z^2 > 0$ ，该驻点即全局极小点；再叠加 $[0, 1]$ 截断以避免外推。几何上， α 是 $V - H$ 在 $F - H$ 上的投影系数。

收缩偏差校正：进一步叠加历史平均残差的收缩估计

$$\text{偏差} = \overline{(V - \text{融合})} \cdot \frac{n}{n + 14}, \quad (5.6)$$

其中 n 为有效样本数。系数 $n/(n + 14)$ 用于抑制小样本下偏差估计的波动：样本充足时接近 1，样本稀少时被压缩向 0。

5.3.2 发布预报的时间对齐

附件 3 的“预报 k 小时”是发布时刻后第 k 个整点的瞬时功率，而非第 k 个小时区间的均值。故采用如下重建方式：

1. 以 24 个整点为锚点，在 10 分钟网格上**线性插值**；
2. 发布瞬间的端点使用**已观测**的实际光伏功率（0 点用前一日末观测值）。

为何不能把整点值当作“小时均值”：设真实光伏功率为正弦形的日间过程，其峰谷差为 A 、周期 $T_0 = 24$ h。若把第 k 个整点的值 $v(k)$ 当作区间 $[k, k + 1)$ 的均值作分段常数填充，相当于用左端点值代替中点值，引入的按时段均值计的系统偏差为

$$\bar{\varepsilon} = \frac{1}{T_0} \int_0^{T_0} [v(k) - v(t)] dt \sim \frac{1}{2} \cdot \frac{dv}{dt} \cdot 1 \text{ h}, \quad (5.7)$$

即在梯度最大处（日出、日落）偏差最大，量级为**半小时**的光伏变化量，故称“1/2 小时量级的系统相位误差”。线性插值则以相邻两整点为端点，把该项偏差从 $O(\Delta)$ 降到 $O(\Delta^2)$ ，且在每个整点上无偏。

实证核对：以附件 2 的实际光伏为基准，用两种方式重建同一预报行的 144 段并逐段比较（除去最后一天以避免跨年缺口），结果见表 5.1。

Table 5.1 两种整点重建方式的全年逐段 MAE（kW，365 天平均）

发布时间	分段常数填充	线性插值（本文）	降幅
0 点	440.64	225.99	−48.7%
6 点	405.15	167.39	−58.7%
12 点	453.61	238.10	−47.5%
18 点	488.43	292.97	−40.0%

线性插值在四个发布时间上把逐段 MAE 降低 40.0%–58.7%，与式 (5.7) 的机理一致：分段常数填充在日出、日落的高梯度处偏差最大，而线性插值恰在这些位置把误差从 $O(\Delta)$ 降到 $O(\Delta^2)$ 。这说明**时间对齐方式本身是预报误差的一个可测量来源**，而非实现细节。需要说明的是，表 5.1 度量的是“给定整点预报下的重建误差”，而图 5.3 展示的是融合后的综合预报精度，两者口径不同、不可直接相减。

5.4 经验场景树

5.4.1 信息限制：非前视约束（本问的建模前提）

问题二已要求“预测只能用历史数据”，本问把它**进一步强化**到“场景树的节点划分也只能依据届时已发布的信息”，原因是本问新增了多个决策时点，若不限制就可能在 6 点“提前知道”12 点才会发布的信息。具体为三条：

- 训练样本仅取不晚于 $d - 2$ 天发布的预报行，以确保跨午夜的未来 24 h 已经实现；
- 每个发布时刻分别训练，不得利用当天后续发布的预报来估计当前权重；
- 样本少于 7 条时退化为直接使用发布预报。

上述约束在模型中的实现方式是“节点内共享变量”，见 §5.4.3；它比额外添加约束更稳健，因为物理上不可能针对已知场景分别决策。

5.4.2 场景构造

在第 a 次发布时刻，场景净负载为

$$N_{s,t} = \hat{N}_t^{(a)} + \left(N_{r_s,t} - \hat{N}_{r_s,t}^{(a)} \right), \quad (5.8)$$

即用历史日 r_s 在**同一发布时刻**的残差去修正当前的发布预报。

5.4.3 信息分支（场景树）

仅用点预测场景会丢失一个重要信息：不同历史日不仅能告诉我们“缺口可能多大”，还能告诉我们“预报会如何变化”。为此把场景组织为树：

1. 历史日 r_s 除净负载残差外，还附带它在**后续发布时刻**的预报修订向量；
2. 把这些修订作为“未来可能观测到的信息”，用**二叉聚类**（按方差最大的维度二分）组织成树；
3. **关键的非前视要求**：每个节点只能依据**届时新发布的预报修订**划分其子节点，不得使用尚未实现的未来负载或实际光伏；
4. 节点概率 $\pi_n = \text{落入该节点的历史日数} / \text{总样本数}$ ；
5. 树最多 4 层、15 个决策节点；**全部 56 条残差样本仍参与期望缺口成本**，未把场景简化为少数均值曲线。

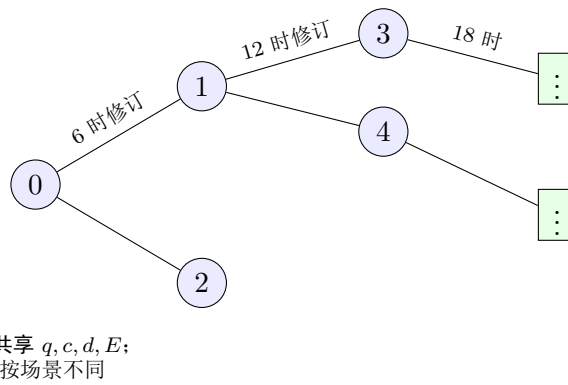


Figure 5.1 经验场景树结构（示意）：顺着“新发布的预报修订”逐层分支，节点内共享控制变量

为何“节点内共享变量”能保证非前视：因为控制变量 q, c, d, E 是**同一个变量**跨越该节点内的所有场景，物理上不可能针对“已经知道是哪个场景”来分别决策——这在变量结构层面（而非仅通过额外约束）排除了前视。

5.5 场景树随机规划模型

以节点 n 为单位，其概率为 π_n ，控制变量为 $q_{n,t}, c_{n,t}, d_{n,t}, E_{n,t}$ ：

$$\min J_3 = \sum_t p_t g_t^0 + \sum_n \pi_n \sum_{t \in n} (1.5 p_t u_{n,t} - 0.5 p_t v_{n,t}) + \frac{1}{S} \sum_s \sum_t 5 p_t e_{s,t} + \underbrace{\text{次日前瞻成本}}_{\text{价值近似}}. \quad (5.9)$$

约束：

$$(\text{调整关系}) \quad q_{n,t} = g_t^0 + u_{n,t} - v_{n,t}, \quad q, u, v \geq 0, \quad (5.10)$$

$$(\text{节点需求}) \quad q_{n,t} + d_{n,t} - c_{n,t} + e_{s,t} \geq N_{s,t}, \quad e_{s,t} \geq 0, \quad (5.11)$$

$$(\text{计划下界}) \quad q_{n,t} + d_{n,t} - c_{n,t} \geq \hat{N}_t, \quad (5.12)$$

$$(\text{SOC 与容量}) \quad E_{n,t} = E_{n,t-1} + 0.9 c_{n,t} - \frac{d_{n,t}}{0.9}, \quad 1200 \leq E_{n,t} \leq 10800, \quad (5.13)$$

$$(\text{功率}) \quad 0 \leq c_{n,t}, d_{n,t} \leq \frac{5000}{6}, \quad (5.14)$$

$$(\text{父子衔接}) \quad E_{n,\text{首}} = E_{\text{父},\text{末}}. \quad (5.15)$$

5.6 跨日价值与贪心执行

跨日前瞻：每次优化至次日 24 点，把次日作为确定性价值近似，避免“只顾当天便宜电价而提前耗空电池”。不同未来节点使用其条件预报修订后的次日点预测。

贪心执行：只执行当前发布到下一允许更新前的控制量：

决策时刻	本次执行区间
0:00	0–6 时
6:00	6–12 时
12:00	12–18 时
18:00	18–24 时

为何不把问题二的 80% 分位直接套用到本问：问题二的 $F^* = 0.80$ 是在“储能固定、单期补救费为 5 倍、无后续调整”条件下导出的单期临界分位；而问题三中 0 点可从场景树评估未来的调整机会，今天的“少买”可在 6/12/18 点以 1.5 倍价补上，故有效风险水平随节点、时段而变，不应硬编码单一分位。这正是不用报童模型而用场景树多阶段模型的原因。（同时充放电的处理与问题一相同：由 §3.4 的等 SOC 扰动自动消去，程序对最终执行结果逐段检查互斥。）

5.7 结果与边际分析

Table 5.2 问题三：更新频率对照与逐级边际收益（2月1日-12月31日）

方案	总费用（元）	紧急电量（kWh）	增量节省（元）
方案 B 框架同口径复算	14389892.50	246099.33	—
仅 0 点融合预报	14181445.03	218589.30	208447.47
0、6 点	14101371.47	186570.89	80073.56
0、6、12 点	14022358.10	175344.39	79013.36
0、6、12、18 点（主方案）	14022396.98	175369.95	-38.88

Table 5.3 问题三主方案费用分解

项目	金额（元）
原计划购电费	13345819.30
增购费（1.5 倍）	89947.28
取消原费退款	-150170.13
取消违约费（50%）	75085.07
净调整费（小计）	14862.22
紧急购电费（5 倍）	661715.46
总费用	14022396.98

相对同口径方案 B，主方案节省 367 495.52 元（2.55%），紧急购电量减少 28.74%。
结论：6 点与 12 点各贡献约 8 万元，收益随调整次数**显著边际递减**；18 点更新在本年度回测中**无可观测收益**（-38.88 元），可视为可选更新。该结论限于本数据与算法：18 点仍可能通过次日预报影响跨日储能，且小型经验树在有限样本下重建，不保证费用随信息增加单调下降。

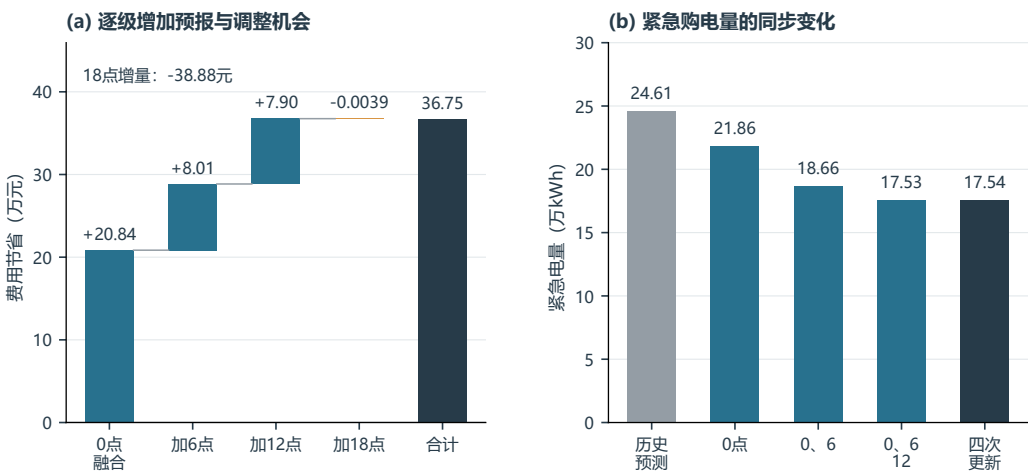


Figure 5.2 固定电价情形下的逐级费用节省与紧急购电量。费用节省以本问同口径历史预测对照为起点，依次引入 0 点融合预报和 6、12、18 点调整；右图按相同顺序比较紧急购电量。

图 5.2 把“信息更新的价值”与“更新次数的增加”分开呈现：仅引入 0 点融合预报即节省 208 447.47 元；随后增加 6 点与 12 点更新各节省约 8 万元；增加 18 点

后费用反而上升 38.88 元。这说明预报融合本身的价值最大，而更新次数存在明显边际递减。

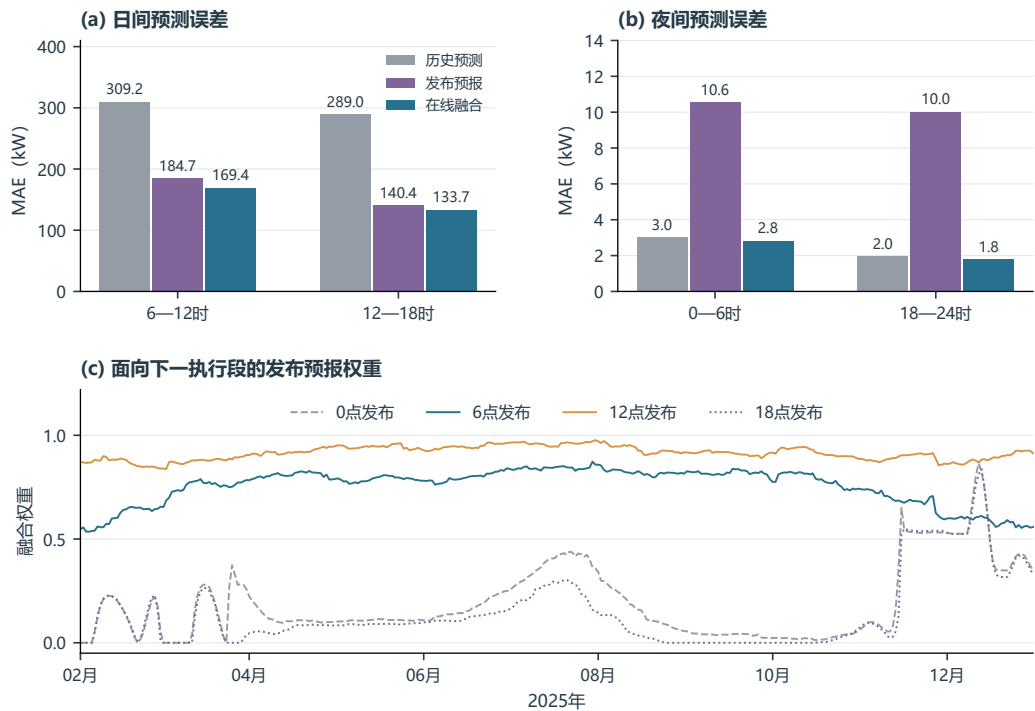


Figure 5.3 发布后六小时光伏预测误差及融合权重。(a)(b) 分别比较日间和夜间的历史预测、发布预报及在线融合 MAE，两面板纵轴尺度不同；(c) 展示各发布时间对应的发布预报权重。

融合预报的精度提升集中在日间：6—12 时段 MAE 由 309.24 kW 降至 169.41 kW（降幅约 45%），12—18 时段由 288.99 kW 降至 133.68 kW（降幅约 54%）。这正是光伏出力大、预报误差绝对值高的时段，也是购电决策对误差最敏感的时段，因此融合预报的主要价值来自在误差最大的时段压降误差。夜间绝对误差基数小，改善空间有限，图中单独展示仅为保留其变化尺度，并不改变上述判断。进一步看，融合权重随历史样本的成熟而更新、并叠加偏差校正，说明融合输出并非两条预报的固定加权平均，而是随数据逐步修正的动态估计。需要强调，**MAE 的下降并不能机械地换算成等比例的费用下降**：前者是统计精度指标，后者还取决于误差落在哪些时段、以及储能能否跨时段吸收这些偏差，故精度结论须与图 5.2 的费用对照结合判断。

图 5.4 揭示了一个容易被忽略的事实：日内调整对最终购电量的修正幅度其实很小。若把调整后的曲线与原计划直接叠加，两条几乎重合的曲线会掩盖这一修正；只有以原计划为零基准，才能看清各执行区间内增购与减购的方向与量级。这说明问题三的收益并非来自大幅改计划，而是来自对原计划做小幅、及时的纠偏。储电量轨迹按跨日滚动衔接，指定日的首末电量并不要求相等，体现了“日末价值留给次日”的滚动视角。图中紧急购电标记仅用于指示发生时段，既不代表缺口大小，也不能仅凭它与储能状态的相邻关系推断因果——实际缺口的形成是计划、储能与预报误差共同作用的结果。

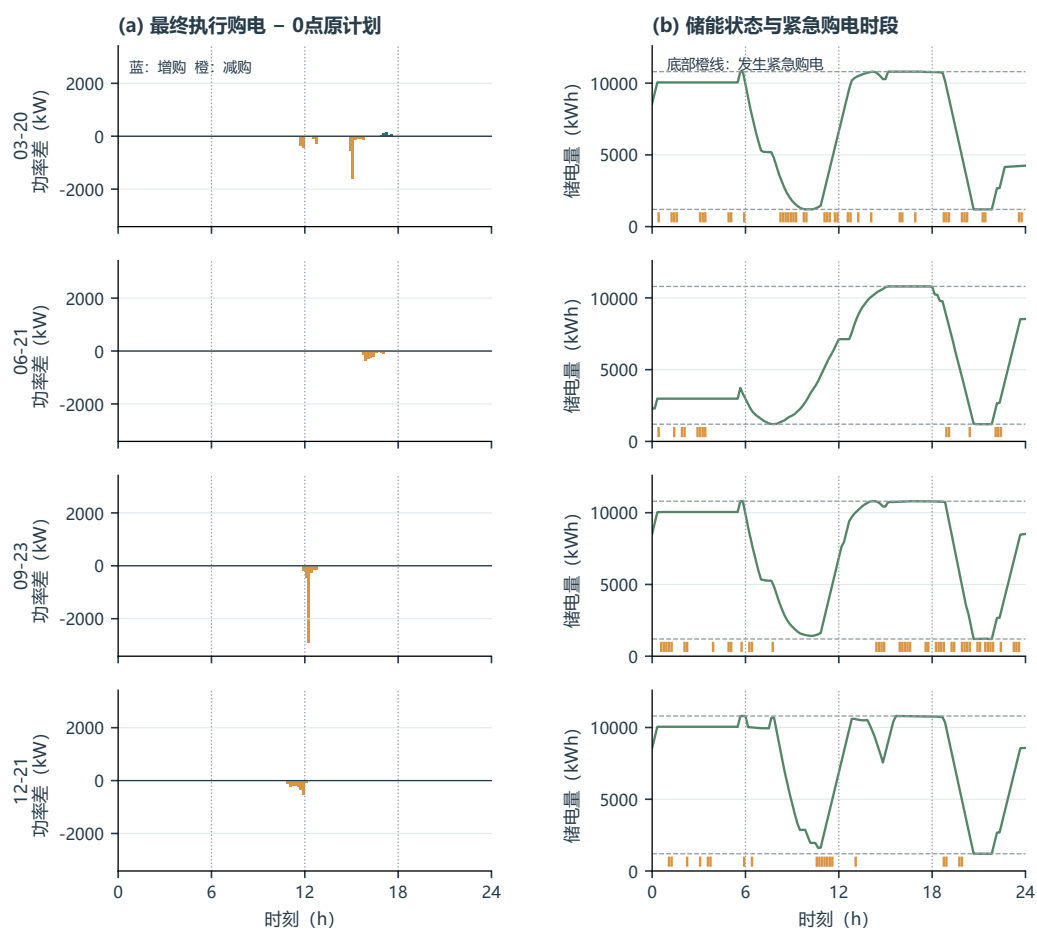


Figure 5.4 四个指定日的日内计划修正与储能状态。左列为最终执行购电相对 0 点原计划的功率差，正值为增购、负值为减购；右列为储能电量，底部橙色短线标记紧急购电大于数值容差的时段。竖虚线表示 6、12、18 点发布时刻。

5.8 敏感性分析与性能边界

完美预见下界：假设每天 0 点即精确知道当天实际净负载（消除全部预报误差），在同一物理约束与计费规则下求解得下界 1 223.16 万元（紧急购电费为 0），与主方案 1 402.24 万元相差 179.08 万元——该缺口全部来自预报误差，而非建模策略。

融合权重的饱和性：逐日自适应融合的加权 MAE 为 93.68 kW，优于含未来信息泄漏的静态最优线性组合（95.08 kW），说明权重估计已实质达到该模型族的精度上界。

参数扫描（30 余组）：对场景缩放、残差窗口、收缩参数、计划下界、多源信息等做同年度扫描，仅“场景缩放 $k = 0.92$ ”有正收益（约 0.47 万元，0.03%），其余全部无效或变差（窗口 $W = 28$ 时 +5.31 万元、计划下界 $0.5\times$ 时 +12.15 万元）。未把 $k = 0.92$ 写入正式结果——同年度扫描存在过拟合风险，收益量级可忽略。

注意到：（1）**弃置电量并非浪费**——全年弃置 267.66 万 kWh，但放松计划下界反而使总费用上升，原因是“多买的电”是“低价购电 + 储能套利”的必要冗余；（2）**增加信息源可能有害**——“上一期发布”虽合法可用，但纳入融合后费用上升 +0.66 万元，因多次发布高度相关，额外信息源主要引入估计噪声。

结论的适用范围：以上只能说明“在已测试模型族与同年度回测下未发现更优”，不足以宣称全局最优；引入外部气象数据仍有提升余地。

6 问题四：实时波动电价下的调度

6.1 问题分析与总体思路

问题四指出外网电价也是**实时波动**的，要求在此条件下重算问题二（记为 4-2）与问题三（记为 4-3）。与前三问的关系如下：

	电价	决策时点
问题二	每日固定（附件 1）	仅 0:00
4-2	实时波动（附件 4）	仅 0:00
问题三	每日固定	0/6/12/18 时
4-3	实时波动	0/6/12/18 时

因此 4-2 是“问题二 + 电价随机性”，4-3 是“问题三 + 电价随机性”。两问均需解决一个新问题：**电价本身也未知，如何构造决策所需的场景：**

6.2 4-2：波动电价下的日前调度

6.2.1 电价信息的假设（本问的关键判断）

题面说明电价实时波动，但未明确每天 0:00 是否已知全天价格。这个信息假设对结果影响很大，故我们保留两种情形并分别完整求解：

Table 6.1 4-2 的两种电价信息情形

情形	假设	性质
主模型	当天电价未知，只用此前日期预测	保守、符合“实时波动”的通常理解
对照	当天 144 段电价已于 0:00 公布	明确放宽信息限制

主模型不把当天或未来的实际电价、负载、光伏作为制定当天计划的输入；附件 4 的实际电价仅用于**事后结算**。

6.2.2 电价预测器选择

只用 1 月 8–31 日的 24 个历史预测日，比较四种规则：

1. 前一日曲线；
2. 近 7 日均值；
3. 近 7 日形状 + 前一日平均水平；
4. 最近 4 次同星期几均值。

以 1 月预测的 RMSE 选定，**2 月起固定方法不再更换**，避免用全年费用反向挑选参数。入选方法为**最近 4 次同星期几均值**：

$$\hat{p}_{d,t} = \frac{1}{|\mathcal{W}_d|} \sum_{i \in \mathcal{W}_d} p_{i,t}, \quad \mathcal{W}_d = \{d - 7j : j = 1, \dots, 4, d - 7j \geq 0\}. \quad (6.1)$$

预测 $d + 1$ 日时进一步要求所有样本日期 $i < d$ ，不得使用尚未发生的第 d 日价格。

6.2.3 电价—净负载联合场景

核心论证：期望的不可分解性：当电价随机时，

$$\mathbb{E}[p_t(N_t - z_t)_+] \neq \mathbb{E}[p_t] \cdot \mathbb{E}[(N_t - z_t)_+]. \quad (6.2)$$

不等式成立的原因是：**高电价可能与缺电同时发生**（例如极端天气既推高需求又推高价格）。若按式 (6.2) 右端建模（即“平均电价近似”），会系统性地**低估**缺电风险的实际代价。

因此场景取自**同一历史日**的两种残差配对：

$$N_{s,t} = \hat{N}_{d,t} + r_{s,t}^N, \quad p_{s,t} = \max\{0.001, \hat{p}_{d,t} + r_{s,t}^p\}, \quad (6.3)$$

其中 $r_{s,t}^N = N_{s,t} - \hat{N}_{d,t}$ 、 $r_{s,t}^p = p_{s,t} - \hat{p}_{d,t}$ 均由历史日 s 产生。0.001 元/kWh 是预先给定的正数下限，用于防止加性误差生成负电价，被截断的场景数逐日记录以便审查。

场景集为 $\mathcal{H}_d = \{\max(7, d - 56), \dots, d - 1\}$ ，等权 $\omega_s = 1/|\mathcal{H}_d|$ 。

6.2.4 目标函数

当天计划 g, c, d 在所有场景间共享，紧急购电 $u_{s,t}$ 可随场景变化：

$$\min J_{42} = \sum_{t=1}^{144} \bar{p}_{d,t} g_t + 5 \sum_s \omega_s \sum_{t=1}^{144} p_{s,t} u_{s,t} + \underbrace{\sum_{t=1}^{144} \hat{p}_{d+1|d,t} g_{144+t}}_{\text{次日前瞻价值（不进报表）}}, \quad (6.4)$$

其中 $\bar{p}_{d,t} = \sum_s \omega_s p_{s,t}$ 为场景平均电价，用于第一阶段购电费（计划量照付）。

风险覆盖的分位解释：固定某段的 c, d ，令 $z = g + d - c$ ，则其覆盖水平可用电价加权分位理解

$$\tilde{\omega}_{s,t} = \frac{\omega_s p_{s,t}}{\bar{p}_{d,t}}, \quad z \approx Q_{0.8}^{\tilde{\omega}}(N_{s,t}), \quad (6.5)$$

即高电价的场景会获得更高权重，从而抬高所需的供给水平。但该式只是启发式解释，实际须与储能跨时段约束一并求解，不能逐段套用。

6.2.5 约束与对照模型

约束沿用问题二：计划下界 (4.3)、场景兜底 (4.4)、SOC 与容量功率 (4.5)–(4.6)，以及 48 h 窗口终端条件。另外设置**平均电价对照**：保留完全相同的净负载场景，但把所有 $p_{s,t}$ 替换为 $\bar{p}_{d,t}$ ，以检验“配对关系”的实际作用。

6.2.6 结果

Table 6.2 4-2 结果对照（2 月 1 日–12 月 31 日）

方案	总费用（元）	较 Q2 重结算（元）	节省比例
原 Q2 计划按附件 4 重结算	15184704.23	—	—
平均电价模型	15160921.45	−23782.79	0.157%
联合场景模型（主）	15163043.89	−21660.34	0.143%
当日电价已知（对照）	15033043.18	−151661.05	0.999%

诚实说明：在同年回测中，平均电价近似**略优于**联合场景（差约 2122 元）。我们不因此改用平均电价模型，理由有二：(i) 联合场景保留了正确的概率结构（式 (6.2) 说明它的期望是准确的），(ii) 单一年度的 2122 元差异不具统计显著性，而结构性正确性可跨年成立。这一现象在结果文件中如实保留。

“当日电价已知”的增益：151 661.05 元（约 1.0%），说明**电价预测误差**是本问剩余成本的主要来源之一。

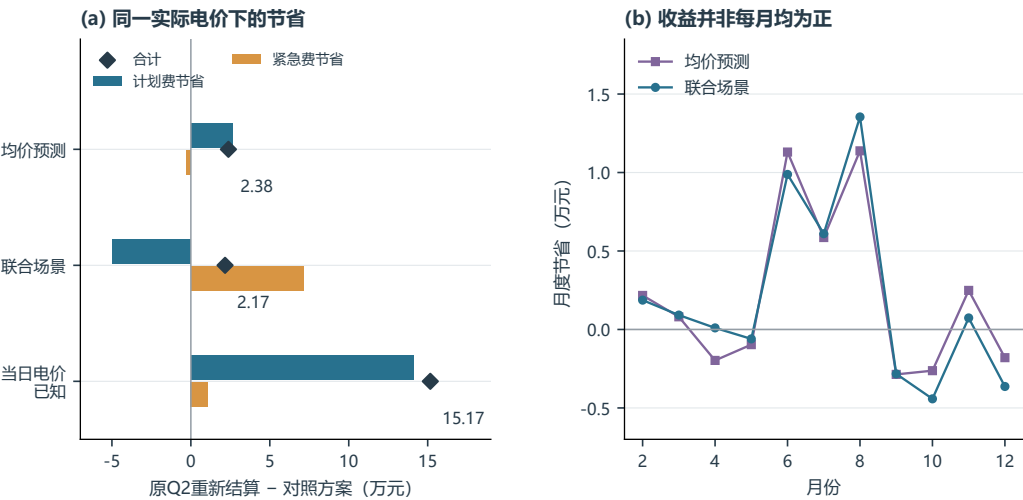


Figure 6.1 相对原第二问策略按附件 4 电价重新结算的费用节省。左图将节省分为计划费与紧急费，菱形表示合计，负值表示该项费用增加；右图展示均价预测与联合场景的月度净节省。当日电价为信息对照。

图 6.1 显示月度节省存在正负变化：年度改善不保证各月均改善，且均价预测方案在本年度略优（见上文说明）。该图同时说明“信息更充分”与“当月费用更低”不总是同步。

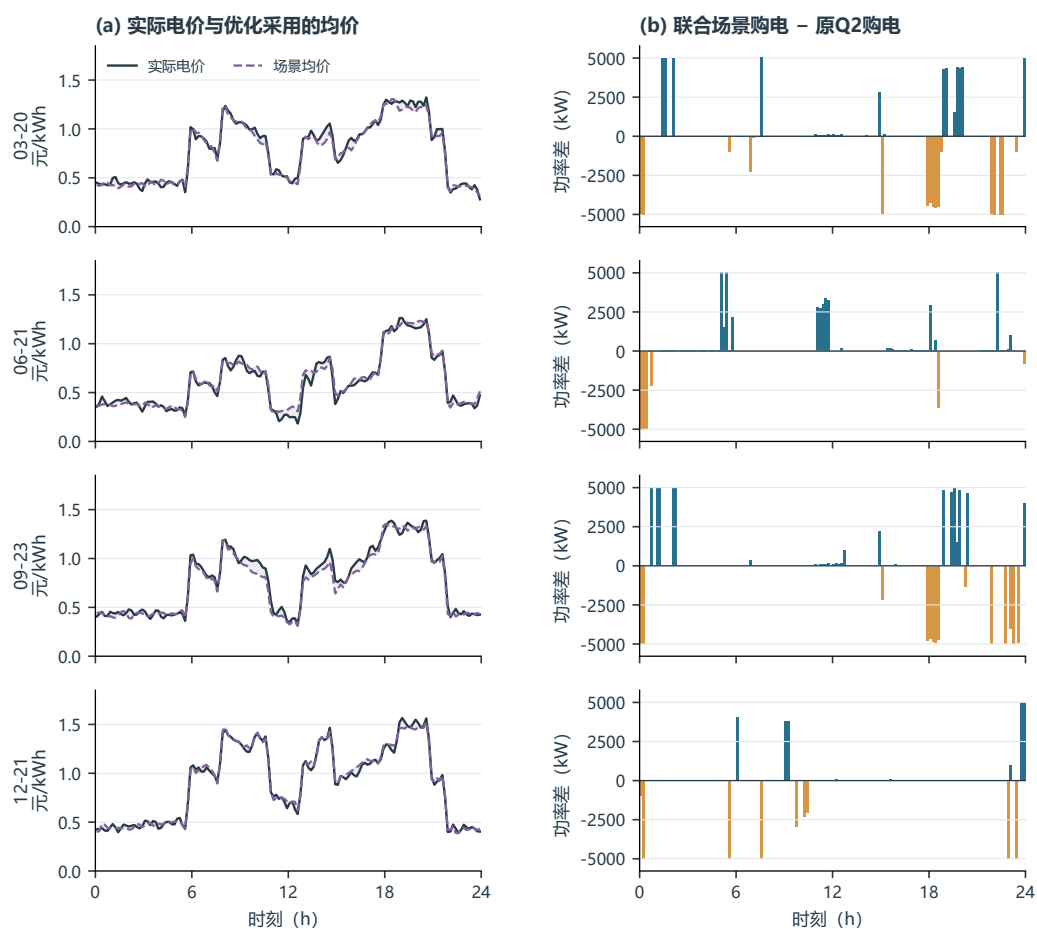


Figure 6.2 四个指定日的实际电价、优化采用的场景均价及购电计划差异。左列阴影仅表示两条价格曲线的差距；右列为联合场景计划购电功率减去原 Q2 计划购电功率，蓝色为增购、橙色为少购。

6.3 4-3：波动电价 + 多时点调整

6.3.1 模型构造

4-3 在 4-2 的基础上叠加问题三的多时点调整机制：每天 0 点定计划，6/12/18 点用新发布的光伏预报与更新的电价预测重新优化尚未执行的调度。计费规则（调增 1.5 倍、调减净成本 -0.5 倍、紧急 5 倍）与问题三完全一致，即沿用式 (5.2)。

电价场景的处理：每次发布时刻对未来电价采用此前 7 天同一时段均值，并与同一历史日的净负载残差配对构造价格场景（保留式 (6.3) 的相关性结构）。次日采用历史价格预测。

场景树、融合与执行：与问题三相同——使用经验场景树保证非前视，用在线融合整合历史预测与发布预报，只执行当前发布到下一更新前的控制量。

6.3.2 结果

Table 6.3 4-3 主方案费用分解

项目	金额（元）
原计划购电费	13457230.68
增购费（1.5 倍）	152250.38
取消原费退款	-173200.59
取消违约费（50%）	86600.30
净调整费（小计）	65650.08
紧急购电费（5 倍）	1538854.67
总费用	15061735.43

6.3.3 关键对比：调整机制价值的同口径分解

固定电价（问题三）与波动电价（4-3）两问都建立了逐级增量的对照方案，故可对两问做口径完全对称的分解。下表左半为各方案总费用（元），右半为由此得到的收益分解（万元）：

Table 6.4 两问逐级增量对照与同口径分解（结果期均 334 天）

方案	固定电价	波动电价	收益来源	固定	波动
无融合，仅 0 点（B）	14389892.50	15516984.54	在线预报融合	20.84	22.19
融合预报，仅 0 点	14181445.03	15295120.10	多时点调整	15.90	23.34
融合 + 0/6 点	14101371.47	15173094.46	加 6 点	8.01	12.20
融合 + 0/6/12 点	14022358.10	15068610.47	加 12 点	7.90	10.45
融合 + 0/6/12/18 点	14022396.98	15061735.43	加 18 点	-0.004	+0.69

“无融合 B 方案 → 主方案”的总改善可拆成融合预报与多时点调整两部分，两者之和恒等于总改善（固定电价 36.75 万、波动电价 45.52 万）。在完全同口径下，波动电价显著放大了多时点调整的价值：由 15.90 万元增至 23.34 万元（高约 47%），而融合预报的价值仅由 20.84 万元增至 22.19 万元。边际递减形态也印证了这一点：固定电价下第三次调整（加 18 点）已零收益（-38.88 元），波动电价下仍有 +6875 元——电价波动推迟了调整机制的饱和点，因为多时点更新不仅能修正光伏预报，还能修正电价预期。

4-2 → 4-3 的主方案差值为 101 308.46 元，但该差值横跨两问、同时含“预测源切换”与“引入调整机制”两种效应，不能作为调整机制的边际价值；上表的逐级增量才是同口径度量。两问主方案绝对费用仍相差约 104 万元，主要来自附件 4 的实时电价整体高于附件 1。

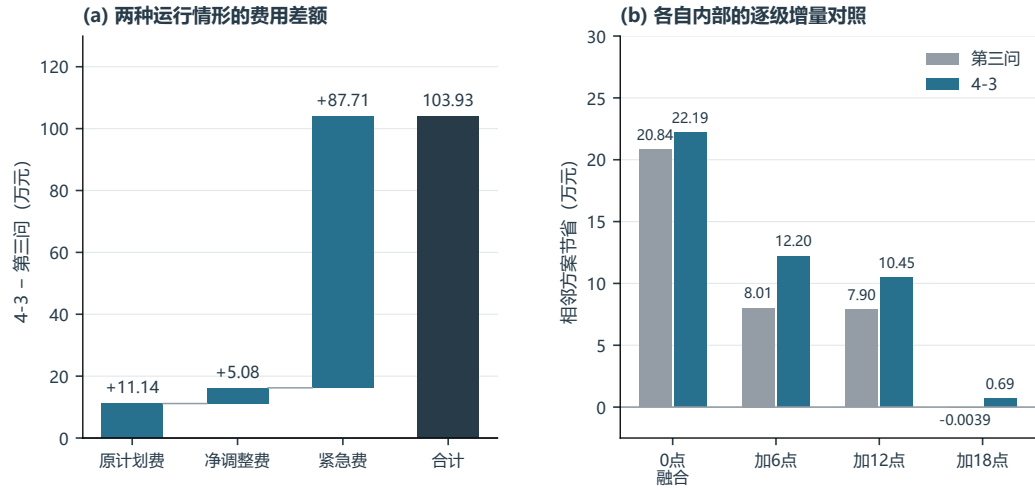


Figure 6.3 4-3 与第三问主方案的费用差额及更新收益比较。左图按原计划费、净调整费和紧急费分解 4-3 减第三问的总费用差；右图分别在两问内部计算相邻更新方案之间的费用节省。

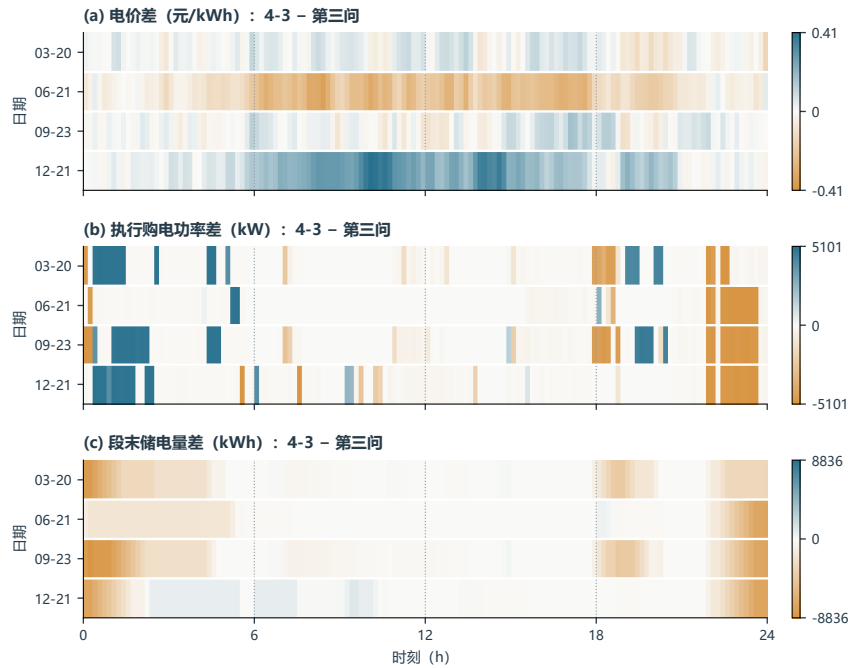


Figure 6.4 四个指定日中，4-3 相对第三问的电价、最终执行购电功率及段末储电量差值。蓝色表示 4-3 更高，橙色表示更低，白色表示接近零；各面板使用独立的对称色标，数值未裁剪，竖虚线为 6、12、18 点。

7 模型评价与推广

7.1 模型优点

本模型的优点在于**相对朴素方案有可量化的优越性**。下表先把各问的增益集中列出，再逐条说明其来源。

Table 7.1 本文方案相对朴素基准的可量化优越性

问题	对照的朴素方案	本文费用	相对节省
一	无储能（光伏优先、余电弃掉）	35126.95 元	26.90%
二	按点预测制定计划（50% 分位）	14389135.08 元	12 861.89 元
三	仅用历史预测、不做预报融合	14022396.98 元	2.55%
四-2	只按均价预测、忽略价格—缺口耦合	15163043.89 元	结构性正确

1. **识别出报童结构，把“安全裕度”变成“可推导的最优分位”**：由计费规则直接导出临界分位 $F^* = 4p/(p + 4p) = 0.80$ ，计划量应取净负载分布的 80% 分位而非均值，把参数设定从**试错**变为**推导**，也解释了为何“按点预测制定计划”会系统性低估购电量、产生大量紧急购电。
2. **预报融合精度越过“含信息泄漏的静态最优”上界**：逐日自适应的因果融合加权 MAE 为 **93.68 kW**，优于含未来信息泄漏的静态最优线性组合（95.08 kW），说明权重估计已实质达到该模型族的精度上界，也解释了为何后续所有“调权重”类尝试均无效果。
3. **多时点调整的价值被定量拆解，并给出“该在哪里停”的判据**：把 367 495.52 元总节省拆成“0 点融合 208 447.47 元、6 点 80 073.56 元、12 点 79 013.36 元、18 点 -38.88 元”，明确回答题面“是否需要引入其他时刻的预报”：**6 点与 12 点值得加，18 点在本年度无可观测收益**。
4. **LP 松弛紧性证明带来计算规模优势**：用等 SOC 扰动证明在往返效率 $0.81 < 1$ 且该时段有购电时，同时充放电必劣于拆开，故**最优解自动满足互斥、无需二元变量**；LP 与 MILP 数值结果完全一致（35 126.95 元，差 0.000000）。
5. **非前视由变量结构保证**：场景树中同一节点内所有场景共享同一组控制变量，使模型在结构上不可能“针对已知场景分别决策”，比仅添加约束更稳健，也便于审阅者直接验证。
6. **电价—净负载联合场景保留风险耦合**：由 $\mathbb{E}[p \cdot (N - z)_+] \neq \mathbb{E}[p]\mathbb{E}[(N - z)_+]$ 指出高电价可能与缺电同时发生，采用同历史日配对的场景构造，避免“平均电价近似”的结构性低估。
7. **严格的因果性与可复现性**：所有预测与融合权重只用决策时刻之前的数据；程序对 48 096 个时段逐段核验能量平衡、效率递推、SOC、功率、互斥、跨日连续、初末状态、缺口与费用，并反读 Excel 输出。

7.2 储能效率口径的稳健性检验

题附录 1 仅给出“充放电效率为 90%”，未指明是**单向**还是**往返**效率。本文主口径取单向各 90%（往返 0.81）；作为稳健性检验，另按往返 0.90 口径（对称拆分

$\eta_c = \eta_d = \sqrt{0.9} = 0.948683$) 重跑全部四问，定义与五组费用对照汇总于表 7.2。

Table 7.2 两种效率口径的定义与总费用对照

口径	含义	电量递推式	单向效率	往返效率
主口径	充、放各 90%	$E_t = E_{t-1} + 0.9c_t - d_t/0.9$	0.900000	0.81
对照口径	放电/充电 = 90%	$E_t = E_{t-1} + \eta c_t - d_t/\eta$	0.948683	0.90
结果集	主口径（元）	对照口径（元）	差额	变化率
问题一	35126.95	33801.50	-1325.45	-3.77%
问题二	14389135.08	13910066.83	-479068.25	-3.33%
问题三	14022396.98	13558297.17	-464099.81	-3.31%
问题四-2	15163043.89	14684028.55	-479015.34	-3.16%
问题四-3	15061735.43	14579714.65	-482020.78	-3.20%

差异机制与结论：往返效率由 0.81 提到 0.90，套利空间扩大；以问题一为例，光伏余电消纳项两口径完全相同（4037.75 元），差额**全部来自套利项**——口径只影响电能搬运效率，不改变余电消纳量。五组降幅高度一致（3.16%–3.77%），说明是效率口径本身的影响；主口径下的各项定性结论（多时点调整价值、18 点无额外收益、联合场景必要性）在对照口径下均**不变**。本文以**单向各 90%（往返 0.81）为主口径**，完整交付表见支撑材料。

7.3 模型局限

1. **非全局最优：**本文为有限经验场景上的线性规划加滚动执行，不宣称全年真实随机控制问题的全局最优解。小型二叉场景树、次日确定性近似、反复滚动重算都是对真实问题的近似。
2. **结果均为同年回测：**所有费用数字来自 2025 年 2–12 月的滚动回测，不证明在其他年份仍最优。例如问题三中“18 点更新无收益”是本年度数据结论。
3. **预测内核参数的选型过程未完全审计：**负载“最近 4 次同星期 + 近 5 日漂移”、光伏“近 7 日 + 近 28 日漂移”等参数继承自前期研发，本次因果性检查只能验证“运行时未使用未来数据”，不能证明参数选择未使用过测试年份信息。
4. **题面若干处的建模解释需公开声明：**问题三“取消退原价、另付 50% 违约金”是本文采用的结算解释，若改为“不退原价、另罚 50%”则费用会变化且应重新优化；储能效率的单向/往返解释同理 (§7.2)。
5. **储能模型简化：**未计入折旧、寿命衰减、温度与功率相关性等工程约束。
6. **场景样本量受限于历史长度：**2 月 1 日仅 24 条残差样本，场景分布尾部刻画精度有限。

7.4 模型推广

1. **引入外部气象数据。**光伏预报误差是主要成本来源（完美预见下界与现实差约 179 万元），引入辐照度、云量等特征可显著降低误差。
2. **自适应场景树深度。**当前固定 4 层、15 节点，可依样本量与预报修订方差自适应确定分支深度。

3. **电价预测的增强。**引入电价曲线形状聚类、天气耦合与节假日效应，改善 4-2/4-3 电价预测精度（该误差对应约 1.0% 费用）。
4. **储能全寿命周期成本建模。**加入循环次数与放电深度的折旧模型，避免“为套利而过度循环”的短视调度。
5. **多微网协同。**场景树框架可直接推广为多主体随机博弈结构。

8 AI 使用工具声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于模型可行性与复杂度咨询、优化方案讨论、数据可视化辅助及论文排版美化，详细使用情况见支撑材料。

参考文献

- [1] Porteus E L. Foundations of Stochastic Inventory Theory. Stanford University Press, 2002. (报童模型与临界分位)
- [2] Birge J R, Louveaux F. Introduction to Stochastic Programming. 2nd ed. Springer, 2011. (两阶段与多阶段随机规划)
- [3] Kall P, Wallace S W. Stochastic Programming. Wiley, 1994. (场景树、非前视约束与信息分支)
- [4] Dupačová J, Gröwe-Kuska N, Römisch W. Scenario reduction in stochastic programming: an approach using probability metrics. Mathematical Programming, 2003, 95(3): 493–511.
- [5] Dontchev A L, Hager W W, Veliov V M. Second-order Runge-Kutta approximations in control constrained optimal control. SIAM Journal on Numerical Analysis, 2000, 38(1): 202–226. (滚动时域优化与终端条件的作用)
- [6] Huangfu Q, Hall J A J. Parallelizing the dual revised simplex method. Mathematical Programming Computation, 2018, 10(1): 119–142. (HiGHS 求解器)
- [7] Virtanen P, Gommers R, Oliphant T E, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. Nature Methods, 2020, 17: 261–272.
- [8] 王成山, 武震, 李鹏. 微电网关键技术研究. 电工技术学报, 2014, 29(2): 1–12.
- [9] Antonanzas J, Osorio N, Escobar R, et al. Review of photovoltaic power forecasting. Solar Energy, 2016, 136: 78–111. (光伏预报方法与误差特性)
- [10] Weron R. Electricity price forecasting: a review of the state-of-the-art with a look into the future. International Journal of Forecasting, 2014, 30(4): 1030–1081.
- [11] Bates J M, Granger C W J. The combination of forecasts. Operational Research Quarterly, 1969, 20(4): 451–468. (预报融合与最优权重)

附录 A 支撑材料文件列表

本次提交包含两部分:论文正文(论文.pdf)与支撑材料文件夹 `final_version/支撑材料/` (压缩为单个文件提交)。下表给出支撑材料的目录结构与内容说明。所有源程序均为完整、可直接运行版本,运行环境见 § 附录 B。

Table 附录 A.1 支撑材料目录结构 (`final_version/支撑材料/`)

路径	内容	问题
一、源程序		
Q1/q1_code/	q1_model (LP/MILP 建模与核验)、run_q1 (主流程与结果表生成)	一
Q2/code/	rolling_window (残差与窗口)、run_experiment (滚动执行与结果表)、verify_window、check_information (核验)、report_experiment (汇总与说明生成)、make_target_tables (题目要求表)	二
Q3/code/	q3_data (校验)、q3_forecast (预报融合)、q3_model (场景树 LP)、run_q3 (滚动执行与结果表)、verify_q3 (核验)	三
Q4/part2/code/	q42_model (电价预测与联合场景)、run_q42 (主流程与结果表)、report_q42 (汇总与说明、指定日表生成)、verify_q42、check_information	四-2
Q4/part3/code/	问题三代码的实时电价扩展 (含 requirements.txt 依赖清单)	四-3
common/	q2_base/ (基础 LP 与数据层)、efficiency/ (附件 5 模板表头, 主程序读模板所必需)	全部
二、结果与图表		
Q1..Q4/*/results/	result*.xlsx 正式结果、summary.json 汇总、schedule_detail.csv 逐时段明细、verification.json 核验记录、figures/ 插图源文件	全部
Q2/results/ 子目录	variants/、extensions/、sensitivity/: 各候选窗口、短窗口扩展与敏感性分析对照	二
Q3/results/comparisons/	四种发布时点方案 (B_aligned、only_0、at_0_6、at_0_6_12) 结果	三
Q4/.../results/	各电价场景方案与指定日结果	四
三、AI 使用说明		
AI 使用说明/	AI 工具使用详情.pdf (7 页, 所用工具与模型、使用目的与环节、典型交互示例) 及同名 Markdown 源文件	一
四、数据		
data/附件/	附件 1--4.xlsx 原始数据; 附件 5/ 结果文件模板	全部

说明: `final_version/支撑材料/` 为自包含目录,解压后无需任何外部文件即可按 § 附录 B 的命令复现全部结果;附录 § 附录 E 的源程序直接取自该目录,与所提交文件逐字一致。

附录 B 运行环境与复现方法

Table 附录 B.1 软件环境

项目	版本 / 说明
操作系统	Windows
Python	3.12 / 3.13
数值计算	numpy、scipy (HiGHS 求解器)
数据处理	pandas、openpyxl
绘图	matplotlib
论文排版	XeLaTeX + ctex (中文支持)

复现步骤 (在 final_version/支撑材料/ 目录下执行):

```
python -m pip install -r Q4/part3/requirements.txt
```

问题一

```
python -X utf8 Q1/q1_code/run_q1.py
```

问题二 (推荐方案: 有效残差 + 56 天等权)

```
python -X utf8 Q2/code/run_experiment.py \
```

```
    --first-residual 7 --only uniform56 --out recommended_only
```

```
python -X utf8 Q2/code/make_target_tables.py      # 题目要求的表1/表2/表3
```

问题三 (四次更新主方案)

```
python -X utf8 Q3/code/run_q3.py --mode all --out reproduced
```

```
python -X utf8 Q3/code/verify_q3.py              # 独立核验
```

问题四

```
python -X utf8 Q4/part2/code/run_q42.py --primary joint_price
```

```
python -X utf8 Q4/part3/code/run_q3.py --mode all --out out
```

```
python -X utf8 Q4/part3/code/verify_q3.py        # 独立核验
```

说明:

- 各脚本内部路径均相对于文件位置, 从任意工作目录用脚本绝对路径启动均可。
- 问题二无参数默认运行的是原始残差口径候选, 不是最终推荐方案, 须按上述命令显式指定 `--first-residual 7`。
- 问题四-2 会读取 Q2/results/ 下的 Q2 结果作为同口径基准, 因此需先运行问题二。
- 运行主程序会覆盖同名结果文件; 论文使用的现有结果应先备份。

附录 C 核验记录摘要

Table 附录 C.1 逐时段核验项目（四问合计 48 096 段/年）

核验项	判据	结果
能量平衡	$g + V + d = L + c + w$ 残差 $< 10^{-9}$	通过
效率递推	$E_t - E_{t-1} - 0.9c_t + d_t/0.9 = 0$	通过
SOC 边界	$1200 \leq E_t \leq 10800$	通过
充放电功率	$0 \leq c_t, d_t \leq 833.33 \text{ kWh}$	通过
充放电互斥	$c_t d_t = 0$ （逐段检查）	通过
跨日连续性	相邻日末初电量一致（偏差 $< 10^{-9}$ ）	通过
初末条件	起始/终止电量符合设定	通过
实际缺口	$e_t = \max(N_t - z_t, 0)$	通过
费用复算	独立重算与程序输出一致	通过
Excel 反读	读回结果文件与内部数组一致	通过
非前视（信息扰动）	扰动未来数据不改变当前决策	通过

信息扰动测试的具体做法（以问题三为例）：在 2 月 1 日、6 月 21 日、12 月 21 日的四个发布时刻，修改当天及未来的全部实际值与未发布预报，重建预测、残差、融合权重、场景树与 LP，检查当前的预测结果与控制量是否保持不变。结果显示决策完全不变，证明程序未使用任何未来信息。

附录 D 题目指定时段与指定日期结果

本附录按题目表 1、表 2、表 3 的字段列示五组主结果。电量单位为 kWh，费用单位为元；十分钟时段按起止时刻定位，四小时充放电由连续 24 段累计，紧急购电按连续发生区间汇总。第三问与 4-3 同时列出 0 点原计划和最终调整后购电，实际总费用包括计划费、净调整费及紧急费。下列金额和电量由未舍入数据汇总后保留两位小数，舍入后的分项相加可能相差 0.01。

附录 D.1 问题一

Table 附录 D.1 问题一：指定时段购电及全天汇总（题表 1，电量单位：kWh）

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	480.41	14:00-14:10	0.00
16:00-16:10	445.43	18:00-18:10	531.89	20:00-20:10	0.00
全天购电量		59482.70	全天购电费（元）		35126.95

Table 附录 D.2 问题一：指定时段充放电及首末储电量（题表 2，单位：kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4500.00	0.00	04:00-08:00	833.33	6365.84
08:00-12:00	4787.96	1703.00	12:00-16:00	5286.04	91.10
16:00-20:00	0.00	5780.13	20:00-24:00	5333.33	2859.87
0:00 储电量		6000.00	24:00 储电量		6000.00

附录 D.2 问题二

Table 附录 D.3 问题二 2025-03-20: 指定时段购电及全天汇总 (题表 1, 电量单位: kWh)

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	626.63	14:00–14:10	0.00
16:00–16:10	494.24	18:00–18:10	706.68	20:00–20:10	0.00
全天计划购电量	65615.94	计划购电费 (元)	40493.15		
全天紧急购电量	689.80	紧急购电费 (元)	2775.85		
全天总购电费 (元)			43269.00		

Table 附录 D.4 问题二 2025-03-20: 指定时段充放电及首末储电量 (题表 2, 单位: kWh)

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	1666.67	0.00	04:00–08:00	833.33	5501.23
08:00–12:00	5912.26	3138.77	12:00–16:00	5454.26	566.88
16:00–20:00	0.00	5671.59	20:00–24:00	3498.06	2968.41
0:00 储电量		8550.00	24:00 储电量		4348.26

Table 附录 D.5 问题二 2025-06-21: 指定时段购电及全天汇总 (题表 1, 电量单位: kWh)

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	250.70	18:00–18:10	0.00	20:00–20:10	0.00
全天计划购电量	46473.41	计划购电费 (元)	25454.37		
全天紧急购电量	338.71	紧急购电费 (元)	1155.49		
全天总购电费 (元)			26609.86		

Table 附录 D.6 问题二 2025-06-21: 指定时段充放电及首末储电量 (题表 2, 单位: kWh)

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	2869.46	0.00	04:00–08:00	833.33	2615.32
08:00–12:00	5087.51	0.00	12:00–16:00	3470.92	0.00
16:00–20:00	0.00	6017.99	20:00–24:00	8166.67	2622.01
0:00 储电量		2670.80	24:00 储电量		8550.00

Table 附录 D.7 问题二 2025-09-23: 指定时段购电及全天汇总 (题表 1, 电量单位: kWh)

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	549.48	14:00–14:10	0.00
16:00–16:10	548.33	18:00–18:10	777.56	20:00–20:10	0.00
全天计划购电量	70464.22	计划购电费 (元)	43599.64		
全天紧急购电量	1179.52	紧急购电费 (元)	5122.78		
全天总购电费 (元)			48722.41		

Table 附录 D.8 问题二 2025-09-23: 指定时段充放电及首末储电量 (题表 2, 单位: kWh)

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	1666.67	0.00	04:00-08:00	833.33	5644.61
08:00-12:00	6191.30	2995.39	12:00-16:00	4841.85	296.85
16:00-20:00	0.00	5695.38	20:00-24:00	8166.67	2944.62
0:00 储电量		8550.00	24:00 储电量		8550.00

Table 附录 D.9 问题二 2025-12-21: 指定时段购电及全天汇总 (题表 1, 电量单位: kWh)

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	989.93	14:00-14:10	0.00
16:00-16:10	863.82	18:00-18:10	719.77	20:00-20:10	0.00
全天计划购电量	98935.42	计划购电费 (元)			63572.16
全天紧急购电量	559.54	紧急购电费 (元)			1933.80
全天总购电费 (元)					65505.96

Table 附录 D.10 问题二 2025-12-21: 指定时段充放电及首末储电量 (题表 2, 单位: kWh)

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	1666.67	0.00	04:00-08:00	1666.67	1508.33
08:00-12:00	5833.33	7806.67	12:00-16:00	8241.15	2760.33
16:00-20:00	0.00	5609.57	20:00-24:00	8166.67	3030.43
0:00 储电量		8550.00	24:00 储电量		8550.00

Table 附录 D.11 问题二: 指定日期全部紧急购电区间 (题表 3, 单位: kWh)

2025.03.20		2025.06.21		2025.09.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
00:20-00:30	12.39	00:20-00:30	40.44	00:30-01:20	22.17	01:00-01:20	29.05
01:10-01:40	37.48	01:20-01:30	21.10	02:00-02:20	55.41	02:10-02:20	0.61
03:00-03:30	70.17	01:50-02:10	13.24	03:50-04:00	11.95	03:00-03:10	15.38
04:50-05:10	48.10	02:50-03:30	99.52	04:50-05:10	34.28	03:30-03:50	108.42
05:50-06:00	7.85	18:50-19:10	78.17	05:40-05:50	10.72	05:50-06:00	8.04
08:50-09:10	26.14	20:20-20:30	15.05	06:20-06:30	1.78	06:10-06:30	41.11
11:10-11:20	8.27	22:00-22:30	71.19	10:00-10:20	13.05	10:30-11:30	271.92
12:30-12:50	26.69	—	—	10:50-11:00	5.98	15:20-15:30	13.09
15:30-16:20	208.66	—	—	14:30-15:20	272.38	18:40-19:00	8.15
16:30-17:20	61.84	—	—	15:50-16:50	147.40	19:40-20:10	63.76
18:40-19:10	73.67	—	—	17:30-17:40	16.92	—	—
20:00-20:20	41.73	—	—	18:10-18:50	111.43	—	—
21:10-21:30	42.68	—	—	19:10-19:30	45.49	—	—
23:30-23:50	24.14	—	—	19:50-20:30	140.64	—	—
—	—	—	—	20:50-21:10	106.59	—	—
—	—	—	—	21:20-22:00	58.24	—	—
—	—	—	—	22:20-22:30	57.72	—	—
—	—	—	—	23:10-23:40	67.37	—	—
合计	689.80	合计	338.71	合计	1179.52	合计	559.54

附录 D.3 问题三

Table 附录 D.12 问题三指定日期费用与紧急电量汇总

日期	原计划费（元）	净调整费（元）	紧急电量（kWh）	总费用（元）
2025-03-20	39999.75	-94.14	873.52	43286.36
2025-06-21	24123.60	-105.75	342.74	25202.45
2025-09-23	43523.27	-155.36	949.18	47566.49
2025-12-21	63209.93	-75.36	607.81	65090.39

四个指定日期的净调整费均为负，即调整机制在指定日期上均实现了“省违约金”的净收益。指定日的完整调度见前文图 5.4。

Table 附录 D.13 问题三 2025-03-20：指定时段购电及全天汇总（题表 1，电量单位：kWh）

时间段	购电量	时间段	购电量	时间段	购电量
0 点原计划购电量					
10:00-10:10	0.00	12:00-12:10	551.93	14:00-14:10	0.00
16:00-16:10	544.34	18:00-18:10	705.08	20:00-20:10	0.00
全天计划购电量		64668.95	计划购电费（元）		39999.75
最终调整后购电量（不含紧急购电）					
10:00-10:10	0.00	12:00-12:10	551.93	14:00-14:10	0.00
16:00-16:10	544.34	18:00-18:10	705.08	20:00-20:10	0.00
全天调整后购电量		64101.47	净调整费（元）		-94.14
全天紧急购电量		873.52	紧急购电费（元）		3380.75
全天总购电费（元）					43286.36

Table 附录 D.14 问题三 2025-03-20：指定时段充放电及首末储电量（题表 2，单位：kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	1629.40	0.00	04:00–08:00	833.33	5973.45
08:00–12:00	6049.15	2666.55	12:00–16:00	5195.34	468.04
16:00–20:00	2.26	5675.55	20:00–24:00	3395.02	2974.39
0:00 储电量		8583.54	24:00 储电量		4246.51

Table 附录 D.15 问题三 2025-06-21：指定时段购电及全天汇总（题表 1，电量单位：kWh）

时间段	购电量	时间段	购电量	时间段	购电量
0 点原计划购电量					
10:00-10:10	0.00	12:00-12:10	0.00	14:00-14:10	0.00
16:00-16:10	223.91	18:00-18:10	0.00	20:00-20:10	0.00
全天计划购电量		43570.65	计划购电费（元）		24123.60
最终调整后购电量（不含紧急购电）					
10:00-10:10	0.00	12:00-12:10	0.00	14:00-14:10	0.00
16:00-16:10	176.27	18:00-18:10	0.00	20:00-20:10	0.00
全天调整后购电量		43321.19	净调整费（元）		-105.75
全天紧急购电量		342.74	紧急购电费（元）		1184.60
全天总购电费（元）					25202.45

Table 附录 D.16 问题三 2025-06-21: 指定时段充放电及首末储电量 (题表 2, 单位: kWh)

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	762.83	0.00	04:00-08:00	857.88	2271.65
08:00-12:00	6553.33	0.00	12:00-16:00	4083.07	0.00
16:00-20:00	0.00	5995.04	20:00-24:00	8149.49	2645.72
0:00 储电量		2292.66	24:00 储电量		8533.70

Table 附录 D.17 问题三 2025-09-23: 指定时段购电及全天汇总 (题表 1, 电量单位: kWh)

时间段	购电量	时间段	购电量	时间段	购电量
0 点原计划购电量					
10:00-10:10	0.00	12:00-12:10	502.56	14:00-14:10	0.00
16:00-16:10	560.29	18:00-18:10	767.73	20:00-20:10	0.00
全天计划购电量	70219.95	计划购电费 (元)			43523.27
最终调整后购电量 (不含紧急购电)					
10:00-10:10	0.00	12:00-12:10	430.27	14:00-14:10	0.00
16:00-16:10	560.29	18:00-18:10	767.73	20:00-20:10	0.00
全天调整后购电量	69537.44	净调整费 (元)			-155.36
全天紧急购电量	949.18	紧急购电费 (元)			4198.58
全天总购电费 (元)					47566.49

Table 附录 D.18 问题三 2025-09-23: 指定时段充放电及首末储电量 (题表 2, 单位: kWh)

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	1682.24	0.00	04:00-08:00	833.33	5844.03
08:00-12:00	6049.67	2598.82	12:00-16:00	4754.75	331.14
16:00-20:00	42.16	5706.45	20:00-24:00	8152.71	2962.87
0:00 储电量		8535.99	24:00 储电量		8517.90

Table 附录 D.19 问题三 2025-12-21: 指定时段购电及全天汇总 (题表 1, 电量单位: kWh)

时间段	购电量	时间段	购电量	时间段	购电量
0 点原计划购电量					
10:00-10:10	0.00	12:00-12:10	965.84	14:00-14:10	0.00
16:00-16:10	872.53	18:00-18:10	708.71	20:00-20:10	0.00
全天计划购电量	98380.06	计划购电费 (元)			63209.93
最终调整后购电量 (不含紧急购电)					
10:00-10:10	0.00	12:00-12:10	954.33	14:00-14:10	0.00
16:00-16:10	872.53	18:00-18:10	708.71	20:00-20:10	0.00
全天调整后购电量	98056.59	净调整费 (元)			-75.36
全天紧急购电量	607.81	紧急购电费 (元)			1955.82
全天总购电费 (元)					65090.39

Table 附录 D.20 问题三 2025-12-21: 指定时段充放电及首末储电量 (题表 2, 单位: kWh)

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	1640.14	0.00	04:00-08:00	1660.82	1605.72
08:00-12:00	5884.07	7358.10	12:00-16:00	7766.93	2767.14
16:00-20:00	7.42	5602.70	20:00-24:00	8200.89	3043.38
0:00 储电量		8573.87	24:00 储电量		8576.96

Table 附录 D.21 问题三: 指定日期全部紧急购电区间 (题表 3, 单位: kWh)

2025.03.20		2025.06.21		2025.09.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
00:20-00:30	12.39	00:20-00:30	40.44	00:30-01:20	22.17	01:00-01:20	29.05
01:10-01:40	37.48	01:20-01:30	21.10	02:00-02:20	55.41	02:10-02:20	0.61
03:00-03:30	70.17	01:50-02:10	13.24	03:50-04:00	11.95	03:00-03:10	15.38
04:50-05:10	48.10	02:50-03:30	99.52	04:50-05:10	34.31	03:30-03:50	108.42
05:50-06:00	6.72	18:50-19:10	79.29	05:40-05:50	17.70	05:50-06:00	8.04
08:10-09:20	147.65	20:20-20:30	18.51	06:10-06:30	0.57	06:20-06:30	15.63
09:40-10:00	56.01	22:00-22:30	70.63	07:40-07:50	5.03	10:30-11:40	363.36
11:00-11:30	114.30	—	—	14:20-15:00	75.47	13:00-13:10	1.78
11:40-12:00	35.27	—	—	15:50-16:40	126.89	18:40-19:00	6.49
12:30-12:50	109.16	—	—	17:30-17:50	19.82	19:40-20:00	59.05
13:10-13:20	0.81	—	—	18:10-18:50	96.87	—	—
14:00-14:10	6.50	—	—	19:10-19:30	41.82	—	—
15:50-16:10	22.60	—	—	19:50-20:30	131.14	—	—
16:50-17:00	0.11	—	—	20:50-21:10	106.59	—	—
18:40-19:10	85.04	—	—	21:20-22:00	78.37	—	—
19:50-20:20	49.50	—	—	22:20-22:30	57.72	—	—
21:10-21:30	47.58	—	—	23:10-23:40	67.37	—	—
23:30-23:50	24.14	—	—	—	—	—	—
合计	873.52	合计	342.74	合计	949.18	合计	607.81

附录 D.4 问题四（4-2）

Table 附录 D.22 问题四（4-2）2025-03-20：指定时段购电及全天汇总（题表 1，电量单位：kWh）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	653.15	14:00–14:10	12.32
16:00–16:10	494.35	18:00–18:10	0.00	20:00–20:10	731.74
全天计划购电量		63984.26	计划购电费（元）		41035.65
全天紧急购电量		631.45	紧急购电费（元）		2706.13
全天总购电费（元）					43741.78

Table 附录 D.23 问题四（4-2）2025-03-20：指定时段充放电及首末储电量（题表 2，单位：kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	2500.00	0.00	04:00–08:00	1500.00	5904.30
08:00–12:00	5922.21	3144.53	12:00–16:00	5058.38	520.46
16:00–20:00	0.00	6405.44	20:00–24:00	833.33	2234.56
0:00 储电量		7950.00	24:00 储电量		1950.00

Table 附录 D.24 问题四（4-2）2025-06-21：指定时段购电及全天汇总（题表 1，电量单位：kWh）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	252.87	18:00–18:10	487.95	20:00–20:10	0.00
全天计划购电量		49515.72	计划购电费（元）		23064.36
全天紧急购电量		311.29	紧急购电费（元）		1037.19
全天总购电费（元）					24101.55

Table 附录 D.25 问题四（4-2）2025-06-21：指定时段充放电及首末储电量（题表 2，单位：kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	0.00	0.00	04:00–08:00	2749.81	2227.34
08:00–12:00	7535.00	0.00	12:00–16:00	3131.66	0.00
16:00–20:00	0.00	6017.99	20:00–24:00	9166.67	2757.01
0:00 储电量		1200.00	24:00 储电量		9300.00

Table 附录 D.26 问题四（4-2）2025-09-23：指定时段购电及全天汇总（题表 1，电量单位：kWh）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	568.09	14:00–14:10	0.00
16:00–16:10	550.06	18:00–18:10	0.00	20:00–20:10	0.00
全天计划购电量		70272.43	计划购电费（元）		45565.16
全天紧急购电量		1064.22	紧急购电费（元）		4972.92
全天总购电费（元）					50538.08

Table 附录 D.27 问题四（4-2）2025-09-23：指定时段充放电及首末储电量（题表 2，单位：kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	4166.67	0.00	04:00–08:00	833.33	5614.15
08:00–12:00	6150.98	3025.85	12:00–16:00	4882.17	296.85
16:00–20:00	0.00	6245.22	20:00–24:00	4833.33	2394.78
0:00 储电量		6300.00	24:00 储电量		5550.00

Table 附录 D.28 问题四（4-2）2025-12-21：指定时段购电及全天汇总（题表 1，电量单位：kWh）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	993.19	14:00–14:10	0.00
16:00–16:10	863.82	18:00–18:10	719.77	20:00–20:10	0.00
全天计划购电量	97387.54	计划购电费（元）			74053.41
全天紧急购电量	526.64	紧急购电费（元）			2403.13
全天总购电费（元）					76456.54

Table 附录 D.29 问题四（4-2）2025-12-21：指定时段充放电及首末储电量（题表 2，单位：kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	666.67	0.00	04:00–08:00	0.00	833.33
08:00–12:00	5833.33	7806.67	12:00–16:00	8256.22	2772.54
16:00–20:00	0.00	5609.30	20:00–24:00	8333.33	3030.70
0:00 储电量		10200.00	24:00 储电量		8700.00

Table 附录 D.30 问题四（4-2）：指定日期全部紧急购电区间（题表 3，单位：kWh）

2025.03.20		2025.06.21		2025.09.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
00:20–00:30	10.64	00:20–00:30	36.51	00:30–00:50	10.00	01:00–01:20	26.83
01:10–01:40	32.72	01:20–01:30	16.72	01:00–01:10	5.36	03:00–03:10	13.56
03:00–03:30	67.32	02:00–02:10	10.40	02:00–02:20	52.79	03:30–03:50	103.82
04:50–05:10	47.52	02:50–03:30	90.63	03:50–04:00	9.70	05:50–06:00	4.79
05:50–06:00	6.72	18:50–19:10	78.17	04:50–05:10	31.67	06:10–06:30	34.39
09:00–09:10	17.38	20:20–20:30	15.05	05:40–05:50	7.01	10:30–11:30	259.80
12:30–12:50	3.28	22:00–22:30	63.81	10:00–10:20	4.62	15:20–15:30	11.52
15:30–16:20	199.58	—	—	14:30–15:20	260.90	18:40–19:00	8.15
16:30–17:20	61.32	—	—	15:50–16:50	122.31	19:40–20:10	63.76
18:40–19:10	78.09	—	—	17:30–17:40	16.92	—	—
20:00–20:20	43.60	—	—	18:10–18:50	96.79	—	—
21:10–21:30	40.07	—	—	19:10–19:30	41.85	—	—
23:30–23:50	23.20	—	—	19:50–20:30	140.17	—	—
—	—	—	—	20:50–21:10	105.01	—	—
—	—	—	—	21:20–22:00	54.90	—	—
—	—	—	—	22:20–22:30	57.72	—	—
—	—	—	—	23:10–23:40	46.50	—	—
合计	631.45	合计	311.29	合计	1064.22	合计	526.64

附录 D.5 问题四（4-3）

Table 附录 D.31 问题四（4-3）2025-03-20：指定时段购电及全天汇总（题表 1，电量单位：kWh）

时间段	购电量	时间段	购电量	时间段	购电量
0 点原计划购电量					
10:00-10:10	0.00	12:00-12:10	545.41	14:00-14:10	0.00
16:00-16:10	525.49	18:00-18:10	0.00	20:00-20:10	719.50
全天计划购电量		67104.27	计划购电费（元）		41826.94
最终调整后购电量（不含紧急购电）					
10:00-10:10	0.00	12:00-12:10	545.41	14:00-14:10	0.00
16:00-16:10	525.49	18:00-18:10	0.00	20:00-20:10	719.50
全天调整后购电量		66322.18	净调整费（元）		-164.65
全天紧急购电量		1936.99	紧急购电费（元）		7681.69
全天总购电费（元）					49343.99

Table 附录 D.32 问题四（4-3）2025-03-20：指定时段充放电及首末储电量（题表 2，单位：kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	7500.00	0.00	04:00-08:00	3166.67	6274.00
08:00-12:00	5993.55	2366.00	12:00-16:00	5097.01	343.95
16:00-20:00	0.73	7170.36	20:00-24:00	65.10	1522.37
0:00 储电量		1200.00	24:00 储电量		1200.00

Table 附录 D.33 问题四（4-3）2025-06-21：指定时段购电及全天汇总（题表 1，电量单位：kWh）

时间段	购电量	时间段	购电量	时间段	购电量
0 点原计划购电量					
10:00-10:10	0.00	12:00-12:10	0.00	14:00-14:10	0.00
16:00-16:10	192.84	18:00-18:10	453.05	20:00-20:10	0.00
全天计划购电量		34666.48	计划购电费（元）		17225.37
最终调整后购电量（不含紧急购电）					
10:00-10:10	0.00	12:00-12:10	0.00	14:00-14:10	0.00
16:00-16:10	134.07	18:00-18:10	453.05	20:00-20:10	0.00
全天调整后购电量		34353.81	净调整费（元）		-93.39
全天紧急购电量		659.63	紧急购电费（元）		2033.32
全天总购电费（元）					19165.30

Table 附录 D.34 问题四（4-3）2025-06-21：指定时段充放电及首末储电量（题表 2，单位：kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	0.00	0.00	04:00-08:00	2586.28	2075.00
08:00-12:00	6553.33	0.00	12:00-16:00	4107.81	15.41
16:00-20:00	0.00	6096.96	20:00-24:00	53.86	2586.67
0:00 储电量		1200.00	24:00 储电量		1200.00

Table 附录 D.35 问题四（4-3）2025-09-23：指定时段购电及全天汇总（题表 1，电量单位：kWh）

时间段	购电量	时间段	购电量	时间段	购电量
0 点原计划购电量					
10:00-10:10	0.00	12:00-12:10	456.65	14:00-14:10	0.00
16:00-16:10	536.21	18:00-18:10	0.00	20:00-20:10	0.00
全天计划购电量	66826.12	计划购电费（元）	43138.23		
最终调整后购电量（不含紧急购电）					
10:00-10:10	0.00	12:00-12:10	306.14	14:00-14:10	0.00
16:00-16:10	536.21	18:00-18:10	0.00	20:00-20:10	0.00
全天调整后购电量	66054.20	净调整费（元）	-57.05		
全天紧急购电量	2121.28	紧急购电费（元）	9624.84		
全天总购电费（元）					52706.02

Table 附录 D.36 问题四（4-3）2025-09-23：指定时段充放电及首末储电量（题表 2，单位：kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	7333.33	0.00	04:00-08:00	3333.33	6441.96
08:00-12:00	6025.31	2198.04	12:00-16:00	4936.43	245.51
16:00-20:00	25.85	6307.45	20:00-24:00	16.57	2360.41
0:00 储电量		1200.00	24:00 储电量		1200.00

Table 附录 D.37 问题四（4-3）2025-12-21：指定时段购电及全天汇总（题表 1，电量单位：kWh）

时间段	购电量	时间段	购电量	时间段	购电量
0 点原计划购电量					
10:00-10:10	0.00	12:00-12:10	918.26	14:00-14:10	0.00
16:00-16:10	843.15	18:00-18:10	703.41	20:00-20:10	0.00
全天计划购电量		95873.43	计划购电费（元）		72035.15
最终调整后购电量（不含紧急购电）					
10:00-10:10	0.00	12:00-12:10	918.26	14:00-14:10	0.00
16:00-16:10	844.01	18:00-18:10	703.41	20:00-20:10	0.00
全天调整后购电量		95320.49	净调整费（元）		-205.31
全天紧急购电量		1295.96	紧急购电费（元）		5831.56
全天总购电费（元）					77661.40

Table 附录 D.38 问题四（4-3）2025-12-21：指定时段充放电及首末储电量（题表 2，单位：kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	10666.67	0.00	04:00-08:00	3.60	864.86
08:00-12:00	5833.33	7351.09	12:00-16:00	7885.43	2899.17
16:00-20:00	12.56	5655.47	20:00-24:00	44.62	3030.85
0:00 储电量		1200.00	24:00 储电量		1200.00

Table 附录 D.39 问题四 (4-3): 指定日期全部紧急购电区间 (题表 3, 单位: kWh)

2025.03.20		2025.06.21		2025.09.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
00:20-00:30	22.46	00:10-00:30	74.72	00:30-01:20	111.02	00:00-00:20	7.92
01:00-02:00	82.55	01:20-02:20	123.00	01:40-02:30	112.37	01:00-01:20	57.62
02:30-02:40	0.58	02:50-03:30	149.65	03:40-04:00	34.84	01:30-01:40	11.18
03:00-03:40	111.77	03:50-04:10	13.90	04:50-05:20	67.28	02:10-02:20	15.66
04:50-05:20	81.18	17:20-17:40	14.78	05:30-05:50	29.41	02:40-02:50	0.88
05:50-06:00	20.46	18:50-19:10	105.08	06:10-06:30	28.01	03:00-03:20	37.48
06:40-07:00	7.10	20:20-20:40	52.87	06:50-07:10	12.98	03:30-03:50	144.79
08:00-09:20	342.10	21:40-22:40	125.64	07:40-08:00	26.75	04:00-04:20	11.40
09:40-10:00	80.43	—	—	09:30-09:50	23.79	05:40-06:30	97.47
10:20-10:30	17.89	—	—	10:00-10:20	7.35	07:10-07:20	3.74
10:50-12:00	461.66	—	—	10:40-11:00	35.51	07:50-08:00	2.03
12:30-13:30	191.60	—	—	11:30-11:50	22.71	08:30-09:00	24.22
14:00-14:20	35.44	—	—	12:50-13:00	0.16	09:30-09:50	39.29
14:50-15:00	7.47	—	—	13:50-14:00	14.31	10:20-11:50	574.75
15:30-16:20	90.12	—	—	14:20-15:20	217.15	12:00-12:20	39.04
16:40-17:00	27.47	—	—	15:50-16:50	263.19	12:40-13:10	63.72
18:40-19:20	127.95	—	—	17:00-17:50	73.03	15:20-15:30	5.86
19:30-20:20	93.09	—	—	18:00-18:50	195.57	16:20-16:30	11.34
21:10-21:40	75.12	—	—	19:10-19:30	82.27	18:00-18:20	11.27
23:30-23:50	60.55	—	—	19:50-22:00	562.42	18:40-19:00	30.66
—	—	—	—	22:10-22:40	77.99	19:40-20:10	88.22
—	—	—	—	23:10-23:40	123.17	21:50-22:00	2.84
—	—	—	—	—	—	22:50-23:00	12.06
—	—	—	—	—	—	23:30-23:40	2.50
合计	1936.99	合计	659.63	合计	2121.28	合计	1295.96

附录 E 建模源程序 (完整可运行)

以下按问题列出生成论文全部数值结果所需源程序。这些代码直接取自支撑材料文件夹 **final_version/支撑材料/**, 因此与所提交文件逐字一致; 运行环境与复现命令见 § 附录 B。全部程序使用 Python 语言与 **numpy/scipy** (HiGHS 求解器), Excel 读写使用 **openpyxl**; 未使用 SPSS 等需手工交互的软件, 故无交互命令需要单独记录。

收录范围: 以上为支撑材料中全部源程序, 即生成论文全部数值结果与结果表所需的代码; 绘图代码已从支撑材料中移除, 故本附录与支撑材料文件清单完全一致。

附录 E.1 问题一: 确定性调度

q1_model.py

"""

C题 问题1: 确定性储能调度优化模型 (LP / MILP)

单位口径

- 附件1 中电价单位 元/kWh, 负载与光伏为功率 kW
- 每段 10 分钟, $dt = 1/6$ h, 功率 $\times dt =$ 该段电量 kWh
- 时间标签为"区间末": 0:10 表示 00:00-00:10, 0:00+1 表示 24:00

储能口径 (附录1)

- 最大容量 12000 kWh, 允许区间 [1200, 10800] kWh

```

- 最大充放电功率 5000 kW，作用于设备外端
- 充/放效率各 90%，往返 81%
- 2025-01-01 00:00 初始电量 6000 kWh
"""

import numpy as np
from scipy.optimize import linprog, milp, Bounds, LinearConstraint
from scipy.sparse import lil_matrix

# ----- 模型常量 -----
DT = 1 / 6          # 每段小时数
T = 144             # 时段数
E_MIN, E_MAX = 1200.0, 10800.0
P_MAX = 5000.0      # kW
CAP = P_MAX * DT    # 每段最大充/放电量 kWh = 833.33
ETA = 0.9           # 单向效率
E_INIT = 6000.0     # 0:00 电量，也是 24:00 必须回到的电量

# ----- 输入 -----
def load_attachment1(path):
    """读取附件1，返回 (labels, price, load_kWh, pv_kWh)。"""
    import openpyxl
    ws = openpyxl.load_workbook(path, data_only=True).worksheets[0]
    rows = list(ws.iter_rows(min_row=2, values_only=True))
    labels, price, load, pv = [], [], [], []
    for t, (time_tag, p, l, v) in enumerate(rows, start=1):
        labels.append(_label(t))
        price.append(float(p))
        load.append(float(l) * DT)      # kW -> kWh
        pv.append(float(v) * DT)       # kW -> kWh
    assert len(price) == T, f"期望 {T} 段，实际 {len(price)} 段"
    return labels, np.array(price), np.array(load), np.array(pv)

def _label(t):
    """第 t 段 (1-based) 的区间末时刻标签，t=144 -> '24:00'。"""
    m = t * 10
    return f"{m // 60:02d}:{m % 60:02d}"

# ----- LP 模型 -----
# 变量布局 x = [g(T), c(T), d(T), w(T), E(T)]，共 5T
def _slices():
    return (slice(0, T), slice(T, 2 * T), slice(2 * T, 3 * T),
            slice(3 * T, 4 * T), slice(4 * T, 5 * T))

def marginal_price(price, load, pv):
    """能量平衡约束的对偶变量 _t，即第 t 时段的系统边际电价（元/kWh）。

    _t 表示该时段多供 1 kWh 电量的边际成本，是解释"为何此时充/放"的经济学依据：

```

充电倾向落在 最低的时段，放电倾向落在 最高的时段。

```

"""
c_obj, A_eq, b_eq, bounds = build_lp(price, load, pv)
res = linprog(c_obj, A_eq=A_eq, b_eq=b_eq, bounds=bounds, method="highs")
assert res.status == 0, f"对偶变量求解失败: {res.message}"
return res.eqlin.marginals[:T]

def build_lp(price, load, pv):
    """构造 LP 的 (目标系数, 等式矩阵, 等式右端, 变量上下界)。

    solve_lp 与 marginal_price 共用, 保证两者约束完全一致——
    对偶变量只有在完整约束集下才有意义 (丢掉 SOC 递推会退化成本问题)。
    """
    G, C, D, W, E = _slices()
    n = 5 * T

    c_obj = np.zeros(n)
    c_obj[G] = price

    A_eq = lil_matrix((2 * T + 1, n))
    b_eq = np.zeros(2 * T + 1)

    # 能量平衡:  $g_t + V_t + d_t = L_t + c_t + w_t$ 
    for t in range(T):
        A_eq[t, G.start + t] = 1.0
        A_eq[t, D.start + t] = 1.0
        A_eq[t, C.start + t] = -1.0
        A_eq[t, W.start + t] = -1.0
        b_eq[t] = load[t] - pv[t]

    # 电量递推:  $E_t = E_{t-1} + c_t - d_t$ 
    for t in range(T):
        r = T + t
        A_eq[r, E.start + t] = 1.0
        if t > 0:
            A_eq[r, E.start + t - 1] = -1.0
        A_eq[r, C.start + t] = -ETA
        A_eq[r, D.start + t] = 1.0 / ETA
        b_eq[r] = E_INIT if t == 0 else 0.0

    # 日末电量回到日初:  $E_T = E_0$ 
    A_eq[2 * T, E.start + T - 1] = 1.0
    b_eq[2 * T] = E_INIT

    lb = np.zeros(n)
    ub = np.full(n, np.inf)
    lb[E], ub[E] = E_MIN, E_MAX
    ub[C] = CAP
    ub[D] = CAP
    # 弃光 w 只设下界 0, 不设上界 (与问题二/三的松弛变量口径统一)。
    # 说明: 平衡为等式  $g+V+d = L+c+w$   $w = g+d-c-(L-V)$ ;

```

```

# 目标只惩罚  $g$ ，故若最优解出现  $w > V$ （即净供给超过负载），
# 可令  $g \leftarrow g -$ 、 $w \leftarrow w -$  使平衡不变而费用严格下降（ $p > 0$ ），与最优性矛盾。
# 最优解自动满足  $w \leq V$ ，显式上界冗余。
ub[W] = np.inf
lb[W] = 0.0

return c_obj, A_eq.tocsr(), b_eq, list(zip(lb, ub))

def solve_lp(price, load, pv):
    """线性规划：min  $\sum p_t g_t$ 。返回 (obj, x)。"""
    c_obj, A_eq, b_eq, bounds = build_lp(price, load, pv)
    res = linprog(c_obj, A_eq=A_eq, b_eq=b_eq, bounds=bounds, method="highs")
    assert res.status == 0, f"LP 未求得最优解：{res.message}"
    return res.fun, res.x

# ----- MILP 模型（加入充放电互斥） -----
# 变量布局在 LP 基础上追加  $z(T)$ ， $z=1$  允许充电， $z=0$  允许放电
def solve_milp(price, load, pv):
    G, C, D, W, E = _slices()
    n = 5 * T
    Z = slice(n, n + T)

    c_obj = np.zeros(n + T)
    c_obj[G] = price

    rows, cols, vals, lo, hi = [], [], [], [], []

    def add_row(coefs, low, high):
        r = len(lo)
        for j, v in coefs:
            rows.append(r), cols.append(j), vals.append(v)
            lo.append(low), hi.append(high)

    # 能量平衡（等式，用 low==high 表达）
    for t in range(T):
        add_row([(G.start + t, 1.0), (D.start + t, 1.0),
                (C.start + t, -1.0), (W.start + t, -1.0)],
                load[t] - pv[t], load[t] - pv[t])

    # 电量递推
    for t in range(T):
        coefs = [(E.start + t, 1.0), (C.start + t, -ETA), (D.start + t, 1.0 / ETA)]
        if t > 0:
            coefs.append((E.start + t - 1, -1.0))
        rhs = E_INIT if t == 0 else 0.0
        add_row(coefs, rhs, rhs)

    # 日末电量
    add_row([(E.start + T - 1, 1.0)], E_INIT, E_INIT)

```

```

# 互斥:  $c_t - CAP \cdot z_t = 0$ ;  $d_t + CAP \cdot z_t = CAP$ 
for t in range(T):
    add_row([(C.start + t, 1.0), (Z.start + t, -CAP)], -np.inf, 0.0)
    add_row([(D.start + t, 1.0), (Z.start + t, CAP)], -np.inf, CAP)

A = np.array(rows), np.array(cols), np.array(vals)
from scipy.sparse import coo_matrix
A_sp = coo_matrix((A[2], (A[0], A[1])), shape=(len(lo), n + T)).tocsr()

lb = np.zeros(n + T)
ub = np.full(n + T, np.inf)
lb[E], ub[E] = E_MIN, E_MAX
ub[C] = CAP
ub[D] = CAP
# 同 build_lp: 不给 w 设上界 (最优解自动满足  $w \leq V$ , 见该函数注释)
ub[W] = np.inf
ub[Z] = 1.0

integrality = np.zeros(n + T)
integrality[Z] = 1

res = milp(c=c_obj,
           constraints=LinearConstraint(A_sp, np.array(lo), np.array(hi)),
           integrality=integrality,
           bounds=Bounds(lb, ub))
assert res.success, f"MILP 未求得最优解: {res.message}"
return res.fun, res.x

# ----- 无储能基线 -----
def baseline_no_storage(price, load, pv):
    """不用储能: 光伏优先供负载, 不足部分按当时电价购买, 余电弃掉。"""
    g = np.maximum(load - pv, 0.0)
    return float(price @ g), g

# ----- 结果打包 -----
def unpack(x):
    G, C, D, W, E = _slices()
    return dict(g=x[G], c=x[C], d=x[D], w=x[W], E=x[E])

def verify(sol, price, load, pv, tol=1e-6):
    """逐条校验解的可行性与结构性, 返回检查项列表。"""
    g, c, d, w, E = sol["g"], sol["c"], sol["d"], sol["w"], sol["E"]
    checks = []

    # 互补性: 同一时段不同时充放电
    both = int(np.sum((c > tol) & (d > tol)))
    checks.append(("同一时段不同时充放电", both == 0, f"违反时段数 = {both}"))

    # SOC 上下限

```

```

checks.append((f"储电量在 [{E_MIN:.0f}, {E_MAX:.0f}] 内",
               bool(E.min() >= E_MIN - 1e-4 and E.max() <= E_MAX + 1e-4),
               f"SOC    [{E.min():.2f}, {E.max():.2f}]"))

# 首末电量
checks.append(("0:00 与 24:00 储电量相同",
               abs(E[-1] - E_INIT) < 1e-4,
               f"E_0 = {E_INIT:.2f}, E_144 = {E[-1]:.2f}"))

# 逐段能量平衡
bal = g + pv + d - load - c - w
checks.append(("逐段能量平衡", np.max(np.abs(bal)) < 1e-6,
               f"最大残差 = {np.max(np.abs(bal)):.2e}"))

# 充放电功率上限
checks.append((f"充/放电量    {CAP:.2f} kWh/段",
               bool(c.max() <= CAP + 1e-6 and d.max() <= CAP + 1e-6),
               f"c_max = {c.max():.2f}, d_max = {d.max():.2f}"))

# 电量递推一致性（独立重算）
E_rec = E_INIT + np.cumsum(ETA * c - d / ETA)
checks.append(("电量递推式自治", np.max(np.abs(E_rec - E)) < 1e-6,
               f"最大偏差 = {np.max(np.abs(E_rec - E)):.2e}"))

# 弃光量非负：模型约束（lb[W] = 0），属硬性可行性条件
checks.append(("弃光量非负", bool(w.min() >= -1e-9),
               f"w_min = {w.min():.2e}, 弃光合计 = {w.sum():.2f} kWh"))

# 弃光量未超光伏出力：本实例的数值观察，**不是**模型约束。
# 模型已不设  $w \leq V$  的上界（见 build_lp 注释），最优解自动满足该式；
# 换一组数据时此项若失败，并不表示求解出错，详见结果说明。
checks.append(("弃光量未超光伏出力（实例观察，非模型约束）",
               bool(np.all(w <= pv + 1e-6)),
               f"max(w - V) = {float(np.max(w - pv)):.2e}"))

checks.append(("购电量非负", bool(g.min() >= -1e-9), f"g_min = {g.min():.2e}"))

return checks

run_q1.py
"""问题1 主流程：读数据 -> 建LP -> 求解 -> 校验 -> MILP对照 -> 导出结果。"""

import sys
import numpy as np
from pathlib import Path

sys.path.insert(0, str(Path(__file__).parent))
from q1_model import (load_attachment1, solve_lp, solve_milp, unpack, verify,
                      baseline_no_storage, marginal_price, CAP, E_INIT, T, DT)

ROOT = Path(__file__).resolve().parents[2]
ATT1 = ROOT / "data" / "附件" / "附件1.xlsx"

```

```

OUT = Path(__file__).resolve().parents[1] / "results"

# 附件5 未提供, 按题目表1/表2 的时间段自行定义
TABLE1_SLOTS = ["10:00-10:10", "12:00-12:10", "14:00-14:10",
                 "16:00-16:10", "18:00-18:10", "20:00-20:10"]
TABLE2_BLOCKS = ["0:00-4:00", "4:00-8:00", "8:00-12:00",
                 "12:00-16:00", "16:00-20:00", "20:00-24:00"]

def slot_index(start_hhmm):
    """'10:00-10:10' -> 该段在 1..144 中的序号。"""
    h, m = map(int, start_hhmm.split("-")[0].split(":"))
    return h * 6 + m // 10 + 1

def block_index(block):
    """'8:00-12:00' -> (起段序号, 止段序号), 含两端。"""
    a, b = block.split("-")
    ha, ma = map(int, a.split(":"))
    hb, mb = map(int, b.split(":"))
    t0 = (ha * 60 + ma) // 10 + 1
    t1 = (hb * 60 + mb) // 10
    return t0, t1

def main():
    labels, price, load, pv = load_attachment1(ATT1)
    print("=" * 68)
    print("输入数据概览 (附件1) ")
    print("=" * 68)
    print(f" 时段数          : {T} (每段 {DT:.4f} h)")
    print(f" 电价 元/kWh       : {price.min():.4f} ~ {price.max():.4f} 均值 {price.mean():.4f}")
    print(f" 负载 kWh/段       : {load.min():.2f} ~ {load.max():.2f} 全天 {load.sum():.2f} kWh")
    print(f" 光伏 kWh/段       : {pv.min():.2f} ~ {pv.max():.2f} 全天 {pv.sum():.2f} kWh")

    # ---- 求解 ----
    obj_lp, x_lp = solve_lp(price, load, pv)
    sol = unpack(x_lp)
    print("\n" + "=" * 68)
    print("求解结果")
    print("=" * 68)
    print(f" LP 全天购电费 : {obj_lp:,.2f} 元")

    obj_milp, x_milp = solve_milp(price, load, pv)
    print(f" MILP全天购电费 : {obj_milp:,.2f} 元 (含充放电互斥约束)")

    base_cost, g_base = baseline_no_storage(price, load, pv)
    print(f" 无储能基线费用 : {base_cost:,.2f} 元")
    save = base_cost - obj_lp
    print(f" 节省费用       : {save:,.2f} 元 ({save / base_cost * 100:.2f}%)")

    # ---- 校验 ----

```

```

print("\n" + "=" * 68)
print("解的性质校验")
print("=" * 68)
for name, ok, detail in verify(sol, price, load, pv):
    print(f" [{ ' ' if ok else ' '}] {name:<28} {detail}")

gap = abs(obj_lp - obj_milp)
print(f"\n LP 与 MILP 目标值差 : {gap:.6f} 元 "
      f"-> {'二者等价, LP 松弛是紧的' if gap < 1e-4 else '存在差异, 需检查'}")

# ---- 分时段电量来源分解 ----
print("\n" + "=" * 68)
print("全天电量构成")
print("=" * 68)
print(f" 外网购电      : {sol['g'].sum():>12,.2f} kWh")
print(f" 光伏发电      : {pv.sum():>12,.2f} kWh")
print(f" 电池放电      : {sol['d'].sum():>12,.2f} kWh")
print(f" 小区负载      : {load.sum():>12,.2f} kWh")
print(f" 电池充电      : {sol['c'].sum():>12,.2f} kWh")
print(f" 弃光          : {sol['w'].sum():>12,.2f} kWh")
print(f" 往返损失      : {ETA_LOSS(sol):>12,.2f} kWh")
print(f" 谷段/峰段购电 : 参见图2")

# ---- 系统边际电价 (能量平衡约束的对偶变量) ----
print("\n" + "=" * 68)
print("系统边际电价 _t (能量平衡约束的对偶变量, 元/kWh) ")
print("=" * 68)
lam = marginal_price(price, load, pv)
srt = np.argsort(lam)
print(" 最低的 6 个时段 (充电首选) :")
for i in srt[:6]:
    print(f" {labels[i]} = {lam[i]:.4f} 电价={price[i]:.4f} 光伏={pv[i]:7.1f} kWh")
print(" 最高的 6 个时段 (放电首选) :")
for i in srt[-6:][::-1]:
    print(f" {labels[i]} = {lam[i]:.4f} 电价={price[i]:.4f} 光伏={pv[i]:7.1f} kWh")
np.savez(OUT / "duals.npz", lam=lam)

# ---- 表1 ----
print("\n" + "=" * 68)
print("表1 微网在指定时间段的购电量及全天的购电量和购电费")
print("=" * 68)
print(f" {'时间段':<16}{'购电量(kWh)':>14}{'电价(元/kWh)':>14}")
for s in TABLE1_SLOTS:
    t = slot_index(s) - 1
    print(f" {s:<16}{sol['g'][t]:>14.2f}{price[t]:>14.4f}")
print(f" {'-' * 44}")
print(f" {'全天购电量':<16}{sol['g'].sum():>14.2f}")
print(f" {'全天购电费':<16}{obj_lp:>14.2f} 元")

# ---- 表2 ----
print("\n" + "=" * 68)
print("表2 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量")

```

```

print("=" * 68)
print(f"  {'时间段':<16}{'充电量(kWh)':>14}{'放电量(kWh)':>14}")
for b in TABLE2_BLOCKS:
    t0, t1 = block_index(b)
    c_sum = sol["c"][t0 - 1:t1].sum()
    d_sum = sol["d"][t0 - 1:t1].sum()
    print(f"  {b:<16}{c_sum:>14.2f}{d_sum:>14.2f}")
print(f"  {'-' * 44}")
print(f"  {'0:00 储电量':<16}{E_INIT:>14.2f}")
print(f"  {'24:00 储电量':<16}{sol['E'][-1]:>14.2f}")

# ---- 导出 ----
export_xlsx(labels, sol, obj_lp, OUT / "result1.xlsx")
export_detail_csv(labels, price, load, pv, sol, OUT / "schedule_detail.csv")
np.savez(OUT / "solution.npz", labels=np.array(labels), price=price, load=load,
        pv=pv, g=sol["g"], c=sol["c"], d=sol["d"], w=sol["w"], E=sol["E"],
        obj_lp=obj_lp, obj_milp=obj_milp, base_cost=base_cost)
print(f"\n已导出: {OUT / 'result1.xlsx'}")
print(f"已导出: {OUT / 'schedule_detail.csv'}")
print(f"已导出: {OUT / 'solution.npz'}")

def ETA_LOSS(sol):
    """往返效率造成的电量损失。"""
    from q1_model import ETA
    return sol["c"].sum() - (sol["E"][-1] - E_INIT) - sol["d"].sum()

def export_xlsx(labels, sol, obj, path):
    """按附件5 result1.xlsx 模板导出（表头复用模板 + 环形映射）。

    模板标签为左端点且首格 0:10-0:20、末格 0:00+1-0:10+1，与内部时段
    相差一格；故模板第 i 行 ← 内部时段 (i+1) % 144。
    依据官方 CUMCM2026Problems/C题/问题一/make_result1.py。
    """
    import openpyxl
    sys.path.insert(0, str(Path(__file__).resolve().parents[2]
                          / "common" / "efficiency"))
    from template_layout import TPL1, block_labels, template_labels
    tpl_labels = template_labels(TPL1, "计划购电量")
    blocks6 = block_labels(TPL1, "充放电量")

    wb = openpyxl.Workbook()

    ws = wb.active
    ws.title = "计划购电量"
    ws.append(["时间段", "购电量"])
    for i, lab in enumerate(tpl_labels):
        ws.append([lab, round(float(sol["g"][(i + 1) % 144]), 6)])
    ws.column_dimensions["A"].width = 16
    ws.column_dimensions["B"].width = 16

```

```

ws2 = wb.create_sheet("充放电电量")
ws2.append(["时间段", "充电量", "放电量", "时刻", "储电量"])
for b in range(6):
    ws2.append([blocks6[b],
                round(float(sol["c"])[b * 24:(b + 1) * 24].sum()), 6),
                round(float(sol["d"])[b * 24:(b + 1) * 24].sum()), 6),
                "0:00" if b == 0 else ("24:00" if b == 1 else None),
                E_INIT if b == 0 else
                (round(float(sol["E"])[-1]), 6) if b == 1 else None]])
for col, wd in zip("ABCDE", (16, 16, 16, 10, 16)):
    ws2.column_dimensions[col].width = wd

wb.save(path)

def export_detail_csv(labels, price, load, pv, sol, path):
    import csv
    with open(path, "w", newline="", encoding="utf-8-sig") as f:
        w = csv.writer(f)
        w.writerow(["时间", "电价(元/kWh)", "负载(kWh)", "光伏(kWh)",
                    "购电量(kWh)", "充电量(kWh)", "放电量(kWh)",
                    "弃光量(kWh)", "储电量(kWh)"])
        for i, lab in enumerate(labels):
            w.writerow([lab, f"{price[i]:.4f}", f"{load[i]:.4f}", f"{pv[i]:.4f}",
                        f"{sol['g'][i]:.4f}", f"{sol['c'][i]:.4f}",
                        f"{sol['d'][i]:.4f}", f"{sol['w'][i]:.4f}",
                        f"{sol['E'][i]:.4f}"])

if __name__ == "__main__":
    main()

```

附录 E.2 问题二：滚动随机日前调度

rolling_window.py

"""Small, auditable extension of scheme B: change only historical scenario weights.

The original forecaster and LP are imported unchanged. The loader is cached in memory to avoid repeatedly opening attachment 1; no original files are written.

```

"""
from dataclasses import dataclass
from functools import lru_cache
from pathlib import Path
import importlib.util
import sys
import numpy as np

HERE = Path(__file__).resolve().parents[1] / "results"
SOURCE = HERE.parents[1] / 'common' / 'q2_base' / 'run_1439.py'
spec = importlib.util.spec_from_file_location('scheme_b_original', SOURCE)
base = importlib.util.module_from_spec(spec)
sys.modules[spec.name] = base
spec.loader.exec_module(base)

```

```
base.load_attachment1 = lru_cache(maxsize=1)(base.load_attachment1)
```

```
@dataclass(frozen=True)
```

```
class Candidate:
```

```
    name: str
```

```
    window: int = 56
```

```
    half_life: float | None = None
```

```
# Small family fixed before evaluating the outcomes. Shorter windows are a
# separate extension; the first comparison uses only the first four entries.
```

```
CANDIDATES = (
```

```
    Candidate('uniform56'), Candidate('half_life7', half_life=7),
```

```
    Candidate('half_life14', half_life=14), Candidate('half_life28', half_life=28),
```

```
    Candidate('uniform28', window=28), Candidate('uniform42', window=42),
```

```
)
```

```
BY_NAME = {c.name: c for c in CANDIDATES}
```

```
PRIMARY_NAMES = tuple(c.name for c in CANDIDATES[:4])
```

```
class Inputs:
```

```
    def __init__(self, load=None, pv=None):
```

```
        dates, source_load, source_pv = base.load_attachment2()
```

```
        self.dates = dates
```

```
        self.load = source_load if load is None else load
```

```
        self.pv = source_pv if pv is None else pv
```

```
        _, self.price, _, _ = base.load_attachment1()
```

```
        self.actual = (self.load - self.pv) * base.DT
```

```
        self.point = np.array([base.forecast_net(self.load, self.pv, d) * base.DT
                                for d in range(len(self.load))])
```

```
        self.tomorrow = np.array([base.forecast_net(self.load, self.pv, d + 1, upto=d) * base.DT
                                    for d in range(len(self.load) - 1)])
```

```
        self.residual = self.actual - self.point
```

```
    def scenario_data(self, day, candidate, first_residual=0):
```

```
        indices = np.arange(max(first_residual, day - candidate.window), day)
```

```
        if len(indices) == 0:
```

```
            return self.point[day][None, :], np.ones(1), indices
```

```
        ages = day - 1 - indices
```

```
        weights = np.ones(len(indices)) if candidate.half_life is None else np.exp2(-ages / candidate.half_life)
```

```
        weights /= weights.sum()
```

```
        return self.point[day][None, :] + self.residual[indices], weights, indices
```

```
    def solve_day(self, day, energy, candidate, first_residual=0):
```

```
        scenario, weights, indices = self.scenario_data(day, candidate, first_residual)
```

```
        prediction = self.point[day] if day == 364 else np.r_[self.point[day], self.tomorrow[day]]
```

```
        terminal = 6000. if day == 364 else energy
```

```
        plan = base.solve_horizon(prediction, scenario, weights,
```

```
                                   np.tile(self.price, len(prediction)//144), energy, terminal)
```

```
        return plan, weights, indices
```

```

def weighted_quantile(values, weights, tau=.8):
    """Left empirical quantile in every column; same scenario weight per day."""
    order = np.argsort(values, axis=0, kind='stable')
    sorted_values = np.take_along_axis(values, order, axis=0)
    cumulative = np.cumsum(weights[order], axis=0)
    positions = np.argmax(cumulative >= tau - 1e-12, axis=0)
    return sorted_values[positions, np.arange(values.shape[1])]

def prequential_scores(inputs, names=PRIMARY_NAMES, first_residual=0):
    """Score forecasts made using only days < d, never refit old predictions."""
    scores = np.full((365, len(names)), np.nan)
    quantiles = np.full((365, len(names), 144), np.nan)
    for day in range(max(7, first_residual + 1), 365):
        for k, name in enumerate(names):
            scenarios, weights, _ = inputs.scenario_data(day, BY_NAME[name], first_residual)
            target = np.maximum(inputs.point[day], weighted_quantile(scenarios, weights))
            error = inputs.actual[day] - target
            quantiles[day, k] = target
            # Includes the point-forecast floor but does not substitute for the SOC-coupled LP.
            scores[day, k] = float(inputs.price @ np.where(error >= 0, .8*error, -.2*error))
    return scores, quantiles

def choose_week(day, scores, names=PRIMARY_NAMES, lookback=28, update_every=7):
    """Every 7 days choose on the previous 28 completed days. Tie order is fixed."""
    decision_day = 31 + ((day - 31)//update_every)*update_every
    lo = max(7, decision_day - lookback)
    historic = scores[lo:decision_day]
    valid = np.isfinite(historic).all(axis=1)
    if valid.sum() < 14:
        return names[0], decision_day - 1, [None]*len(names)
    means = historic[valid].mean(axis=0)
    return names[int(np.argmin(means))], decision_day - 1, means.tolist()

run_experiment.py

"""Run comparable scenario-window experiments without changing the source B.

Default: four fixed-weight strategies plus the prespecified weekly selector.
--extended adds uniform 28/42-day windows and a six-candidate selector.
"""

from datetime import datetime, timedelta
from pathlib import Path
import argparse
import hashlib
import json
import sys
import time
import numpy as np
import pandas as pd
from rolling_window import HERE, SOURCE, Inputs, BY_NAME, PRIMARY_NAMES, CANDIDATES, prequential_sc

```

```

# 附件5 模板的表头与「环形映射」规则（见 common/efficiency/template_layout.py）
_ETA_DIR = Path(__file__).resolve().parents[2] / "common" / "efficiency"
if str(_ETA_DIR) not in sys.path:
    sys.path.insert(0, str(_ETA_DIR))
from template_layout import (TPL2, block_labels, circular, # noqa: E402
                             template_labels)

def label(slot):
    return '24:00' if slot == 144 else f'{slot//6:02d}:{slot%6*10:02d}'

def warmup(inputs):
    """Single common baseline January, preserving the original initial condition."""
    energy = 6000.
    rows = []
    for day in range(31):
        p, _, _ = inputs.solve_day(day, energy, BY_NAME['uniform56'])
        rows.append(dict(day=day, energy_start_kwh=energy, energy_end_kwh=float(p['E'][143])))
        energy = float(p['E'][143])
    return energy, rows

def simulate(inputs, name, initial, scores=None, names=PRIMARY_NAMES, first_residual=0, report_days=
    energy = initial
    rows, days, selections = [], [], []
    started = time.time()
    for day in range(31, min(365, 31+report_days)):
        date = (datetime(2025, 1, 1)+timedelta(days=day)).strftime('%Y-%m-%d')
        if name.startswith('online'):
            selected, last_observed, means = choose_week(day, scores, names)
        else:
            selected, last_observed, means = name, day-1, []
        candidate = BY_NAME[selected]
        p, weights, indices = inputs.solve_day(day, energy, candidate, first_residual)
        g, c, d, E = (p[k][:144] for k in ('g', 'c', 'd', 'E'))
        before = np.r_[energy, E[:-1]]
        z = g+d-c
        emergency = np.maximum(inputs.actual[day]-z, 0)
        dump = np.maximum(z-inputs.actual[day], 0)
        planned_cost = inputs.price*g
        emergency_cost = 5*inputs.price*emergency
        total = planned_cost+emergency_cost
        for t in range(144):
            rows.append(dict(date=date, slot=t, start=label(t), end=label(t+1), price_yuan_per_kwh=inputs
                            forecast_net_kwh=inputs.point[day, t], actual_net_kwh=inputs.actual[day, t],
                            g_kwh=g[t], c_kwh=c[t], d_kwh=d[t], energy_before_kwh=before[t], energy_af
                            z_kwh=z[t], emergency_kwh=emergency[t], dump_kwh=dump[t],
                            planned_cost_yuan=planned_cost[t], emergency_cost_yuan=emergency_cost[t],
            days.append(dict(date=date, planned_cost_yuan=planned_cost.sum(), emergency_cost_yuan=emergen
                            total_cost_yuan=total.sum(), g_kwh=g.sum(), c_kwh=c.sum(), d_kwh=d.sum(), emer

```

```

        dump_kwh=dump.sum(),energy_start_kwh=energy,energy_end_kwh=E[-1],chosen_candidate=indices[indices>0].sum(),
        sample_count=len(indices),effective_sample_count=1/np.sum(weights**2),
        max_history_day=(datetime(2025,1,1)+timedelta(days=int(indices[-1]))).strftime('%Y-%m-%d'),
        selection_last_observed_day=(datetime(2025,1,1)+timedelta(days=last_observed)).strftime('%Y-%m-%d'),
        selections.append(dict(date=date,candidate=selected,score_cutoff_day=last_observed,
                                candidate_names=list(names),mean_past_pinball_scores=means))

    energy = float(E[-1])
    if (day-31)%60==0 or day==min(364,30+report_days):
        print(f'{name:16s} {date} {sum(r["total_cost_yuan"] for r in days)/1e4:.3f} wan {time.time()-started:.3f}')
    detail,daily = pd.DataFrame(rows),pd.DataFrame(days)
    summary=dict(name=name,days=len(daily),initial_energy_kwh=initial,final_energy_kwh=energy,
                first_residual_index=first_residual,seconds=time.time()-started)
    for col in ('planned_cost_yuan','emergency_cost_yuan','total_cost_yuan','g_kwh','c_kwh','d_kwh'):
        summary[col]=float(daily[col].sum())
    summary['h1_cost_yuan']=float(daily.loc[daily.date<'2025-07-01','total_cost_yuan'].sum())
    summary['h2_cost_yuan']=float(daily.loc[daily.date>='2025-07-01','total_cost_yuan'].sum())
    return detail,daily,summary,selections

def export_workbook(detail, daily, path):
    import openpyxl
    from openpyxl.styles import Font
    # 模板表头（左端点，首格 0:10-0:20、末格 0:00+1-0:10+1）+ 环形映射：
    # 模板第 i 格 + 内部时段 (i+1) % 144。依据官方问题一 make_result1.py。
    from template_layout import template_labels, circular, block_labels, TPL2
    tpl_labels = template_labels(TPL2, '计划购电量')
    blocks6 = block_labels(TPL2, '充放电量')
    w=openpyxl.Workbook(); ws=w.active; ws.title='计划购电量'
    ws.append(['日期\\时间']+tpl_labels+['全天购电量','全天购电费'])
    for date,g in detail.groupby('date',sort=False):
        vals = circular(g.g_kwh.to_numpy(float))
        ws.append([datetime.fromisoformat(date)]+[round(v,6) for v in vals]
                +[g.g_kwh.sum(),g.planned_cost_yuan.sum()])
    ws.freeze_panes='B2'
    ws=w.create_sheet('充放电量'); ws.append(['日期','时间段','充电量','放电量','时刻','储电量'])
    for date,g in detail.groupby('date',sort=False):
        for b in range(6):
            block=g.iloc[b*24:(b+1)*24]
            ws.append([datetime.fromisoformat(date) if b==0 else None,blocks6[b],
                    block.c_kwh.sum(),block.d_kwh.sum(),'0:00' if b==0 else ('24:00' if b==1 else
                    g.iloc[0].energy_before_kwh if b==0 else (g.iloc[-1].energy_after_kwh if b==1 else None))])
    ws=w.create_sheet('紧急购电量'); ws.append(['日期','购电时间段','购电量'])
    for date,g in detail.groupby('date',sort=False):
        e=g.emergency_kwh.to_numpy(); blocks=[]; t=0
        while t<144:
            if e[t]<=1e-7: t+=1; continue
            start=t
            while t<144 and e[t]>1e-7: t+=1
            blocks.append((label(start)+'-'+label(t),e[start:t].sum()))
        for i,(span,amount) in enumerate(blocks or [('-',0.)]):
            ws.append([datetime.fromisoformat(date) if i==0 else None,span,amount])
    for ws in w:

```

```

        for cell in ws[1]: cell.font=Font(bold=True)
        ws.column_dimensions['A'].width=14
        for row in ws.iter_rows(min_row=2):
            for cell in row:
                if isinstance(cell.value,datetime):cell.number_format='yyyy-mm-dd'
                elif isinstance(cell.value,(int,float)):cell.number_format='0.0000'
w.save(path)

def save_variant(out,name,result):
    detail,daily,summary,selections=result
    folder=out/'variants'/name;folder.mkdir(parents=True,exist_ok=True)
    detail.to_csv(folder/'schedule_detail.csv.gz',index=False,encoding='utf-8-sig',compression='gzip')
    daily.to_csv(folder/'daily_summary.csv',index=False,encoding='utf-8-sig')
    (folder/'summary.json').write_text(json.dumps(summary,ensure_ascii=False,indent=2),encoding='utf-8')
    (folder/'selection_log.json').write_text(json.dumps(selections,ensure_ascii=False,indent=2),encoding='utf-8')
    export_workbook(detail,daily,folder/'result2.xlsx')
    return summary

def main():
    ap=argparse.ArgumentParser()
    ap.add_argument('--extended',action='store_true')
    ap.add_argument('--first-residual',type=int,default=0)
    ap.add_argument('--initial',type=float,default=None)
    ap.add_argument('--days',type=int,default=334)
    ap.add_argument('--out',type=Path,default=HERE)
    ap.add_argument('--only',nargs='+',default=None)
    args=ap.parse_args()
    out=args.out if args.out.is_absolute() else HERE/args.out; out.mkdir(parents=True,exist_ok=True)
    before=hashlib.sha256(SOURCE.read_bytes()).hexdigest()
    inputs=Inputs()
    initial,january=warmup(inputs) if args.initial is None else (args.initial,[])
    names=tuple(c.name for c in CANDIDATES) if args.extended else PRIMARY_NAMES
    scores,quantiles=prequential_scores(inputs,names,args.first_residual)
    scoreframe=pd.DataFrame(scores,columns=names)
    scoreframe.insert(0,'date',pd.date_range('2025-01-01',periods=365))
    scoreframe.to_csv(out/'prequential_scores.csv',index=False,encoding='utf-8-sig')
    np.savez_compressed(out/'prequential_quantiles.npz',values=quantiles,candidates=np.array(names))
    (out/'common_january.json').write_text(json.dumps(january,indent=2),encoding='utf-8')
    modes=args.only or list(names)+['online_extended' if args.extended else 'online_primary']
    summaries=[]
    for name in modes:
        result=simulate(inputs,name,initial,scores,names,args.first_residual,args.days)
        summaries.append(save_variant(out,name,result))
    comparison=pd.DataFrame(summaries)
    if 'uniform56' in comparison.name.values:
        baseline=float(comparison.loc[comparison.name=='uniform56','total_cost_yuan'].iloc[0])
        comparison['saving_yuan']=baseline-comparison.total_cost_yuan
        comparison['saving_percent']=comparison.saving_yuan/baseline*100
    comparison.to_csv(out/'comparison.csv',index=False,encoding='utf-8-sig')
    assert before==hashlib.sha256(SOURCE.read_bytes()).hexdigest()

```

```

        metadata=dict(source_file=str(SOURCE),source_sha256=before,first_residual=args.first_residual,
                        initial_soc=initial,year_end_terminal_soc=6000.,year_end_executed=args.days>=334,
                        horizon_days=2,candidates=[c.__dict__ for c in CANDIDATES if c.name in names],
                        online_rule='Every 7 days, select minimum mean 0.8-pinball loss over previous 28',
                        fixed_parameter_comparison='Exploratory retrospective comparison, not an untouched',
                        inherited_forecaster='Original sw4/drift5 + t7/drift28 without changes, including',
                        python=sys.version)
    (out/'run_metadata.json').write_text(json.dumps(metadata,ensure_ascii=False,indent=2),encoding='utf-8')
    print(comparison.to_string(index=False),flush=True)

if __name__=='__main__': main()

verify_window.py
"""Independently verify window experiments against raw attachments and exports.

No optimizer, forecast, or settlement routine is imported here. Monetary and
physical checks are recalculated from the exported decisions and raw inputs.
"""
from __future__ import annotations

import argparse
import json
import re
from datetime import datetime, time
from pathlib import Path
from typing import Any

import numpy as np
import openpyxl
import pandas as pd

HERE = Path(__file__).resolve().parents[1] / "results"
DT = 1 / 6
T = 144
TOL = 1e-5

def _date(value: Any) -> str:
    return pd.Timestamp(value).strftime("%Y-%m-%d")

def _time_label(slot: int) -> str:
    minute = slot * 10
    return f"{minute // 60:02d}:{minute % 60:02d}"

def _minutes(value: Any) -> int:
    if isinstance(value, (time, datetime)):
        return value.hour * 60 + value.minute
    text = str(value).strip()
    if text in {"0:00+1", "00:00+1"}:

```

```

        return 1440
    match = re.fullmatch(r"(\d{1,2}):(\d{2})(?::00)?", text)
    if not match:
        raise ValueError(f"Unrecognized time: {value!r}")
    return int(match[1]) * 60 + int(match[2])

def _interval(value: Any) -> tuple[int, int]:
    parts = re.split(r"[-—~]", str(value).strip())
    if len(parts) != 2:
        raise ValueError(f"Unrecognized interval: {value!r}")
    return _minutes(parts[0]), _minutes(parts[1])

def _max_abs(values: Any) -> float:
    arr = np.asarray(values, dtype=float)
    return float(np.max(np.abs(arr))) if arr.size else 0.0

def _read_raw(raw_dir: Path) -> tuple[np.ndarray, dict[str, np.ndarray]]:
    """Read XLSX independently; attachment rows denote interval end times."""
    attachments = raw_dir if (raw_dir / "附件1.xlsx").exists() else raw_dir / "附件"
    wb1 = openpyxl.load_workbook(attachments / "附件1.xlsx", read_only=True, data_only=True)
    rows1 = list(wb1.active.iter_rows(min_row=2, values_only=True))
    wb1.close()
    if len(rows1) != T:
        raise ValueError(f"Attachment 1 has {len(rows1)} data rows instead of 144")
    if any(_minutes(row[0]) % 1440 != ((slot + 1) * 10) % 1440
          for slot, row in enumerate(rows1)):
        raise ValueError("Attachment 1 interval-end labels are not consecutive ten-minute times")
    price = np.asarray([row[1] for row in rows1], dtype=float)
    wb2 = openpyxl.load_workbook(attachments / "附件2.xlsx", read_only=True, data_only=True)
    channels: list[dict[str, np.ndarray]] = []
    for name in ["小区负载", "光伏发电实际功率"]:
        header = next(wb2[name].iter_rows(min_row=1, max_row=1, values_only=True))
        if len(header) != T + 1 or any(_minutes(value) % 1440 != ((slot + 1) * 10) % 1440
                                      for slot, value in enumerate(header[1:])):
            raise ValueError(f"Attachment 2 time columns are inconsistent: {name}")
        data: dict[str, np.ndarray] = {}
        for row in wb2[name].iter_rows(min_row=2, values_only=True):
            day = _date(row[0])
            values = np.asarray(row[1:], dtype=float)
            if len(values) != T or not np.isfinite(values).all():
                raise ValueError(f"Invalid raw row {name}: {day}")
            if day in data:
                raise ValueError(f"Duplicate raw date {name}: {day}")
            data[day] = values
        channels.append(data)
    wb2.close()
    if channels[0].keys() != channels[1].keys():
        raise ValueError("Attachment 2 load/PV dates do not match")
    net = {day: (channels[0][day] - channels[1][day]) * DT for day in channels[0]}

```

```

return price, net

def check_schedule(
    detail: pd.DataFrame,
    daily: pd.DataFrame,
    expected_initial: float,
    raw_dir: str | Path | None = None,
) -> dict[str, Any]:
    """Check all 334 reporting days, accounting, feasibility, and date metadata.

    Historical dates are necessary checks, not a proof that hidden fitting code
    is causal; a separate perturbation test must recompute that pipeline.
    """
    errors: list[str] = []
    metrics: dict[str, Any] = {}

    def check(name: str, condition: Any, metric: Any = None) -> None:
        if metric is not None:
            metrics[name] = metric
        if not bool(condition):
            errors.append(name)

    required_detail = {
        "date", "slot", "start", "end", "price_yuan_per_kwh", "forecast_net_kwh",
        "actual_net_kwh", "g_kwh", "c_kwh", "d_kwh", "energy_before_kwh",
        "energy_after_kwh", "z_kwh", "emergency_kwh", "dump_kwh",
        "planned_cost_yuan", "emergency_cost_yuan", "total_cost_yuan",
    }
    required_daily = {
        "date", "planned_cost_yuan", "emergency_cost_yuan", "total_cost_yuan",
        "g_kwh", "c_kwh", "d_kwh", "emergency_kwh", "dump_kwh",
        "energy_start_kwh", "energy_end_kwh", "chosen_candidate", "sample_count",
        "effective_sample_count", "max_history_day", "selection_last_observed_day",
    }
    missing_detail = sorted(required_detail - set(detail.columns))
    missing_daily = sorted(required_daily - set(daily.columns))
    if missing_detail or missing_daily:
        return {"pass": False, "errors": ["missing_columns"],
                "missing_detail_columns": missing_detail, "missing_daily_columns": missing_daily}
    detail, daily = detail.copy(), daily.copy()
    try:
        detail["date"] = detail["date"].map(_date)
        daily["date"] = daily["date"].map(_date)
        num_detail = sorted(required_detail - {"date", "start", "end"})
        num_daily = sorted(required_daily - {"date", "chosen_candidate", "max_history_day",
                                             "selection_last_observed_day"})
        for col in num_detail:
            detail[col] = pd.to_numeric(detail[col], errors="raise")
        for col in num_daily:
            daily[col] = pd.to_numeric(daily[col], errors="raise")
        finite = bool(np.isfinite(detail[num_detail].to_numpy()).all())

```

```

        and np.isfinite(daily[num_daily].to_numpy()).all())
    check("finite_numeric_values", finite)
    if not finite or detail.empty or daily.empty:
        return {"pass": False, "errors": errors + (["empty_output"] if detail.empty or daily.empty
            "metrics": metrics}
except (TypeError, ValueError) as exc:
    return {"pass": False, "errors": ["invalid_column_values"], "detail": str(exc)}

expected_dates = pd.date_range("2025-02-01", "2025-12-31", freq="D").strftime("%Y-%m-%d").tolist()
actual_dates = detail["date"].drop_duplicates().tolist()
check("report_date_range", actual_dates == expected_dates)
check("daily_date_range", daily["date"].tolist() == expected_dates)
check("report_row_count", len(detail) == len(expected_dates) * T, int(len(detail)))
check("daily_row_count", len(daily) == len(expected_dates), int(len(daily)))
check("unique_date_slot", not detail.duplicated(["date", "slot"]).any())
check("unique_daily_date", not daily["date"].duplicated().any())
check("chronological_rows", list(zip(detail["date"], detail["slot"])) ==
    sorted(zip(detail["date"], detail["slot"])))
slot_errors = 0
for _, frame in detail.groupby("date", sort=False):
    slot_errors += int(frame["slot"].tolist() != list(range(T)))
check("complete_slots_per_day", slot_errors == 0, slot_errors)
try:
    start = detail["start"].map(_minutes).to_numpy()
    end = detail["end"].map(_minutes).to_numpy()
    slots = detail["slot"].to_numpy()
    check("ten_minute_interval_labels", np.array_equal(start, slots * 10)
        and np.array_equal(end, (slots + 1) * 10))
except (TypeError, ValueError) as exc:
    check("ten_minute_interval_labels", False, str(exc))

g, c, d = (detail[col].to_numpy(dtype=float) for col in ["g_kwh", "c_kwh", "d_kwh"])
eb, ea = (detail[col].to_numpy(dtype=float) for col in ["energy_before_kwh", "energy_after_kwh"])
p = detail["price_yuan_per_kwh"].to_numpy(dtype=float)
n = detail["actual_net_kwh"].to_numpy(dtype=float)
z = g + d - c
em = np.maximum(n - z, 0)
dump = np.maximum(z - n, 0)
expected_values = {
    "z_kwh": z,
    "emergency_kwh": em,
    "dump_kwh": dump,
    "planned_cost_yuan": p * g,
    "emergency_cost_yuan": 5 * p * em,
    "total_cost_yuan": p * g + 5 * p * em,
}
for col, expected in expected_values.items():
    deviation = _max_abs(detail[col].to_numpy() - expected)
    check(f"{col}_max_error", deviation <= TOL, deviation)
residual = _max_abs(ea - eb - 0.9 * c + d / 0.9)
check("soc_equation_max_error_kwh", residual <= TOL, residual)
residual = _max_abs(eb[1:] - ea[:-1])

```

```

check("soc_continuity_max_error_kwh", residual <= TOL, residual)
check("initial_soc_kwh", abs(eb[0] - expected_initial) <= TOL, float(eb[0]))
check("final_soc_kwh", abs(ea[-1] - 6000) <= TOL, float(ea[-1]))
check("soc_min_kwh", min(eb.min(), ea.min()) >= 1200 - TOL, float(min(eb.min(), ea.min())))
check("soc_max_kwh", max(eb.max(), ea.max()) <= 10800 + TOL, float(max(eb.max(), ea.max())))
check("charge_max_kw", c.max() / DT <= 5000 + TOL, float(c.max() / DT))
check("discharge_max_kw", d.max() / DT <= 5000 + TOL, float(d.max() / DT))
check("nonnegative_grid_charge_discharge", min(g.min(), c.min(), d.min()) >= -TOL)
check("nonnegative_emergency_dump", min(detail["emergency_kwh"].min(), detail["dump_kwh"].min())
simultaneous = int(((c > 1e-6) & (d > 1e-6)).sum())
check("simultaneous_charge_discharge_slots", simultaneous == 0, simultaneous)
margin = float((z - detail["forecast_net_kwh"].to_numpy()).min())
check("plan_forecast_min_margin_kwh", margin >= -TOL, margin)
balance = _max_abs(z + detail["emergency_kwh"] - n - detail["dump_kwh"])
check("actual_balance_max_error_kwh", balance <= TOL, balance)

try:
    raw_path = Path(raw_dir) if raw_dir is not None else HERE.parents[1] / "data"
    raw_price, raw_net = _read_raw(raw_path)
    reference_price = np.asarray([raw_price[int(s)] for s in detail["slot"]])
    reference_net = np.asarray([raw_net[day][int(slot)] for day, slot in
                                zip(detail["date"], detail["slot"])])
    deviation = _max_abs(p - reference_price)
    check("raw_price_max_error_yuan_per_kwh", deviation <= 1e-10, deviation)
    deviation = _max_abs(n - reference_net)
    check("raw_net_energy_max_error_kwh", deviation <= TOL, deviation)
except (KeyError, IndexError, ValueError, OSError) as exc:
    check("raw_attachment_match", False, str(exc))

aggregate_cols = ["planned_cost_yuan", "emergency_cost_yuan", "total_cost_yuan", "g_kwh",
                  "c_kwh", "d_kwh", "emergency_kwh", "dump_kwh"]
if not daily["date"].duplicated().any():
    aggregate = detail.groupby("date")[aggregate_cols].sum()
    target = daily.set_index("date")
    common_dates = aggregate.index.intersection(target.index)
    check("detail_daily_matching_dates", len(common_dates) == len(aggregate) == len(target))
    for col in aggregate_cols:
        deviation = _max_abs(aggregate.loc[common_dates, col] - target.loc[common_dates, col])
        check(f"daily_{col}_max_error", deviation <= 1e-4, deviation)
    for expected_col, field, endpoint in [("energy_start_kwh", "energy_before_kwh", "first"),
                                           ("energy_end_kwh", "energy_after_kwh", "last")]:
        measured = detail.groupby("date")[field].agg(endpoint)
        deviation = _max_abs(measured.loc[common_dates] - target.loc[common_dates, expected_col])
        check(f"daily_{expected_col}_max_error", deviation <= TOL, deviation)

history_violations = []
selection_violations = []
for row in daily.itertuples(index=False):
    for attr, failures, required in [("max_history_day", history_violations, True),
                                     ("selection_last_observed_day", selection_violations, False)]:
        value = getattr(row, attr)
        if pd.isna(value) or str(value).strip() == "":

```

```

        if required:
            failures.append(row.date)
        continue
    try:
        if _date(value) >= row.date:
            failures.append(row.date)
    except (TypeError, ValueError):
        failures.append(row.date)
    check("history_dates_before_decision", not history_violations, history_violations)
    check("selection_dates_before_decision", not selection_violations, selection_violations)
    counts = daily["sample_count"].to_numpy()
    effective = daily["effective_sample_count"].to_numpy()
    check("valid_sample_counts", bool(np.all(counts >= 1) and np.all(counts <= 56)
                                         and np.all(counts == counts.astype(int))))
    check("effective_sample_count_bounds", bool(np.all(effective >= 1 - TOL)
                                                    and np.all(effective <= counts + TOL)))
    check("candidate_names_present", daily["chosen_candidate"].notna().all()
                                         and daily["chosen_candidate"].astype(str).str.len().gt(0).all())
    metrics["recomputed_total_cost_yuan"] = float((p * g + 5 * p * em).sum())
    metrics["recomputed_emergency_kwh"] = float(em.sum())
    metrics["recomputed_dump_kwh"] = float(dump.sum())
    return {"pass": not errors, "errors": errors, "metrics": metrics,
           "scope": "Exported physical and accounting constraints; independent raw data; historical  

                    Causality of fitting and selection requires the separate future-perturbation"}

def check_workbook(path: str | Path, detail: pd.DataFrame) -> dict[str, Any]:
    """Read all three result2 worksheets back, including interval alignment."""
    errors: list[str] = []
    deviations: dict[str, float] = {}

def numeric(name: str, actual: Any, expected: float, tolerance: float = 5.1e-5) -> None:
    try:
        delta = abs(float(actual) - float(expected))
        if not np.isfinite(delta):
            raise ValueError("nonfinite cell")
    except (TypeError, ValueError):
        delta = float("inf")
    if np.isfinite(delta):
        deviations[name] = max(deviations.get(name, 0), delta)
    if delta > tolerance and name not in errors:
        errors.append(name)

frame = detail.copy()
frame["date"] = frame["date"].map(_date)
frame = frame.sort_values(["date", "slot"])
days = list(frame.groupby("date", sort=True))
try:
    wb = openpyxl.load_workbook(path, read_only=True, data_only=True)
    required = ["计划购电量", "充放电量", "紧急购电量"]
    if any(name not in wb.sheetnames for name in required):
        wb.close()

```

```

        return {"pass": False, "errors": ["missing_worksheets"]}

plan = list(wb[required[0]].iter_rows(values_only=True))
if len(plan) != len(days) + 1 or len(plan[0]) != T + 3:
    errors.append("plan_table_dimensions")
expected_labels = [f"_{time_label(t)}-{_time_label(t + 1)}" for t in range(T)]
if list(plan[0][1:T + 1]) != expected_labels:
    errors.append("plan_interval_headers")
for row, (day, group) in zip(plan[1:], days):
    if _date(row[0]) != day:
        errors.append(f"plan_date:{day}")
    for value, expected in zip(row[1:T + 1], group["g_kwh"]):
        numeric("plan_slot_energy_max_error", value, expected)
    numeric("plan_daily_energy_max_error", row[T + 1], group["g_kwh"].sum())
    numeric("plan_daily_cost_max_error", row[T + 2],
            (group["g_kwh"] * group["price_yuan_per_kwh"]).sum(), 0.0051)

storage = list(wb[required[1]].iter_rows(values_only=True))
if len(storage) != 1 + 6 * len(days):
    errors.append("storage_table_dimensions")
for i, (day, group) in enumerate(days):
    for block in range(6):
        ix = 1 + 6 * i + block
        if ix >= len(storage):
            continue
        row = storage[ix]
        if block == 0 and _date(row[0]) != day:
            errors.append(f"storage_date:{day}")
        if _interval(row[1]) != (block * 240, (block + 1) * 240):
            errors.append(f"storage_interval:{day}:{block}")
        slots = group.iloc[block * 24:(block + 1) * 24]
        numeric("storage_charge_block_max_error", row[2], slots["c_kwh"].sum())
        numeric("storage_discharge_block_max_error", row[3], slots["d_kwh"].sum())
        if block < 2:
            expected_time = 0 if block == 0 else 1440
            if _minutes(row[4]) != expected_time:
                errors.append(f"storage_soc_time:{day}:{block}")
            expected_soc = group["energy_before_kwh"].iloc[0] if block == 0 else group["ene
            numeric("storage_soc_max_error", row[5], expected_soc)

expected_emergency: list[tuple[str, tuple[int, int] | None, float]] = []
for day, group in days:
    emergency = group["emergency_kwh"].to_numpy()
    cursor = 0
    found = False
    while cursor < T:
        if emergency[cursor] <= 1e-6:
            cursor += 1
            continue
        start, amount = cursor, 0.0
        while cursor < T and emergency[cursor] > 1e-6:
            amount += emergency[cursor]

```

```

        cursor += 1
        expected_emergency.append((day, (start * 10, cursor * 10), amount))
        found = True
    if not found:
        expected_emergency.append((day, None, 0.0))
    emergency_rows = list(wb[required[2]].iter_rows(min_row=2, values_only=True))
    if len(emergency_rows) != len(expected_emergency):
        errors.append("emergency_interval_row_count")
    active_date = None
    for row, (day, expected_interval, amount) in zip(emergency_rows, expected_emergency):
        if row[0] is not None:
            active_date = _date(row[0])
        if active_date != day:
            errors.append(f"emergency_date:{day}")
        if expected_interval is None:
            if row[1] not in {None, "", "—", "-", "无"}:
                errors.append(f"emergency_empty_label:{day}")
            elif _interval(row[1]) != expected_interval:
                errors.append(f"emergency_interval:{day}:{expected_interval}")
        numeric("emergency_interval_energy_max_error", row[2], amount)
    wb.close()
except (OSError, IndexError, TypeError, ValueError) as exc:
    errors.append(f"workbook_read_error:{exc}")
return {"pass": not errors, "errors": errors, "max_deviations": deviations,
        "workbook": str(path)}

```

```

def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--root", type=Path, default=HERE)
    parser.add_argument("--raw-dir", type=Path, default=HERE.parents[1] / "data")
    parser.add_argument("--expected-initial", type=float, default=8550.0)
    args = parser.parse_args()
    reports: dict[str, Any] = {}
    for variant in sorted((args.root / "variants").glob("*")):
        detail_path = variant / "schedule_detail.csv.gz"
        daily_path = variant / "daily_summary.csv"
        if not detail_path.exists() or not daily_path.exists():
            continue
        detail = pd.read_csv(detail_path)
        daily = pd.read_csv(daily_path)
        report = {"schedule": check_schedule(detail, daily, args.expected_initial, args.raw_dir)}
        workbook = variant / "result2.xlsx"
        if workbook.exists():
            report["workbook"] = check_workbook(workbook, detail)
        report["pass"] = all(item["pass"] for item in report.values())
        reports[variant.name] = report
        print(f"{variant.name}: {'PASS' if report['pass'] else 'FAIL'}", flush=True)
    result = {"pass": bool(reports) and all(item["pass"] for item in reports.values()),
              "expected_initial_kwh": args.expected_initial, "variants": reports}
    (args.root / "verification.json").write_text(json.dumps(result, ensure_ascii=False, indent=2),
    print(f"Verified {len(reports)} variants. Overall: {'PASS' if result['pass'] else 'FAIL'}")

```

```

        if not result["pass"]:
            raise SystemExit(1)

if __name__ == "__main__":
    main()

check_information.py
"""Rebuild inputs after altering future observations; current controls must not move."""
import json
import numpy as np
import pandas as pd
from rolling_window import HERE, Inputs, BY_NAME, PRIMARY_NAMES, prequential_scores, choose_week, b

def main():
    inputs=Inputs()
    scores,quantiles=prequential_scores(inputs)
    tests=[]
    for day in (31,151,333):
        load,pv=inputs.load.copy(),inputs.pv.copy()
        load[day:]=load[day:]*1.6+120
        pv[day:]=pv[day:]*.3+100
        changed=Inputs(load,pv)
        other_scores,other_quantiles=prequential_scores(changed)
        selected,last,_=choose_week(day,scores)
        selected_other,last_other,_=choose_week(day,other_scores)
        scenario,w,idx=inputs.scenario_data(day,BY_NAME[selected])
        scenario_other,w_other,idx_other=changed.scenario_data(day,BY_NAME[selected_other])
        a,_=inputs.solve_day(day,8550.,BY_NAME[selected])
        b,_=changed.solve_day(day,8550.,BY_NAME[selected_other])
        changes=dict(point=float(abs(inputs.point[day]-changed.point[day]).max()),
                    next_day_point=float(abs(inputs.tomorrow[day]-changed.tomorrow[day]).max()),
                    scenario=float(abs(scenario-scenario_other).max()),
                    weights=float(abs(w-w_other).max()),
                    historical_scores=float(np.nanmax(abs(scores[:day]-other_scores[:day]))),
                    current_quantiles=float(abs(quantiles[day]-other_quantiles[day]).max()),
                    decisions=max(float(abs(a[k]-b[k]).max()) for k in ('g','c','d','E')))
        tests.append(dict(day=day,selected=selected,last_observed_day=last,
                        selected_unchanged=selected==selected_other and last==last_other,
                        max_changes=changes,pass_check=selected==selected_other and last==last_observed_day
                        and np.array_equal(idx,idx_other) and max(changes.values())<1e-7))
    # A changing score later in a week may not alter the choice made at its start.
    late_day=35
    edited=scores.copy();edited[31:late_day]=1e12
    frozen=choose_week(late_day,scores)[0]==choose_week(late_day,edited)[0]
    unit=[]
    for weights,expected in [(np.array([.85,.1,.05]),34200.), (np.array([.55,.2,.25]),57600.)]:
        scenario=np.repeat(np.array([100.,200.,400.])[:,None],144,axis=1)
        r=base.solve_horizon(np.full(144,150.),scenario,weights,np.ones(144),6000.,6000.)
        z=r['g']+r['d']-r['c']
        value=float(r['g'].sum()+5*np.sum(weights[:,None]*np.maximum(scenario-z,0)))

```

```

        unit.append(dict(weights=weights.tolist(),expected=expected,actual=value,pass_check=abs(val
# Paired marginal-distribution identity explains why correlation is not the gain here.
identity_x=np.array([-20.,0.,13.]);identity_z=np.array([1.,3.,7.])
err=identity_x-identity_z
equality=float(abs(identity_z+5*np.maximum(err,0)-identity_x-5*np.where(err>=0,.8*err,-.2*err))
result=dict(future_perturbations=tests,weekly_choice_locked=bool(frozen),analytical_lp_tests=un
            pinball_identity_max_error=equality,
            pass_check=bool(all(r['pass_check'] for r in tests) and frozen and all(r['pass_chec
(HERE/'information_verification.json').write_text(json.dumps(result,ensure_ascii=False,indent=2
print(json.dumps(result,ensure_ascii=False,indent=2))
if not result['pass_check']:raise SystemExit(1)

if __name__=='__main__':main()

report_experiment.py
"""Publish the structural warm-up fix; report all window attempts, including losses."""
from pathlib import Path
import hashlib
import json
import shutil
import numpy as np
import pandas as pd
from rolling_window import HERE, SOURCE

LABELS={
    'uniform56':'56天等权','half_life7':'56天/半衰期7天','half_life14':'56天/半衰期14天',
    'half_life28':'56天/半衰期28天','online_primary':'四候选每周选择',
    'uniform28':'28天等权','uniform42':'42天等权','online_extended':'六候选每周选择'}

def collect():
    groups=[('原样本口径',HERE),('短窗口扩展',HERE/'extensions'/'short_windows'),
            ('排除回退残差',HERE/'sensitivity'/'valid_residuals')]
    rows=[]
    for group,folder in groups:
        x=pd.read_csv(folder/'comparison.csv')
        x['group']=group
        x['relative_folder']=[str((folder/'variants'/name).relative_to(HERE)).replace('\\','/') for
        rows.append(x)
    df=pd.concat(rows,ignore_index=True)
    baseline=float(df.loc[(df.group=='原样本口径')&(df.name=='uniform56'),'total_cost_yuan'].iloc[0]
    df['saving_vs_original_yuan']=baseline-df.total_cost_yuan
    df.to_csv(HERE/'all_comparisons.csv',index=False,encoding='utf-8-sig')
    return df,baseline

def main():
    roots={'primary':HERE,'valid_residuals':HERE/'sensitivity'/'valid_residuals',
           'short_windows':HERE/'extensions'/'short_windows'}
    bundle={key:json.loads((folder/'verification.json').read_text(encoding='utf-8')) for key,folder
    bundle['information']=json.loads((HERE/'information_verification.json').read_text(encoding='utf

```

```

if not all(bundle[key].get('pass') is True for key in roots) or bundle['information'].get('pass'):
    raise RuntimeError('All schedule, workbook and information checks must pass before publication')
df, baseline = collect()
old = json.loads((SOURCE.parent/'summary.json').read_text(encoding='utf-8'))
old_gap = abs(baseline - old['total_cost_yuan'])
if old_gap > 1e-6: raise RuntimeError('Original baseline was not exactly reproduced')
recommended = HERE/'sensitivity'/'valid_residuals'/'variants'/'uniform56'
base_days = pd.read_csv(HERE/'variants'/'uniform56'/'daily_summary.csv')
recommended_days = pd.read_csv(recommended/'daily_summary.csv')
if not base_days.date.equals(recommended_days.date):
    raise RuntimeError('Comparison dates do not match')
cost_difference = base_days.total_cost_yuan - recommended_days.total_cost_yuan
changed_dates = base_days.loc[cost_difference.abs() > 1e-6, 'date']
# This choice follows the first valid weekday forecast, not a tuned numeric cutoff.
for name in ('result2.xlsx', 'daily_summary.csv', 'schedule_detail.csv.gz'):
    shutil.copy2(recommended/name, HERE/name)
summary = json.loads((recommended/'summary.json').read_text(encoding='utf-8'))
summary.update(recommended_variant='sensitivity/valid_residuals/variants/uniform56',
               recommendation_reason='Use only genuine historical weekday forecasts: skip Jan1-15',
               baseline_total_cost_yuan=baseline, saving_yuan=baseline - summary['total_cost_yuan'],
               saving_percent=(1 - summary['total_cost_yuan']/baseline)*100,
               evaluation_start=str(base_days.date.iloc[0]), evaluation_end=str(base_days.date.iloc[-1]),
               cost_difference_last_date=str(changed_dates.iloc[-1]) if len(changed_dates) else None,
               weighted_window_conclusion='No robust improvement after controlling early fallback')
(HERE/'summary.json').write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding='utf-8')
# The root files above are the published first_residual=7 result. Preserve
# the first_residual=0 experiment metadata next to its variants, then make
# the root metadata describe the files users actually receive.
root_metadata_path = HERE/'run_metadata.json'
root_metadata = json.loads(root_metadata_path.read_text(encoding='utf-8'))
if root_metadata.get('first_residual') == 0:
    (HERE/'variants'/'run_metadata.json').write_text(
        json.dumps(root_metadata, ensure_ascii=False, indent=2), encoding='utf-8')
experiment_metadata = json.loads(
    (HERE/'sensitivity'/'valid_residuals'/'run_metadata.json').read_text(encoding='utf-8'))
candidate = next(c for c in experiment_metadata['candidates'] if c['name'] == summary['name'])
published_metadata = dict(
    metadata_scope='published_recommended_result',
    published_variant=summary['recommended_variant'],
    candidate=candidate,
    first_residual=experiment_metadata['first_residual'],
    initial_soc=summary['initial_energy_kwh'],
    terminal_soc=summary['final_energy_kwh'],
    horizon_days=experiment_metadata['horizon_days'],
    source_file_at_run=experiment_metadata['source_file'],
    source_sha256_at_run=experiment_metadata['source_sha256'],
    inherited_forecaster=experiment_metadata['inherited_forecaster'],
    python_at_run=experiment_metadata['python'])
root_metadata_path.write_text(
    json.dumps(published_metadata, ensure_ascii=False, indent=2), encoding='utf-8')
bundle['published_metadata_matches_summary'] = (
    published_metadata['first_residual'] == summary['first_residual_index'])

```

```

        and published_metadata['published_variant']==summary['recommended_variant']
        and published_metadata['candidate']['name']==summary['name'])
bundle['original_cost_reproduction_error']=old_gap
bundle['published_file_hashes']={name:hashlib.sha256((HERE/name).read_bytes()).hexdigest()
                                for name in ('result2.xlsx','daily_summary.csv','schedule_details.xlsx')}
bundle['published_files_identical_to_verified_variant']=all((HERE/n).read_bytes()==(recommended_files/n).read_bytes()
                    for n in bundle['published_file_hashes'])
bundle['pass']=(bundle['published_files_identical_to_verified_variant']
                and bundle['published_metadata_matches_summary'] and old_gap<=1e-6)
if not bundle['pass']:
    raise RuntimeError('Published result files or metadata differ from the verified variant')
(HERE/'verification_all.json').write_text(json.dumps(bundle,ensure_ascii=False,indent=2),encoding='utf-8')
repo=SOURCE.parents[2]
dependencies=[SOURCE,repo/'common'/'q2_base'/'q2_data.py']
dependencies += [repo/'data'/'附件'/'附件{i}.xlsx' for i in (1,2)]
(HERE/'source_fingerprints.json').write_text(json.dumps({str(p.relative_to(repo)):hashlib.sha256(p.read_bytes()).hexdigest()
    for p in dependencies}),encoding='utf-8')
# Expose contributions month by month, rather than only showing a winning total.
monthly=[]
for _,r in df.iterrows():
    daily=pd.read_csv(HERE/r.relative_folder/'daily_summary.csv')
    m=pd.DataFrame({'month':pd.to_datetime(daily.date).dt.month,
                   'saving_yuan':base_days.total_cost_yuan-daily.total_cost_yuan})
    for month,row in m.groupby('month').sum().iterrows():
        monthly.append(dict(group=r.group,name=r['name'],month=month,saving_yuan=row.saving_yuan))
pd.DataFrame(monthly).to_csv(HERE/'monthly_savings.csv',index=False,encoding='utf-8-sig')
clean=df[df.group=='排除回退残差'].set_index('name')
primary=df[df.group=='原样本口径'].set_index('name')
best_weight=primary.loc['half_life28']
clean_base=clean.loc['uniform56']
f28=pd.read_csv(HERE/'variants'/'half_life28'/'daily_summary.csv')
feb=(pd.to_datetime(base_days.date).dt.month==2)
feb_gain=float((base_days.total_cost_yuan-f28.total_cost_yuan)[feb].sum())
recommended_feb_gain=float(cost_difference[feb].sum())
recommended_march_gain=float(cost_difference[pd.to_datetime(base_days.date).dt.month==3].sum())
lines=['# 方案B滚动窗口优化结果','',
       '**结论：近期加权和缩短窗口没有显示稳定的额外收益；本轮推荐修正有效残差筛选，保留原56天等权基线。',
       f'原目录基线已精确复现：**{baseline:,.2f}元**。推荐方案为 **{summary["total_cost_yuan"]:.2f}元**。',
       '**费用统计期为2025年2月1日至12月31日，共334天；1月仅作共同预热，不计入下表费用。** 所有费用均为月费用。',
       '## 1. 保持原样本口径的窗口实验','',
       '| 方法 | 2—12月费用（元） | 较原基线节省（元，负数为变差） | 紧急电量（kWh） |','|---|---|---|']
for _,r in df[df.group!='排除回退残差'].iterrows():
    lines.append(f'| {LABELS[r["name"]]} | {r.total_cost_yuan:,.2f} | {r.saving_vs_original_yuan:,.2f} |')
lines += ['',f'28天半衰期在统计期内减少 {best_weight.saving_vs_original_yuan:,.2f} 元，其中2月减少 {best_weight.saving_vs_original_yuan:,.2f} 元。',
          '',f'固定候选的年度回测最佳属于回顾性比较，不能直接称为严格独立测试后的最优参数。两种方案中，',
          '',f'## 2. 为什么要修正有效残差','',
          f'原负载预测器在1月1—7日没有同星期几历史，使用附件1典型日回退；1月8日起才具有至少一次同星期几历史。',
          f'前7天残差是合法的因果预测误差，并非数值或计算错误。这里的“有效残差”指由具有同星期几历史的数据点计算出的残差。',
          f'因此采用由预测器可用性决定的边界：场景历史仅取 day >= 7（1月8日起），而非通过遍历历史数据点计算残差。',
          '',f'## 3. 排除前7天回退残差后的配对检验','',
          '| 方法 | 总费用（元） | 较有效残差等权基线增加（元） |','|---|---|---|']
for name,r in clean.iterrows():
    lines.append(f'| {LABELS[name]} | {r.total_cost_yuan:,.2f} | {r.total_cost_yuan-clean_base.loc[name]:,.2f} |')
lines += ['',f'在这组更一致的历史误差口径下，加权与四候选在线选择都没有优于等权。因此最终交付采用等权基线。']

```

```

'', '## 4. 推荐结果与变化解释', '',
'| 项目 | 原方案 (元) | 推荐方案 (元) |', '|---|---:|---:|',
f'| 计划购电费 | {primary.loc["uniform56"].planned_cost_yuan:,.2f} | {summary["planned_cost_yuan":,.2f]} |',
f'| 紧急购电费 | {primary.loc["uniform56"].emergency_cost_yuan:,.2f} | {summary["emergency_cost_yuan":,.2f]} |',
f'| 总费用 | {baseline:,.2f} | {summary["total_cost_yuan":,.2f]} |', '|',
'排除来自回退预测、与正常预测器口径不同的早期残差后，计划购电费用下降。紧急购电有所增
'', f'**这是启动阶段的改善。** 2月节省 {recommended_feb_gain:,.2f} 元，3月增加 {-recommended_mar_gain:,.2f} 元。',
'', '可使用根目录 [result2.xlsx](result2.xlsx)、[daily_summary.csv](daily_summary.csv) 查看结果。',
'', '## 5. 已完成核验', '',
'- 11组完整统计期方案均逐段复算48,096个时段，并反读各自Excel三表；检查能量守恒、效率、
'- 基线费用与原目录汇总一致；分时充放电导出按24段/4小时，表头按真实00:00—00:10起始时
'- 在2月1日、6月1日、11月30日修改当天及未来负载/光伏，重建预测、残差、评分及LP后，当前
'- 两组有解析解的加权LP费用分别为34,200与57,600元，与求解器一致。人工注入SOC、电价、缺
'- 核验范围见 `verification_all.json`、三个实验根目录的 `verification.json` 及 `information.json`。',
'', '## 6. 结论边界', '',
'这里检验的是继承方案B参数后的滚动执行与窗口规则。原预测参数当初如何选择仍依赖其研发记录。
'', '截断日依据预测器可用性确定，但推荐方案的费用改善来自同一年度回测，尚未经过另一个完整统计期。
'', '原方案中光伏点预测可能为负、1月预热使用附件1回退等其他口径保持不变，未把这些独立变量纳入模型。',
'',
(HERE/'结果说明.md').write_text('\n'.join(lines), encoding='utf-8')
print(json.dumps(summary, ensure_ascii=False, indent=2))

if __name__ == '__main__': main()

make_target_tables.py
"""为问题2 生成题目要求的「表1/表2/表3」数据结构。

题目要求（问题2）：
    • 按表1 格式给出 4 个指定日期的计划购电量（指定 10 分钟时段）+ 全天购电量/购电费
    • 按表2 格式给出储能设备在指定时间段的充放电量 + 0:00 与 24:00 储电量
    • 按表3 格式给出紧急购电量（全部紧急时间区间）

指定日期：2025-03-20、2025-06-21、2025-09-23、2025-12-21
指定 10 分钟时段：10:00-10:10、12:00-12:10、14:00-14:10、16:00-16:10、18:00-18:10、20:00-20:10
指定 4 小时时段：0:00-4:00、4:00-8:00、8:00-12:00、12:00-16:00、16:00-20:00、20:00-24:00

输入：Q2/results/schedule_detail.csv.gz
输出：Q2/results/target_table1.csv / target_table2.csv / target_table3.csv
      Q2/results/target_daily.csv
"""

from __future__ import annotations

import pathlib

import pandas as pd

REPO = pathlib.Path(__file__).resolve().parents[2]
RES = REPO / "Q2" / "results"
DETAIL = RES / "schedule_detail.csv.gz"

TARGETS = ["2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"]

```

```

SLOTS = [10, 12, 14, 16, 18, 20]          # 指定 10 分钟时段的起点小时
BLOCKS = 6                                # 6 个 4 小时块
T = 144

def intervals(values, tol=1e-7):
    """把逐时段量压缩成连续区间的（起止标签，合计）。"""
    start = None
    n = len(values)
    for t in range(n + 1):
        active = t < n and values[t] > tol
        if active and start is None:
            start = t
        elif not active and start is not None:
            a = f"{start * 10 // 60:02d}:{start * 10 % 60:02d}"
            b = "24:00" if t == n else f"{t * 10 // 60:02d}:{t * 10 % 60:02d}"
            yield f"{a}-{b}", float(values[start:t].sum())
            start = None

def main():
    det = pd.read_csv(DETAIL)
    print(f"读入 {DETAIL.name}: {det.shape[0]:,} 行, {det.date.nunique()} 天")

    r1, r2, r3, rd = [], [], [], []
    for date in TARGETS:
        g = det[det.date == date].reset_index(drop=True)
        if not len(g):
            print(f" ! {date} 无数据, 跳过")
            continue
        assert len(g) == T, f"{date} 段数 {len(g)} != {T}"

        # ---- 表1: 指定 10 分钟时段的计划购电量 ----
        for h in SLOTS:
            r = g.iloc[h * 6]
            r1.append(dict(date=date, span=f"{r['start']}-{r['end']}",
                           planned_kwh=float(r["g_kwh"])))

        # 全天合计
        r1.append(dict(date=date, span="全天",
                       planned_kwh=float(g["g_kwh"].sum())))

        # ---- 表2: 6 个 4 小时块的充放电量 + 首末储电量 ----
        for b in range(BLOCKS):
            blk = g.iloc[b * 24:(b + 1) * 24]
            a = f"{b * 4:02d}:00"
            bb = "24:00" if b == 5 else f"{(b + 1) * 4:02d}:00"
            r2.append(dict(date=date, span=f"{a}-{bb}",
                           charge_kwh=float(blk["c_kwh"].sum()),
                           discharge_kwh=float(blk["d_kwh"].sum())))
        r2.append(dict(date=date, span="0:00储电量",
                       charge_kwh=float(g.iloc[0]["energy_before_kwh"]),
                       discharge_kwh=None))

```

```

r2.append(dict(date=date, span="24:00储电量",
               charge_kwh=float(g.iloc[-1]["energy_after_kwh"]),
               discharge_kwh=None))

# ---- 表3: 全部紧急购电区间 ----
for span, amount in intervals(g["emergency_kwh"].to_numpy()):
    r3.append(dict(date=date, span=span, emergency_kwh=amount))

# ---- 日汇总 ----
rd.append(dict(
    date=date,
    planned_kwh_day=float(g["g_kwh"].sum()),
    planned_cost_yuan=float(g["planned_cost_yuan"].sum()),
    emergency_kwh_day=float(g["emergency_kwh"].sum()),
    emergency_cost_yuan=float(g["emergency_cost_yuan"].sum()),
    total_cost_yuan=float(g["total_cost_yuan"].sum()),
    energy_00=float(g.iloc[0]["energy_before_kwh"]),
    energy_24=float(g.iloc[-1]["energy_after_kwh"]),
))

pd.DataFrame(r1).to_csv(RES / "target_table1.csv", index=False, encoding="utf-8-sig")
pd.DataFrame(r2).to_csv(RES / "target_table2.csv", index=False, encoding="utf-8-sig")
pd.DataFrame(r3).to_csv(RES / "target_table3.csv", index=False, encoding="utf-8-sig")
pd.DataFrame(rd).to_csv(RES / "target_daily.csv", index=False, encoding="utf-8-sig")

print(f"\n已导出: ")
print(f" target_table1.csv  {len(r1):3d} 行")
print(f" target_table2.csv  {len(r2):3d} 行")
print(f" target_table3.csv  {len(r3):3d} 行 (紧急区间)")
print(f" target_daily.csv   {len(rd):3d} 行")
print()
print(pd.DataFrame(rd).to_string(index=False))

if __name__ == "__main__":
    main()

```

附录 E.3 问题三：预报融合与场景树多时点调整

```

q3_data.py

"""Read the original attachments; all power samples refer to interval ends."""
from pathlib import Path
import hashlib
import numpy as np
import openpyxl

HERE = Path(__file__).resolve().parents[1] / "results"
RAW = HERE.parents[1] / "data"
T, DT = 144, 1 / 6
HOURS = (0, 6, 12, 18)

def load_data():

```

```

files = [RAW / "附件" / f"附件{i}.xlsx" for i in (1, 2, 3)]
w = openpyxl.load_workbook(files[0], read_only=True, data_only=True)
a = np.array([r[1:4] for r in w.active.iter_rows(min_row=2, values_only=True)], float)
w.close()
w = openpyxl.load_workbook(files[1], read_only=True, data_only=True)
rows = [list(w[s].iter_rows(min_row=2, values_only=True))
         for s in ("小区负载", "光伏发电实际功率")]
dates = [str(r[0])[:10] for r in rows[0]]
assert [r[0] for r in rows[0]] == [r[0] for r in rows[1]]
load, pv = (np.array([r[1:145] for r in rr], float) for rr in rows)
w.close()
w = openpyxl.load_workbook(files[2], read_only=True, data_only=True)
rr = list(w.active.iter_rows(min_row=2, values_only=True))
assert [str(r[1]) for r in rr[:4]] == ["0:00", "6:00", "12:00", "18:00"]
external = np.array([r[2:26] for r in rr], float).reshape(365, 4, 24)
w.close()
assert load.shape == pv.shape == (365, T)
assert all(np.isfinite(x).all() for x in (a, load, pv, external))
assert min(load.min(), pv.min(), external.min()) >= 0
return dict(price=a[:, 0], load=load, pv=pv, external=external,
            dates=dates, source_sha256={str(p.relative_to(RAW)): hashlib.sha256(p.read_bytes())
                                         for p in files})

def label(t):
    return "24:00" if t == T else f"{t // 6:02d}:{t % 6 * 10:02d}"

def intervals(values, tol=1e-7):
    start = None
    for t in range(len(values) + 1):
        active = t < len(values) and values[t] > tol
        if active and start is None:
            start = t
        elif not active and start is not None:
            yield label(start) + "-" + label(t), float(values[start:t].sum())
            start = None

q3_forecast.py
"""Scheme B kernels plus causal, online fusion with the released PV forecasts.

Forecast arrays may be precomputed, but decision code only reads today's current
issue and complete historical rows. Actual future values are used for settlement.
"""
import numpy as np
from q3_data import T, HOURS

WINDOW = 56
FIRST_RESIDUAL = 7

def kernel(x, target, known, same_week=False, window=7):

```

```

    if same_week:
        ix = [target - 7 * j for j in range(1, 5) if 0 <= target - 7 * j < known]
        return x[ix].mean(0) if ix else None
    hi = min(target, known)
    return x[max(0, hi - window):hi].mean(0) if hi else None

def channel(x, target, known, same_week, drift):
    raw = kernel(x, target, known, same_week)
    if raw is None:
        raw = x[:known].mean(0) if known else np.zeros(T)
    errors = []
    for d in range(max(0, known - drift), known):
        f = kernel(x, d, d, same_week)
        if f is not None:
            errors.append(x[d] - f)
    return raw + np.mean(errors, axis=0) if errors else raw

def history_day(data, d):
    l = np.concatenate([channel(data['load'], j, d, True, 5) for j in (d, d + 1)])
    v = np.concatenate([channel(data['pv'], j, d, False, 28) for j in (d, d + 1)])
    return np.maximum(l, 0), np.maximum(v, 0)

def raw_issue(data, d, stage, history_pv):
    """Linear interpolation of +1..+24 h forecasts. Anchor is observed issue-time PV."""
    start = stage * 36
    flat = data['pv'].ravel()
    anchor = float(flat[d * T + start - 1]) if d * T + start else 0.0
    result = history_pv.copy()
    ends = np.arange(start + 1, start + T + 1)
    result[start:start + T] = np.interp(
        ends, np.arange(25) * 6 + start,
        np.r_[anchor, data['external'][d, stage]])
    return result

class Forecaster:
    def __init__(self, data):
        self.data = data
        n = len(data['load'])
        self.load = np.zeros((n, 2 * T))
        self.hist = np.zeros_like(self.load)
        self.raw = np.zeros((n, 4, 2 * T))
        self.fused = np.zeros_like(self.raw)
        self.alpha = np.ones((n, 4, 4))
        self.bias = np.zeros((n, 4, T))
        flat = data['pv'].ravel()
        for d in range(n):
            self.load[d], self.hist[d] = history_day(data, d)
            for stage in range(4):

```

```

start = stage * 36
self.raw[d, stage] = raw_issue(data, d, stage, self.hist[d])
out = self.raw[d, stage].copy()
# d-1 rows may contain next-day outcomes: train only fully mature rows <= d-2.
past = np.arange(max(FIRST_RESIDUAL, d - WINDOW), max(FIRST_RESIDUAL, d - 1))
if len(past) >= 7:
    y = np.array([flat[r * T + start:r * T + start + T] for r in past])
    ext = self.raw[past, stage, start:start + T]
    hist = self.hist[past, start:start + T]
    for b in range(4):
        sl = slice(b * 36, (b + 1) * 36)
        delta = ext[:, sl] - hist[:, sl]
        den = np.sum(delta * delta)
        a = np.clip(np.sum(delta * (y[:, sl] - hist[:, sl])) / den, 0, 1) if den >
        self.alpha[d, stage, b] = a
        old = a * ext[:, sl] + (1 - a) * hist[:, sl]
        bias = (y[:, sl] - old).mean(0) * len(past) / (len(past) + 14)
        self.bias[d, stage, sl] = bias
        dst = slice(start + b * 36, start + (b + 1) * 36)
        out[dst] = a * self.raw[d, stage, dst] + (1 - a) * self.hist[d, dst] + bias
    self.fused[d, stage] = np.maximum(out, 0)

def point(self, d, stage, source='fusion'):
    if source == 'history':
        v = self.hist[d]
    elif source == 'external':
        v = self.raw[d, stage]
    else:
        v = self.fused[d, stage]
    return (self.load[d] - v) / 6

def scenarios(self, d, stage, source='fusion'):
    past = np.arange(max(FIRST_RESIDUAL, d - WINDOW), d)
    if not len(past):
        return self.point(d, stage, source)[None, :T], np.array([], dtype=int)
    residual = np.array([(self.data['load'][r] - self.data['pv'][r]) / 6
                        - self.point(r, stage, source)[:T] for r in past])
    return self.point(d, stage, source)[None, :T] + residual, past

def diagnostics(self, start=31, end=365):
    import pandas as pd
    rows = []
    for d in range(start, end):
        for s, hour in enumerate(HOURS):
            sl = slice(s * 36, (s + 1) * 36)
            actual = self.data['pv'][d, sl]
            rows.append(dict(date=self.data['dates'][d], issue_hour=hour,
                            alpha_next_6h=self.alpha[d, s, 0],
                            history_mae_kw=np.mean(abs(self.hist[d, sl] - actual)),
                            external_mae_kw=np.mean(abs(self.raw[d, s, sl] - actual)),
                            fusion_mae_kw=np.mean(abs(self.fused[d, s, sl] - actual))))
    return pd.DataFrame(rows)

```

q3_model.py

```
"""Finite, nonanticipative scenario-tree LP, solved again at each allowed issue.
```

```
Only the root block is executed. Future nodes group historical forecast revisions,  
never future realized loads. Stage controls are literally shared variables.
```

```
"""
```

```
from dataclasses import dataclass  
import numpy as np  
from scipy.optimize import linprog  
from scipy.sparse import coo_matrix  
from q3_data import T
```

```
ETA = 0.9
```

```
E_MIN, E_MAX, E_INIT = 1200., 10800., 6000.
```

```
CAP = 5000 / 6
```

```
@dataclass
```

```
class Node:
```

```
    stage: int
```

```
    stop: int
```

```
    members: np.ndarray
```

```
    parent: int | None
```

```
    q: np.ndarray | None = None
```

```
    c: np.ndarray | None = None
```

```
    d: np.ndarray | None = None
```

```
    e: np.ndarray | None = None
```

```
def split_observable(features, members):
```

```
    """Deterministic binary k-means on newly observable forecast revisions."""
```

```
    if len(members) < 6:
```

```
        return [members]
```

```
    x = features[members]
```

```
    j = int(np.argmax(x.var(0)))
```

```
    if x[:, j].std() < 1e-5:
```

```
        return [members]
```

```
    centers = x[[np.argmin(x[:, j]), np.argmax(x[:, j])]].copy()
```

```
    prev = None
```

```
    for _ in range(12):
```

```
        group = ((x - centers[1]) ** 2).sum(1) < ((x - centers[0]) ** 2).sum(1)
```

```
        if min(group.sum(), (~group).sum()) < 3:
```

```
            order = np.argsort(x[:, j], kind='stable')
```

```
            return [members[order[:len(order)//2]], members[order[len(order)//2:]]]
```

```
        if prev is not None and np.array_equal(group, prev):
```

```
            break
```

```
        centers = np.array([x[~group].mean(0), x[group].mean(0)])
```

```
        prev = group
```

```
    return [members[~group], members[group]]
```

```
def make_tree(forecaster, past, stage, stages, source):
```

```

remaining = [s for s in stages if s >= stage]
nodes = [Node(stage, (remaining[1] * 36 if len(remaining) > 1 else T),
              np.arange(max(len(past), 1)), None)]
parents = [0]
for k, s in enumerate(remaining[1:], 1):
    stop = remaining[k + 1] * 36 if k + 1 < len(remaining) else T
    previous = remaining[k - 1]
    features = np.array([forecaster.point(r, s, source)[s * 36:]
                        - forecaster.point(r, previous, source)[s * 36:] for r in past])
    children = []
    for parent in parents:
        groups = split_observable(features, nodes[parent].members) if len(past) else [nodes[parent].members]
        for members in groups:
            children.append(len(nodes))
            nodes.append(Node(s, stop, members, parent))
    parents = children
return nodes

class LP:
    def __init__(self):
        self.cost, self.bounds = [], []
        self.er, self.ec, self.ev, self.eb = [], [], [], []
        self.ur, self.uc, self.uv, self.ub = [], [], [], []

    def var(self, n, cost=0., lo=0., hi=None):
        ids = np.arange(len(self.cost), len(self.cost) + n)
        self.cost.extend(np.broadcast_to(cost, n).tolist())
        self.bounds.extend([(lo, hi)] * n)
        return ids

    def eq(self, terms, rhs=0.):
        row = len(self.eb)
        for col, val in terms:
            self.er.append(row); self.ec.append(int(col)); self.ev.append(val)
        self.eb.append(rhs)

    def le(self, terms, rhs):
        row = len(self.ub)
        for col, val in terms:
            self.ur.append(row); self.uc.append(int(col)); self.uv.append(val)
        self.ub.append(rhs)

    def solve(self):
        n = len(self.cost)
        eq = coo_matrix((self.ev, (self.er, self.ec)), shape=(len(self.eb), n)).tocsr()
        ub = coo_matrix((self.uv, (self.ur, self.uc)), shape=(len(self.ub), n)).tocsr()
        res = linprog(self.cost, A_ub=ub, b_ub=self.ub, A_eq=eq, b_eq=self.eb,
                      bounds=self.bounds, method='highs')
        if not res.success:
            raise RuntimeError(res.message)
        return res, eq, ub

```

```

def solve(forecaster, day, stage, stages, energy, day_start_energy,
         original=None, source='fusion', deterministic=False, branch=True):
    price = forecaster.data['price']
    point = forecaster.point(day, stage, source)
    scenarios, past = forecaster.scenarios(day, stage, source)
    if deterministic:
        scenarios, past = point[None, :T], np.array([], dtype=int)
    nodes = make_tree(forecaster, past, stage, stages, source)
    if not branch:
        # Preserve update times but assume no future information gain.
        remaining = [s for s in stages if s >= stage]
        nodes = [Node(s, remaining[k + 1] * 36 if k + 1 < len(remaining) else T,
                     np.arange(len(scenarios)), k - 1 if k else None)
                 for k, s in enumerate(remaining)]
    S = len(scenarios)
    lp = LP()
    base = lp.var(T, price) if original is None else None
    raw_overlap = 0
    for ni, node in enumerate(nodes):
        start, stop = node.stage * 36, node.stop
        n = stop - start
        prob = len(node.members) / S
        node.q = lp.var(n)
        # Tiny throughput tie-breaker only; yuan scale is < 0.1 per year.
        node.c = lp.var(n, prob * 1e-9, hi=CAP)
        node.d = lp.var(n, prob * 1e-9, hi=CAP)
        node.e = lp.var(n, lo=E_MIN, hi=E_MAX)
        if node.stage > 0:
            up = lp.var(n, prob * 1.5 * price[start:stop])
            down = lp.var(n, -prob * .5 * price[start:stop])
            old_change = np.zeros(2 * T)
            if node.stage != stage and len(past):
                old_change = np.mean([forecaster.point(past[s], node.stage, source)
                                     - forecaster.point(past[s], stage, source)
                                     for s in node.members], axis=0)
            node_point = point + old_change
            for j, t in enumerate(range(start, stop)):
                terms = [(node.q[j], 1)]
                if node.stage > 0:
                    terms += [(up[j], -1), (down[j], 1)]
                if base is not None:
                    terms.append((base[t], -1))
                lp.eq(terms, 0 if base is not None else original[t])
                terms = [(node.e[j], 1), (node.c[j], -ETA), (node.d[j], 1 / ETA)]
                if j:
                    terms.append((node.e[j - 1], -1))
                elif node.parent is not None:
                    terms.append((nodes[node.parent].e[-1], -1))
                lp.eq(terms, energy if j == 0 and node.parent is None else 0)
                lp.le([(node.q[j], -1), (node.d[j], -1), (node.c[j], 1)], -node_point[t])

```

```

emergency = lp.var(len(node.members) * n, np.tile(5 * price[start:stop] / S, len(node.members)))
for k, s in enumerate(node.members):
    for j, t in enumerate(range(start, stop)):
        lp.le([(node.q[j], -1), (node.d[j], -1), (node.c[j], 1),
                (emergency[k * n + j], -1)], -scenarios[s, t])
if stop == T:
    if day == len(forecaster.data['load']) - 1:
        lp.eq([(node.e[-1], 1)], E_INIT)
    else:
        # Tomorrow is a deterministic value approximation, conditional on the node's forecast
        q = lp.var(T, prob * price)
        c = lp.var(T, prob * 1e-9, hi=CAP)
        d = lp.var(T, prob * 1e-9, hi=CAP)
        e = lp.var(T, lo=E_MIN, hi=E_MAX)
        for t in range(T):
            lp.eq([(e[t], 1), (c[t], -ETA), (d[t], 1 / ETA),
                    (e[t - 1] if t else node.e[-1], -1)])
            lp.le([(q[t], -1), (d[t], -1), (c[t], 1)], -node_point[T + t])
            lp.eq([(e[-1], 1)], day_start_energy)
res, eq, ub = lp.solve()
root = nodes[0]
q, c, d, e = (res.x[ids].copy() for ids in (root.q, root.c, root.d, root.e))
raw_overlap = int(np.sum((c > 1e-7) & (d > 1e-7)))
# This preserves internal SOC and increases useful supply; no disposal upper bound.
remove = np.minimum(c, d / ETA**2)
c -= remove; d -= ETA**2 * remove
return dict(original=res.x[base].copy() if base is not None else original.copy(),
            q=q, c=c, d=d, E=e, stop=root.stop,
            objective=float(res.fun), nodes=len(nodes), scenarios=S,
            raw_overlap=raw_overlap,
            lp_eq_max=float(np.max(np.abs(eq @ res.x - lp.lb))),
            lp_ineq_max=float(max(0., np.max(ub @ res.x - lp.ub))),
            tree=[dict(stage=n.stage, stop=n.stop, parent=n.parent,
                        historical_days=past[n.members].tolist() if len(past) else [],
                        probability=len(n.members) / S) for n in nodes])

```

run_q3.py

"""Run Q3 and comparable ablations. Outputs stay in this directory.

```

python run_q3.py # full-year solution and all comparisons
python run_q3.py --mode all --days 4 --out smoke
"""
from pathlib import Path
from datetime import datetime, timedelta
import argparse
import json
import os
import platform
import sys
import time
import numpy as np
import pandas as pd

```

```

from q3_data import HERE, T, load_data, label, intervals
from q3_forecast import Forecaster
from q3_model import solve, E_INIT

# 附件5 result3.xlsx 模板（用于取原表头，保证导出与模板逐列一致）
_ETA_DIR = Path(__file__).resolve().parents[2] / "common" / "efficiency"
if str(_ETA_DIR) not in sys.path:
    sys.path.insert(0, str(_ETA_DIR))
from template_layout import TPL3 as TPL # noqa: E402

MODES = {
    'B_aligned': ('history', (0,)),
    'only_0': ('fusion', (0,)),
    'at_0_6': ('fusion', (0, 1)),
    'at_0_6_12': ('fusion', (0, 1, 2)),
    'all': ('fusion', (0, 1, 2, 3)),
}

def simulate(f, mode, end=365):
    source, stages = MODES[mode]
    price = f.data['price']
    energy = E_INIT
    records, daily, trees = [], [], {}
    started = time.time()
    for day in range(31, end):
        date = (datetime(2025, 1, 1) + timedelta(days=day)).strftime('%Y-%m-%d')
        day_start = energy
        original = None
        q = np.zeros(T); c = np.zeros(T); d = np.zeros(T); E = np.zeros(T)
        issues = np.zeros(T, dtype=int); forecast = np.zeros(T)
        worst_eq = worst_ineq = 0.
        max_nodes = 0
        overlap = 0
        for stage in stages:
            r = solve(f, day, stage, stages, energy, day_start, original, source)
            original = r['original']
            start, stop = stage * 36, r['stop']
            q[start:stop], c[start:stop], d[start:stop], E[start:stop] = r['q'], r['c'], r['d'], r[
            energy = E[stop - 1]
            issues[start:stop] = stage * 6
            forecast[start:stop] = f.point(day, stage, source)[start:stop]
            worst_eq = max(worst_eq, r['lp_eq_max'])
            worst_ineq = max(worst_ineq, r['lp_ineq_max'])
            max_nodes = max(max_nodes, r['nodes'])
            overlap += r['raw_overlap']
            if day in (31, 78, 171, 265, 354) and stage == 0:
                trees[date] = r['tree']
        actual = (f.data['load'][day] - f.data['pv'][day]) / 6
        z = q + d - c
        emergency = np.maximum(actual - z, 0)
        dumped = np.maximum(z - actual, 0)

```

```

up, down = np.maximum(q - original, 0), np.maximum(original - q, 0)
planned_cost = price * original
added_cost, cancellation_refund, breach_cost = 1.5 * price * up, price * down, .5 * price *
adjustment_cost = added_cost - cancellation_refund + breach_cost
emergency_cost = 5 * price * emergency
total = planned_cost + adjustment_cost + emergency_cost
before = np.r_[day_start, E[:-1]]
for t in range(T):
    records.append(dict(date=date, slot=t, start=label(t), end=label(t + 1),
                        issue_hour=int(issues[t]), price_yuan_per_kwh=price[t],
                        original_kwh=original[t], adjusted_kwh=q[t], increase_kwh=up[t], decrease_kwh=down[t],
                        charge_kwh=c[t], discharge_kwh=d[t], energy_before_kwh=before[t], energy_kwh=energy[t],
                        load_kwh=f.data['load'][day,t]/6, pv_kwh=f.data['pv'][day,t]/6,
                        forecast_net_kwh=forecast[t], net_supply_kwh=z[t], emergency_kwh=emergency[t],
                        planned_cost_yuan=planned_cost[t], increase_cost_yuan=added_cost[t],
                        cancellation_refund_yuan=cancellation_refund[t], breach_cost_yuan=breach_cost[t],
                        adjustment_net_yuan=adjustment_cost[t], emergency_cost_yuan=emergency_cost[t],
                        total_cost_yuan=total[t],
                        energy_start_kwh=day_start, energy_end_kwh=energy[t],
                        lp_eq_max=worst_eq, lp_ineq_max=worst_ineq, max_tree_nodes=max_nodes,
                        raw_simultaneous_slots=overlap))
daily.append(dict(date=date, planned_cost_yuan=planned_cost.sum(), increase_cost_yuan=added_cost.sum(),
                  cancellation_refund_yuan=cancellation_refund.sum(), breach_cost_yuan=breach_cost.sum(),
                  adjustment_net_yuan=adjustment_cost.sum(), emergency_cost_yuan=emergency_cost.sum(),
                  total_cost_yuan=total.sum(), original_kwh=original.sum(), adjusted_kwh=q.sum(),
                  increase_kwh=up.sum(), decrease_kwh=down.sum(), emergency_kwh=emergency.sum(),
                  dump_kwh=dumped.sum(), charge_kwh=c.sum(), discharge_kwh=d.sum(),
                  energy_start_kwh=day_start, energy_end_kwh=energy,
                  lp_eq_max=worst_eq, lp_ineq_max=worst_ineq, max_tree_nodes=max_nodes,
                  raw_simultaneous_slots=overlap))
if (day - 31) % 30 == 0 or day == end - 1:
    print(f'{mode:12s} {date} cost={sum(x["total_cost_yuan"] for x in daily)/1e4:.2f} wan;
detail, days = pd.DataFrame(records), pd.DataFrame(daily)
sums = days.select_dtypes(include='number').sum().to_dict()
summary = {k:float(sums[k]) for k in ('planned_cost_yuan','increase_cost_yuan','cancellation_refund_yuan',
    'breach_cost_yuan','adjustment_net_yuan','emergency_cost_yuan','total_cost_yuan',
    'original_kwh','adjusted_kwh','increase_kwh','decrease_kwh','emergency_kwh','dump_kwh')}
summary.update(mode=mode, source=source, update_hours=[s*6 for s in stages], days=len(days),
               initial_energy_kwh=float(days.iloc[0].energy_start_kwh),
               final_energy_kwh=float(days.iloc[-1].energy_end_kwh), seconds=time.time()-start_time)
return detail, days, summary, trees

def export_excel(detail, days, path):
    import openpyxl
    from openpyxl.styles import Font, Alignment
    from openpyxl.comments import Comment
    import sys as _sys
    _sys.path.insert(0, str(Path(__file__).resolve().parents[2]
                           / "common" / "efficiency"))
    from template_layout import block_labels, circular, template_labels
    book = openpyxl.Workbook()
    book.remove(book.active)
    # 模板表头（左端点，首格 0:10-0:20）+ 环形映射 out[i]=data[(i+1)%144]
    headers = template_labels(TPL, '计划购电量')
    blocks6 = block_labels(TPL, '充放电量')

```

```

for name, col, cost in [('计划购电量','original_kwh','planned_cost_yuan'),
                        ('调整购电量','adjusted_kwh','adjustment_net_yuan')]:
    ws = book.create_sheet(name)
    ws.append(['日期\\时间']+headers+['全天购电量','全天购电费'])
    if col == 'adjusted_kwh':
        ws.cell(1,147).comment = Comment('此列为相对0点计划的净调整费用：1.5p×增购-0.5p×减购；表
for date, group in detail.groupby('date',sort=False):
    row = days.loc[days.date == date].iloc[0]
    vals = circular(group[col].to_numpy(float))
    ws.append([datetime.fromisoformat(date)]+[round(v,6) for v in vals]
              +[group[col].sum(),float(row[cost])])
ws.freeze_panes='B2'
ws.column_dimensions['A'].width=13
ws=book.create_sheet('充放电量')
ws.append(['日期','时间段','充电量','放电量','时刻','储电量'])
for date, group in detail.groupby('date',sort=False):
    for b in range(6):
        g=group.iloc[b*24:(b+1)*24]
        ws.append([datetime.fromisoformat(date) if b==0 else None,
                    blocks6[b],g.charge_kwh.sum(),g.discharge_kwh.sum(),
                    '0:00' if b==0 else ('24:00' if b==1 else None),
                    group.iloc[0].energy_before_kwh if b==0 else (group.iloc[-1].energy_after_kwh
ws=book.create_sheet('紧急购电量')
ws.append(['日期','购电时间段','购电量'])
for date, group in detail.groupby('date',sort=False):
    blocks=list(intervals(group.emergency_kwh.to_numpy())) or [('-',0.)]
    for i,(span,amount) in enumerate(blocks):
        ws.append([datetime.fromisoformat(date) if i==0 else None,span,amount])
for ws in book:
    for cell in ws[1]:
        cell.font=Font(bold=True); cell.alignment=Alignment(horizontal='center')
    for row in ws.iter_rows(min_row=2):
        for cell in row:
            if isinstance(cell.value,datetime): cell.number_format='yyyy-mm-dd'
            elif isinstance(cell.value,(float,int)): cell.number_format='0.0000'
    if ws.max_column<10:
        for col in 'ABCDEF': ws.column_dimensions[col].width=18
book.save(path)

```

```

def target_tables(detail, days, out):
    targets=['2025-03-20','2025-06-21','2025-09-23','2025-12-21']
    rows1, rows2, rows3=[],[],[]
    for date in targets:
        g=detail[detail.date==date]
        if not len(g): continue
        for hour in (10,12,14,16,18,20):
            r=g.iloc[hour*6]
            rows1.append(dict(date=date,span=r['start']+'-'+r['end'],original_kwh=r.original_kwh,ad
for b in range(6):
    block=g.iloc[b*24:(b+1)*24]
    rows2.append(dict(date=date,span=label(b*24)+'-'+label((b+1)*24),charge_kwh=block.charge

```

```

        for span,amount in intervals(g.emergency_kwh.to_numpy()): rows3.append(dict(date=date,span=span,amount=amount))
pd.DataFrame(rows1).to_csv(out/'target_table1.csv',index=False,encoding='utf-8-sig')
pd.DataFrame(rows2).to_csv(out/'target_table2.csv',index=False,encoding='utf-8-sig')
pd.DataFrame(rows3).to_csv(out/'target_table3.csv',index=False,encoding='utf-8-sig')
days[days.date.isin(targets)].to_csv(out/'target_daily.csv',index=False,encoding='utf-8-sig')

def write_report(out):
    """Keep the readable result notes tied to the actual exported numbers."""
    comp=pd.read_csv(out/'comparison.csv')
    report_days=int(comp.iloc[0]['days'])
    last_date=(datetime(2025,2,1)+timedelta(days=report_days-1)).strftime('%Y年%m月%d日')
    model_doc=Path(os.path.relpath(HERE.parent/'建模说明.md',out)).as_posix()
    names={'B_aligned':'方案B框架同口径复算','only_0':'仅使用0点融合预报',
          'at_0_6':'0、6点','at_0_6_12':'0、6、12点','all':'0、6、12、18点（主方案）'}
    lines=['# 第三问结果说明','',
          f'结果期为2025年2月1日至{last_date}，共{report_days}天。采用在线预报融合、经验场景树LP和',
          '', '**费用口径**：取消部分退原费、另付50%违约金；费用按供电时段电价计算。原计划费、增购',
          '', '## 全年对照','',
          '| 方案 | 总费用（元） | 紧急购电量（kWh） |','|---|---:|---:|']
    for _,r in comp.iterrows():
        lines.append(f'| {names[r["mode"]]} | {r.total_cost_yuan:,.2f} | {r.emergency_kwh:,.2f} |')
    idx=comp.set_index('mode')
    if 'all' in idx.index:
        a=idx.loc['all']
        lines+=['','## 主方案费用分解','',
              '| 项目 | 金额（元） |','|---|---:|',
              f'| 原计划购电费 | {a.planned_cost_yuan:,.2f} |',
              f'| 增购费（1.5倍） | {a.increase_cost_yuan:,.2f} |',
              f'| 取消原费退款 | -{a.cancellation_refund_yuan:,.2f} |',
              f'| 取消违约金（50%） | {a.breach_cost_yuan:,.2f} |',
              f'| 净调整费 | {a.adjustment_net_yuan:,.2f} |',
              f'| 紧急购电费（5倍） | {a.emergency_cost_yuan:,.2f} |',
              f'| **总费用** | **{a.total_cost_yuan:,.2f}** |','',
              '净调整费已经包含增购费、退款和违约金，计算总额时不能重复叠加这三项。']
    if 'B_aligned' in idx.index:
        b=idx.loc['B_aligned']
        lines += ['','f'相对同口径方案B，主方案节省 **{b.total_cost_yuan-a.total_cost_yuan:,.2f}**',
              '', '## 是否需要每次都调整','',
              'for old,new in [('B_aligned','only_0'),('only_0','at_0_6'),('at_0_6','at_0_6_12')]:
                  if old in idx.index and new in idx.index:
                      delta=idx.loc[old].total_cost_yuan-idx.loc[new].total_cost_yuan
                      lines.append(f'- 从“{names[old]}”改为“{names[new]}”，全年减少费用 {delta:,.2f}')
    if 'at_0_6_12' in idx.index:
        delta=a.total_cost_yuan-idx.loc['at_0_6_12'].total_cost_yuan
        lines += ['','f'加入18点更新后，全年费用变化为 **{delta:+,.2f}元**。此差值很小，数据未显',
              '', '该结论限于本数据和算法：18点仍可能通过次日预报影响跨日储能，不能断言其永',
              '', '主结果 [result3.xlsx](result3.xlsx) 保留预设的四次更新方案；[0、6、12点结',
              'if (out/'target_daily.csv').exists():
                  target=pd.read_csv(out/'target_daily.csv')
                  lines += ['','## 题目指定日期','',
                          '| 日期 | 原计划费（元） | 净调整费（元） | 紧急电量（kWh） | 紧急费（元） | 总费']

```

```

        '|---|---:|---:|---:|---:|---:|---:|']
    for _,r in target.iterrows():
        lines.append(f'| {r.date} | {r.planned_cost_yuan:,.2f} | {r.adjustment_net_yuan:,.2f} |')
    lines += ['|', '表1的指定十分钟购电量、表2的六个四小时充放电量、表3的全部紧急时间区间，分别保
lines += ['|', '## 核验与解释边界', '|',
        '- 主方案及对照的初末SOC统一为6000 kWh，能量递推、上下限、功率、充放电互斥、跨日连续和
        '- `verify_q3.py` 另对三天四个发布时刻做未来信息扰动，反读Excel并独立核算费用；应以该
        '- 方案B的历史内核参数由既有方案继承，不能据本次因果执行检查声称原参数选择过程一定没有
        '- 取销量若不退还原费，同一主调度的费用会增加退款总额；该敏感性写入核验JSON。若更换结
        '- 主方案为有限经验场景上的LP加滚动执行，不宣称全年真实随机问题全局最优。',
        '|', '## 图表', '|',
        '|![费用与紧急电量对照](figures/cost_comparison.png)', '|',
        '|![四个指定日购电与储能](figures/target_dispatch.png)', '|',
        '|![发布预报与历史信息融合](figures/forecast_fusion.png)', '|']
    (out/'结果说明.md').write_text('\n'.join(lines),encoding='utf-8')

```

```

def main():
    ap=argparse.ArgumentParser()
    ap.add_argument('--mode',choices=list(MODES)+['suite'],default='suite')
    ap.add_argument('--days',type=int,default=334)
    ap.add_argument('--out',type=Path,default=HERE)
    args=ap.parse_args()
    out=args.out if args.out.is_absolute() else HERE/args.out
    out.mkdir(parents=True,exist_ok=True)
    data=load_data(); f=Forecaster(data)
    end=min(365,31+args.days)
    summaries=[]
    modes=MODES if args.mode=='suite' else [args.mode]
    for mode in modes:
        detail,days,summary,trees=simulate(f,mode,end)
        dest=out if mode=='all' else out/'comparisons'/mode
        dest.mkdir(parents=True,exist_ok=True)
        detail.to_csv(dest/'schedule_detail.csv',index=False,encoding='utf-8-sig')
        days.to_csv(dest/'daily_summary.csv',index=False,encoding='utf-8-sig')
        (dest/'summary.json').write_text(json.dumps(summary,ensure_ascii=False,indent=2),encoding='utf-8-sig')
        summaries.append(summary)
        from verify_q3 import check_schedule
        checks=check_schedule(detail,days,data,require_year_end=end==365)
        (dest/'verification.json').write_text(json.dumps(checks,ensure_ascii=False,indent=2),encoding='utf-8-sig')
        if not checks['pass']: raise RuntimeError(f'Independent verification failed: {mode}')
        if mode=='at_0_6_12':
            # A ready-to-review lower-frequency alternative, kept separate from the preset main policy
            export_excel(detail,days,dest/'result3.xlsx')
        if mode=='all':
            export_excel(detail,days,out/'result3.xlsx')
            target_tables(detail,days,out)
            (out/'scenario_trees.json').write_text(json.dumps(trees,ensure_ascii=False,indent=2),encoding='utf-8-sig')
    pd.DataFrame(summaries).to_csv(out/'comparison.csv',index=False,encoding='utf-8-sig')
    f.diagnostics(end=end).to_csv(out/'forecast_diagnostics.csv',index=False,encoding='utf-8-sig')
    import scipy
    metadata=dict(source_sha256=data['source_sha256'],python=platform.python_version(),numpy=np.__version__)

```

```

        residual_window_days=56,first_residual_day='2025-01-08',history_load='sw4/drift5'
        initialization='January battery idle at 6000 kWh; all reports start February 1 at
        end_condition='48h end equals current day start; Dec 31 ends at 6000',
        settlement='p*g0 + 1.5*p*max(q-g0,0) - 0.5*p*max(g0-q,0) + 5*p*emergency',
        execution='Only next issue block is committed; each delivery slot adjusted at mos
        method='Empirical binary scenario-tree LP; deterministic next-day tail; receding=
(out/'run_metadata.json').write_text(json.dumps(metadata,ensure_ascii=False,indent=2),encoding=
write_report(out)
print(pd.DataFrame(summaries)[['mode','total_cost_yuan','emergency_kwh']].to_string(index=False

if __name__=='__main__': main()

verify_q3.py
"""Independent physical, accounting, workbook and information-boundary checks."""
from pathlib import Path
import json
import numpy as np
import pandas as pd
from q3_data import HERE, T, load_data

def check_schedule(detail, daily, data, require_year_end=True):
    x=detail
    p=np.tile(data['price'],len(daily))
    g,q,c,d=(x[k].to_numpy() for k in ('original_kwh','adjusted_kwh','charge_kwh','discharge_kwh'))
    day_indices=(pd.to_datetime(x.date)-pd.Timestamp('2025-01-01')).dt.days.to_numpy()
    slots=x.slot.to_numpy(int)
    net=(data['load'][day_indices,slots]-data['pv'][day_indices,slots])/6
    emergency=np.maximum(net+c-d-q,0)
    down=np.maximum(g-q,0); up=np.maximum(q-g,0)
    # Independently use paid delivered base + breach + new purchases.
    cost=p*np.minimum(g,q)+.5*p*down+1.5*p*up+5*p*emergency
    before=x.energy_before_kwh.to_numpy(); after=x.energy_after_kwh.to_numpy()
    balance=q+d+x.emergency_kwh.to_numpy()-net-c-x.dump_kwh.to_numpy()
    sumcols=['planned_cost_yuan','increase_cost_yuan','cancellation_refund_yuan','breach_cost_yuan'
            'adjustment_net_yuan','emergency_cost_yuan','total_cost_yuan','original_kwh','adjusted
            'increase_kwh','decrease_kwh','emergency_kwh','dump_kwh','charge_kwh','discharge_kwh']
    summed=x.groupby('date',sort=False)[sumcols].sum().to_numpy()
    errs=dict(
        periods=int(len(x)), duplicate_date_slots=int(x.duplicated(['date','slot']).sum()),
        energy_min_kwh=float(min(after.min(),before.min())), energy_max_kwh=float(max(after.max(),b
        energy_recursion_max_error=float(np.max(abs(after-before-.9*c+d/.9))),
        chronological_continuity_max_error=float(np.max(abs(after[:-1]-before[1:]))),
        initial_energy_kwh=float(before[0]), final_energy_kwh=float(after[-1]),
        max_charge_kwh=float(c.max()), max_discharge_kwh=float(d.max()),
        min_flow_kwh=float(min(c.min(),d.min(),q.min(),g.min())),
        simultaneous_periods=int(((c>1e-7)&(d>1e-7)).sum()),
        energy_balance_max_error=float(abs(balance).max()),
        emergency_max_error=float(np.max(abs(emergency-x.emergency_kwh.to_numpy()))),
        settlement_per_slot_max_error=float(np.max(abs(cost-x.total_cost_yuan.to_numpy()))),
        daily_sum_max_error=float(np.max(abs(summed-daily[sumcols].to_numpy()))),

```

```

cost_recomputed_yuan=float(cost.sum()),
altered_past_periods=int(np.sum(x.issue_hour.to_numpy()>slots/6)),
midnight_block_plan_max_error=float(np.max(abs((q-g)[x.issue_hour.to_numpy()==0]))),
lp_eq_max=float(daily.lp_eq_max.max()),lp_ineq_max=float(daily.lp_ineq_max.max()),
alternative_no_refund_cost_same_schedule_yuan=float((cost+p*down).sum()))
errs['pass']=bool(
    len(x)==len(daily)*T and errs['duplicate_date_slots']==0 and
    errs['energy_min_kwh']>=1200-1e-6 and errs['energy_max_kwh']<=10800+1e-6 and
    errs['energy_recursion_max_error']<1e-6 and errs['chronological_continuity_max_error']<1e-6
    abs(before[0]-6000)<1e-6 and (not require_year_end or abs(after[-1]-6000)<1e-6) and
    max(c.max(),d.max())<=5000/6+1e-6 and errs['min_flow_kwh']>=-1e-6 and
    errs['simultaneous_periods']==0 and errs['energy_balance_max_error']<1e-6 and
    errs['emergency_max_error']<1e-6 and errs['settlement_per_slot_max_error']<1e-6 and
    errs['daily_sum_max_error']<1e-5 and errs['altered_past_periods']==0 and
    errs['midnight_block_plan_max_error']<1e-6 and errs['lp_eq_max']<1e-6 and errs['lp_ineq_max']<1e-6
return errs

def information_checks(data):
    from q3_forecast import Forecaster
    from q3_model import solve, make_tree
    base=Forecaster(data)
    tests=[]
    for day in (31,171,354):
        original=None; energy=6000.
        for stage in range(4):
            a=solve(base,day,stage,(0,1,2,3),energy,6000.,original)
            changed={k:(v.copy() if isinstance(v,np.ndarray) else v) for k,v in data.items()}
            # Includes all still-unrealized current-day values, all later days,
            # and all forecasts released strictly after the current issue.
            cut=day*T+stage*36
            changed['load'].ravel()[cut:]*=1.7
            changed['pv'].ravel()[cut:]+=321.
            changed['external'][day,stage+1:]+=777.
            changed['external'][day+1:]+=555.
            other=Forecaster(changed)
            b=solve(other,day,stage,(0,1,2,3),energy,6000.,original)
            prediction_error=float(np.max(abs(base.point(day,stage)[stage*36:]-other.point(day,stage*36:))))
            control_error=max(float(np.max(abs(a[k]-b[k]))) for k in ('original','q','c','d','E'))
            tests.append(dict(day=day,issue_hour=stage*6,prediction_max_change=prediction_error,
                             decision_max_change=control_error,pass_check=control_error<1e-7 and p
                             original=a['original']; energy=a['E'][-1])
        # Test zero-update identity: same optimization returns no adjustment.
        r=solve(base,171,0,(0,),6000.,6000.)
        identity=float(np.max(abs(r['q']-r['original'])))
        result=dict(future_perturbation_tests=tests, no_update_identity_max_error=identity,
                    pass_check=all(t['pass_check'] for t in tests) and identity<1e-7)
    return result

def _workbook_slot_order(headers):
    """Map result-table headers to internal slots, including template wraparound."""

```

```

slots=[]
for label in headers:
    try:
        start=str(label).split('-',1)[0].replace('+1','')
        hour,minute=map(int,start.split(':'))
        slots.append((hour*6+minute//10)%T)
    except (AttributeError,TypeError,ValueError):
        return []
return slots if len(slots)==T and sorted(slots)==list(range(T)) else []

def workbook_checks(path,detail):
    import openpyxl
    w=openpyxl.load_workbook(path,read_only=True,data_only=True)
    errors=[]
    for name,col in [('计划购电量','original_kwh'),('调整购电量','adjusted_kwh')]:
        s=w[name]
        slot_order=_workbook_slot_order([s.cell(1,col).value for col in range(2,146)])
        if not slot_order:
            w.close()
            return dict(max_roundtrip_error=float('inf'),pass_check=False,
                        errors=[f'{name}: invalid interval headers'])
        actual=np.array([r[1:145] for r in s.iter_rows(min_row=2,values_only=True)],float)
        expected=detail[col].to_numpy().reshape(-1,T)[: ,slot_order]
        errors.append(float(np.max(abs(actual-expected))))
    rows=list(w['充放电量'].iter_rows(min_row=2,values_only=True))
    charge=np.array([r[2] for r in rows]); discharge=np.array([r[3] for r in rows])
    errors.extend([float(abs(charge-detail.charge_kwh.to_numpy().reshape(-1,24).sum(1)).max()),
                  float(abs(discharge-detail.discharge_kwh.to_numpy().reshape(-1,24).sum(1)).max())])
    total_emergency=sum(float(r[2] or 0) for r in w['紧急购电量'].iter_rows(min_row=2,values_only=True))
    errors.append(abs(total_emergency-detail.emergency_kwh.sum()))
    w.close()
    # Workbook values are exported to six decimals; summing rounded blocks can
    # accumulate a few 1e-6 units, so use a rounding-aware tolerance.
    return dict(max_roundtrip_error=float(max(errors)),pass_check=bool(max(errors)<5.1e-5))

def main():
    data=load_data()
    d=pd.read_csv(HERE/'schedule_detail.csv'); days=pd.read_csv(HERE/'daily_summary.csv')
    result=check_schedule(d,days,data)
    result['information_boundary']=information_checks(data)
    result['workbook']=workbook_checks(HERE/'result3.xlsx',d)
    result['pass']=bool(result['pass'] and result['information_boundary']['pass_check'] and result[
    alternative=HERE/'comparisons'/ 'at_0_6_12'
    if (alternative/'result3.xlsx').exists():
        result['alternative_workbook']=workbook_checks(alternative/'result3.xlsx',pd.read_csv(alternative/'schedule_detail.csv'))
        result['pass']=bool(result['pass'] and result['alternative_workbook']['pass_check'])
    (HERE/'verification.json').write_text(json.dumps(result,ensure_ascii=False,indent=2),encoding='utf-8')
    print(json.dumps(result,ensure_ascii=False,indent=2))
    if not result['pass']: raise SystemExit(1)

```

```
if __name__=='__main__': main()
```

附录 E.4 问题四-2：波动电价下的日前调度

q42_model.py

```
"""Question 4-2: causal variable-price extension of the improved question 2.

Only the objective's prices change; the load/PV forecast, historical net-load
scenarios, physical constraints, and two-day horizon retain their Q2 meanings.
"""

from __future__ import annotations

from functools import lru_cache
from pathlib import Path
import sys

import numpy as np
import openpyxl
from scipy.optimize import linprog
from scipy.sparse import coo_matrix

HERE = Path(__file__).resolve().parents[1] / "results"
REPO = HERE.parents[2]
BASE_DIR = REPO / 'Q2' / 'results'
RAW = REPO / 'data' / '附件'
sys.path.insert(0, str(BASE_DIR.parent / "code"))
from rolling_window import Inputs as Q2Inputs, BY_NAME, base # noqa: E402
from run_experiment import export_workbook, label # noqa: E402

T, DT = 144, 1 / 6
PRICE_FLOOR = 0.001
FORECASTERS = ('previous_day', 'mean7', 'profile7_lastlevel', 'weekday4')

def read_prices():
    book = openpyxl.load_workbook(RAW / '附件4.xlsx', read_only=True, data_only=True)
    rows = list(book.active.values)
    book.close()
    expected = [(i // 6, i % 6 * 10) for i in range(1, 144)]
    actual = [(v.hour, v.minute) for v in rows[0][1:144]]
    if actual != expected or rows[0][144] != '0:00+1':
        raise ValueError('Attachment 4 time headers do not represent interval endpoints')
    dates = [r[0] for r in rows[1:]]
    price = np.asarray([r[1:145] for r in rows[1:]], float)
    if price.shape != (365, T) or not np.isfinite(price).all() or (price <= 0).any():
        raise ValueError('Expected 365 x 144 finite, positive prices')
    return dates, price

def price_forecast(price, target_day, cutoff, method, fallback):
    """Use observations strictly before cutoff, including for tomorrow's forecast."""
    if not 0 <= cutoff <= target_day:
```

```

        raise ValueError('Invalid forecast information boundary')
    if cutoff == 0:
        return np.asarray(fallback).copy()
    historic = price[:cutoff]
    if method == 'previous_day':
        value = historic[-1]
    elif method == 'mean7':
        value = historic[-7:].mean(axis=0)
    elif method == 'profile7_lastlevel':
        value = historic[-7:].mean(axis=0) + historic[-1].mean() - historic[-7:].mean()
    elif method == 'weekday4':
        indices = [target_day - 7 * j for j in range(1, 5)
                    if 0 <= target_day - 7 * j < cutoff]
        value = price[indices].mean(axis=0) if indices else historic[-7:].mean(axis=0)
    else:
        raise ValueError(method)
    return np.maximum(value, PRICE_FLOOR)

def select_forecaster(price, fallback):
    """Select once on January 8-31, before the February-December reporting period."""
    rows = []
    for method in FORECASTERS:
        errors = np.array([price[d] - price_forecast(price, d, d, method, fallback)
                           for d in range(7, 31)])
        rows.append(dict(method=method, training_start='2025-01-08',
                          training_end='2025-01-31', days=24,
                          rmse=float(np.sqrt(np.mean(errors ** 2))),
                          mae=float(np.mean(np.abs(errors)))))
    return min(rows, key=lambda r: r['rmse'])['method'], rows

class Inputs:
    def __init__(self, price=None, load=None, pv=None):
        self.q2 = Q2Inputs(load=load, pv=pv)
        dates, actual_price = read_prices()
        if dates != self.q2.dates:
            raise ValueError('Dates in attachments 2 and 4 differ')
        self.dates = dates
        self.price = actual_price if price is None else np.asarray(price, float)
        if self.price.shape != (365, T) or not np.isfinite(self.price).all() or (self.price <= 0).any():
            raise ValueError('Prices must be finite and positive')
        self.method, self.training_scores = select_forecaster(self.price, self.q2.price)
        self.price_point = np.array([price_forecast(self.price, d, d, self.method, self.q2.price)
                                     for d in range(365)])
        self.price_tomorrow = np.array([price_forecast(self.price, d+1, d, self.method, self.q2.price)
                                         for d in range(364)])
        self.price_residual = self.price - self.price_point

    def planning_data(self, day, mode):
        scenarios, weights, indices = self.q2.scenario_data(day, BY_NAME['uniform56'], first_residual=0)
        if day < 31:

```

```

        raise ValueError('Q4 price forecaster is selected on January; report starts February 1')
    raw_price_scenarios = self.price_point[day] + self.price_residual[indices]
    price_scenarios = np.maximum(raw_price_scenarios, PRICE_FLOOR)
    price_mean = weights @ price_scenarios
    if mode == 'mean_price':
        price_scenarios = np.broadcast_to(price_mean, scenarios.shape).copy()
    elif mode == 'known_today_price':
        price_mean = self.price[day].copy()
        price_scenarios = np.broadcast_to(price_mean, scenarios.shape).copy()
    elif mode != 'joint_price':
        raise ValueError(mode)
    net_plan = self.q2.point[day].copy()
    plan_price = price_mean.copy()
    if day < 364:
        net_plan = np.r_[net_plan, self.q2.tomorrow[day]]
        plan_price = np.r_[plan_price, self.price_tomorrow[day]]
    return dict(net_plan=net_plan, net_scenarios=scenarios, weights=weights,
                plan_price=plan_price, scenario_price=price_scenarios,
                indices=indices, clipped_prices=int((raw_price_scenarios < PRICE_FLOOR).sum()))

def solve_day(self, day, energy, mode):
    data = self.planning_data(day, mode)
    plan = solve_horizon(data['net_plan'], data['net_scenarios'], data['weights'],
                        data['plan_price'], data['scenario_price'], energy,
                        6000. if day == 364 else energy)
    return plan, data

@lru_cache(maxsize=64)
def matrices(K, S):
    """Price-independent LP structure, cached for repeated daily solves."""
    n = 4 * K + S * T
    er, ec, ev = [], [], []
    for k in range(K):
        er.extend([k] * 3)
        ec.extend([3*K+k, K+k, 2*K+k])
        ev.extend([1., -.9, 1/.9])
        if k:
            er.append(k); ec.append(3*K+k-1); ev.append(-1.)
    er.append(K); ec.append(4*K-1); ev.append(1.)
    aeq = coo_matrix((ev, (er, ec)), shape=(K+1, n)).tocsr()
    ur, uc, uv = [], [], []
    for k in range(K):
        ur.extend([k] * 3); uc.extend([k, K+k, 2*K+k]); uv.extend([-1., 1., -1.])
    for s in range(S):
        for t in range(T):
            row = K+s*T+t
            ur.extend([row] * 4)
            uc.extend([t, K+t, 2*K+t, 4*K+s*T+t])
            uv.extend([-1., 1., -1., -1.])
    aub = coo_matrix((uv, (ur, uc)), shape=(K+S*T, n)).tocsr()
    bounds = [(0., None)]*K + [(0., 5000/6)]*(2*K) + [(1200., 10800.)]*K + [(0., None)]*(S*T)

```

```

return aeq, aub, bounds

def solve_horizon(net_plan, net_scenarios, weights, plan_price, scenario_price, initial, terminal):
    """Expected planned cost + expected 5x-price emergency recourse, sparse LP."""
    K = len(net_plan)
    S = len(weights)
    if K not in (T, 2*T) or net_scenarios.shape != (S, T) or scenario_price.shape != (S, T):
        raise ValueError('Invalid LP dimensions')
    if np.any(plan_price <= 0) or np.any(scenario_price <= 0) or np.any(weights < 0) or not np.iscl:
        raise ValueError('Positive prices and normalized nonnegative probabilities required')
    objective = np.r_[plan_price, np.zeros(3*K), (5*weights[:, None]*scenario_price).ravel()]
    aeq, aub, bounds = matrices(K, S)
    beq = np.zeros(K+1); beq[0] = initial; beq[-1] = terminal
    result = linprog(objective, A_ub=aub, b_ub=-np.r_[net_plan, net_scenarios.ravel()],
                     A_eq=aeq, b_eq=beq, bounds=bounds, method='highs')
    if not result.success:
        raise RuntimeError(result.message)
    g, c, d, energy = [result.x[i*K:(i+1)*K].copy() for i in range(4)]
    raw_overlap = int(((c > 1e-7) & (d > 1e-7)).sum())
    # Same SOC-preserving projection as Q2. With free surplus disposal and
    # positive emergency prices, increasing supply cannot increase cost.
    overlap = np.minimum(c, d/.81)
    c -= overlap; d -= .81*overlap
    c[np.abs(c) < 1e-9] = 0.; d[np.abs(d) < 1e-9] = 0.
    z = g+d-c
    emergency = np.maximum(net_scenarios-z[:,T], 0.)
    projected_objective = float(plan_price@g + np.sum(5*weights[:, None]*scenario_price*emergency))
    if abs(projected_objective-result.fun) > 1e-4:
        raise RuntimeError('SOC-preserving projection changed the LP optimum beyond tolerance')
    return dict(g=g, c=c, d=d, E=energy, z=z, objective=projected_objective,
               raw_overlap=raw_overlap, solver_status=int(result.status))

run_q42.py

"""Reproduce Q4-2 with common Q2 January warm-up and actual-price settlement."""
from __future__ import annotations

import argparse
import hashlib
import json
import sys
import time

import numpy as np
import pandas as pd

from q42_model import (HERE, BASE_DIR, RAW, REPO, Inputs, BY_NAME, T,
                       export_workbook, label)

TOTAL_COLUMNS = ('planned_cost_yuan', 'emergency_cost_yuan', 'total_cost_yuan',
                  'g_kwh', 'c_kwh', 'd_kwh', 'emergency_kwh', 'dump_kwh')

```

```

def daily_table(detail):
    by_day = detail.groupby('date', sort=False)
    daily = by_day[list(TOTAL_COLUMNS)].sum()
    daily['energy_start_kwh'] = by_day.energy_before_kwh.first()
    daily['energy_end_kwh'] = by_day.energy_after_kwh.last()
    daily['emergency_slots'] = by_day.emergency_kwh.apply(lambda x: int((x > 1e-6).sum()))
    return daily.reset_index()

def make_day(inputs, day, plan, initial, mode, data=None):
    g, c, d, energy = (plan[k][:T] for k in ('g', 'c', 'd', 'E'))
    z = g+d-c
    actual = inputs.q2.actual[day]
    emergency = np.maximum(actual-z, 0)
    price = inputs.price[day]
    frame = pd.DataFrame(dict(date=inputs.dates[day].strftime('%Y-%m-%d'), slot=np.arange(T),
        start=[label(t) for t in range(T)], end=[label(t+1) for t in range(T)],
        price_yuan_per_kwh=price, price_point_forecast=inputs.price_point[day],
        objective_price_yuan_per_kwh=inputs.q2.price if data is None else data['plan_price'][:T],
        forecast_net_kwh=inputs.q2.point[day], actual_net_kwh=actual,
        g_kwh=g, c_kwh=c, d_kwh=d, energy_before_kwh=np.r_[initial, energy[:-1]],
        energy_after_kwh=energy, z_kwh=z, emergency_kwh=emergency,
        dump_kwh=np.maximum(z-actual, 0), planned_cost_yuan=price*g,
        emergency_cost_yuan=5*price*emergency, total_cost_yuan=price*(g+5*emergency)))
    frame['strategy'] = mode
    return frame

def common_warmup(inputs):
    """Retain the already optimized Q2's feasible January policy for every mode."""
    energy = 6000.
    days = []
    for day in range(31):
        plan, _, _ = inputs.q2.solve_day(day, energy, BY_NAME['uniform56'])
        frame = make_day(inputs, day, plan, energy, 'common_q2_january')
        # Price forecasts selected after January are not January decisions.
        frame['price_point_forecast'] = np.nan
        days.append(frame)
        energy = float(plan['E'][T-1])
    detail = pd.concat(days, ignore_index=True)
    reference = json.loads((BASE_DIR/'common_january.json').read_text(encoding='utf-8'))
    actual_soc = daily_table(detail).energy_end_kwh.to_numpy()
    if not np.allclose(actual_soc, [r['energy_end_kwh'] for r in reference], atol=1e-7, rtol=0):
        raise RuntimeError('The common January warm-up changed')
    return detail, energy

def repriced_q2(inputs):
    """Hold the improved Q2's decisions fixed; change only actual settlement prices."""
    frame = pd.read_csv(BASE_DIR/'schedule_detail.csv.gz')
    if len(frame) != 334*T:

```

```

        raise ValueError('Improved Q2 output is incomplete')
    frame['objective_price_yuan_per_kwh'] = frame.price_yuan_per_kwh
    frame['price_point_forecast'] = inputs.price_point[31:].ravel()
    frame['price_yuan_per_kwh'] = inputs.price[31:].ravel()
    frame['planned_cost_yuan'] = frame.price_yuan_per_kwh*frame.g_kwh
    frame['emergency_cost_yuan'] = 5*frame.price_yuan_per_kwh*frame.emergency_kwh
    frame['total_cost_yuan'] = frame.planned_cost_yuan+frame.emergency_cost_yuan
    frame['strategy'] = 'q2_repriced'
    return frame

def save_variant(name, detail, diagnostics=None):
    folder = HERE/'variants'/name
    folder.mkdir(parents=True, exist_ok=True)
    daily = daily_table(detail)
    if diagnostics is not None:
        daily = daily.merge(pd.DataFrame(diagnostics), on='date', validate='one_to_one')
    summary = dict(name=name, days=len(daily), report_start=daily.date.iloc[0],
                   report_end=daily.date.iloc[-1], initial_energy_kwh=float(daily.energy_start_kwh),
                   final_energy_kwh=float(daily.energy_end_kwh.iloc[-1]),
                   **{col: float(daily[col].sum()) for col in TOTAL_COLUMNS})
    summary['emergency_slots'] = int(daily.emergency_slots.sum())
    summary['emergency_days'] = int((daily.emergency_kwh > 1e-6).sum())
    summary['objective_assumption'] = ('Current-day prices available at 00:00; tomorrow prices fore
        if name == 'known_today_price' else 'Only historical observations available at 00:00')
    detail.to_csv(folder/'schedule_detail.csv.gz', index=False, encoding='utf-8-sig', compression='gzip')
    daily.to_csv(folder/'daily_summary.csv', index=False, encoding='utf-8-sig')
    (folder/'summary.json').write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding='utf-8-sig')
    export_workbook(detail, daily, folder/'result4-2.xlsx')
    return summary

def run_variant(inputs, name, initial):
    frames, diagnostics = [], []
    energy = initial
    started = time.perf_counter()
    cumulative_cost = 0.
    for day in range(31, 365):
        plan, data = inputs.solve_day(day, energy, name)
        frame = make_day(inputs, day, plan, energy, name, data)
        frames.append(frame)
        cumulative_cost += float(frame.total_cost_yuan.sum())
        diagnostics.append(dict(date=frame.date.iloc[0], sample_count=len(data['indices']),
                                max_history_day=inputs.dates[int(data['indices'][-1])].strftime('%Y-%m-%d'),
                                clipped_scenario_price_count=data['clipped_prices'],
                                horizon_expected_cost_yuan=plan['objective'], solver_status=plan['solver_status'],
                                raw_simultaneous_slots=plan['raw_overlap']))
        energy = float(plan['E'][T-1])
        if (day-31) % 60 == 0 or day == 364:
            print(f'{name:18s} {frame.date.iloc[0]} {cumulative_cost:,.2f} yuan '
                  f'{time.perf_counter()-started:.1f}s', flush=True)
    return save_variant(name, pd.concat(frames, ignore_index=True), diagnostics)

```

```

def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--primary', choices=('joint_price', 'known_today_price'), default='joint_price')
    args = parser.parse_args()
    inputs = Inputs()
    dependencies = [BASE_DIR.parent/'code'/'rolling_window.py', BASE_DIR.parent/'code'/'run_experiment.py',
                    BASE_DIR/'schedule_detail.csv.gz', BASE_DIR/'summary.json',
                    REPO/'common'/'q2_base'/'run_1439.py',
                    REPO/'common'/'q2_base'/'q2_data.py']
    dependencies += [RAW/f'附件{i}.xlsx' for i in (1, 2, 4)]
    fingerprints = {str(p.relative_to(REPO)): hashlib.sha256(p.read_bytes()).hexdigest() for p in dependencies}
    (HERE/'source_fingerprints.migrated.json').write_text(json.dumps(fingerprints, ensure_ascii=False))
    pd.DataFrame(inputs.training_scores).to_csv(HERE/'price_training_scores.csv', index=False, encoding='utf-8')
    price_report = dict(selected_forecaster=inputs.method, selection_completed='2025-01-31',
                        shape=list(inputs.price.shape), min=float(inputs.price.min()), max=float(inputs.price.max()),
                        mean=float(inputs.price.mean()), missing=int(np.isnan(inputs.price).sum()),
                        report_rmse=float(np.sqrt(np.mean((inputs.price[31:]-inputs.price_point[31:])**2))),
                        report_mae=float(np.mean(np.abs(inputs.price[31:]-inputs.price_point[31:]))))
    (HERE/'price_data_summary.json').write_text(json.dumps(price_report, indent=2), encoding='utf-8')
    print(json.dumps(price_report, indent=2), flush=True)
    np.savez_compressed(HERE/'forecast_archive.npz', price_actual=inputs.price,
                      price_point=inputs.price_point, price_tomorrow=inputs.price_tomorrow,
                      price_residual=inputs.price_residual, net_point=inputs.q2.point,
                      net_tomorrow=inputs.q2.tomorrow, net_residual=inputs.q2.residual)
    january, initial = common_warmup(inputs)
    january.to_csv(HERE/'january_warmup.csv.gz', index=False, encoding='utf-8-sig', compression='gzip')
    daily_table(january).to_csv(HERE/'january_summary.csv', index=False, encoding='utf-8-sig')
    if abs(initial-json.loads((BASE_DIR/'summary.json').read_text(encoding='utf-8'))['initial_energy_kwh']) > 1000:
        raise RuntimeError('Q2 and Q4 initial states differ')
    summaries = [save_variant('q2_repriced', repriced_q2(inputs))]
    for mode in ('mean_price', 'joint_price', 'known_today_price'):
        summaries.append(run_variant(inputs, mode, initial))
    comparison = pd.DataFrame(summaries)
    baseline = float(comparison.iloc[0].total_cost_yuan)
    comparison['saving_vs_repriced_q2_yuan'] = baseline-comparison.total_cost_yuan
    comparison['saving_percent'] = 100*comparison.saving_vs_repriced_q2_yuan/baseline
    comparison.to_csv(HERE/'comparison.csv', index=False, encoding='utf-8-sig')
    metadata = dict(primary_variant=args.primary, q2_source=str(BASE_DIR.relative_to(REPO)),
                    report_period=['2025-02-01', '2025-12-31'], days=334, first_residual_index=7,
                    scenario_window=56, scenario_weights='equal', horizon_days=2,
                    price_forecaster=inputs.method, price_training_period=['2025-01-08', '2025-01-31'],
                    price_floor=0.001, initial_energy_kwh=initial, year_end_energy_kwh=6000.,
                    january_policy='Common unchanged Q2 policy, from January 1 SOC 6000; repriced but excluded Q4',
                    price_information='Default: only days strictly before decision date. Known-today sensitivity analysis: only days strictly before decision date.',
                    settlement='Actual attachment 4 price times planned kWh + 5 times actual price times emergency price times',
                    second_day='Point net-load forecast and historical price forecast only; no second-day scenario',
                    primary_selection='Determined by the price-information assumption, not by retrospective analysis',
                    python=sys.version)
    (HERE/'run_metadata.json').write_text(json.dumps(metadata, ensure_ascii=False, indent=2), encoding='utf-8')
    for p in dependencies:

```

```

        if hashlib.sha256(p.read_bytes()).hexdigest() != fingerprints[str(p.relative_to(REPO))]:
            raise RuntimeError(f'Source file changed: {p}')
    print(comparison[['name', 'total_cost_yuan', 'saving_vs_repriced_q2_yuan']].to_string(index=False))

if __name__ == '__main__':
    main()

verify_q42.py
"""Independent raw-XLSX, physical and cash-accounting audit of Q4-2 exports.

No optimizer, forecast or settlement function is used for these checks.
"""
from __future__ import annotations

import hashlib
import json
from pathlib import Path
import sys

import numpy as np
import openpyxl
import pandas as pd

HERE = Path(__file__).resolve().parents[1] / "results"
REPO = HERE.parents[2]
RAW = REPO/'data'/'附件'
sys.path.insert(0, str(REPO/'Q2'/'code'))
from verify_window import check_workbook # independent Excel parser, no model imports

def raw_arrays():
    book = openpyxl.load_workbook(RAW/'附件2.xlsx', read_only=True, data_only=True)
    load_rows = list(book['小区负载'].iter_rows(min_row=2, values_only=True))
    pv_rows = list(book['光伏发电实际功率'].iter_rows(min_row=2, values_only=True))
    book.close()
    book = openpyxl.load_workbook(RAW/'附件4.xlsx', read_only=True, data_only=True)
    price_rows = list(book.active.iter_rows(min_row=2, values_only=True))
    book.close()
    expected_dates = list(pd.date_range('2025-01-01', '2025-12-31'))
    for rows in (load_rows, pv_rows, price_rows):
        if [pd.Timestamp(r[0]) for r in rows] != expected_dates:
            raise ValueError('Raw dates do not cover the year exactly')
    load, pv, price = [np.asarray([r[1:145] for r in rows], float)
                        for rows in (load_rows, pv_rows, price_rows)]
    if any(a.shape != (365, 144) or not np.isfinite(a).all() for a in (load, pv, price)):
        raise ValueError('Invalid raw data')
    return (load-pv)/6, price

def check_schedule(frame, daily, net, price, start_day=31, initial=8550., terminal=6000.):
    errors, metrics = [], {}

```

```

def check(name, passed, value=None):
    if value is not None:
        metrics[name] = value
    if not bool(passed):
        errors.append(name)

def near(name, x, y, tol=1e-5):
    difference = float(np.max(np.abs(np.asarray(x)-np.asarray(y))))
    check(name, np.isfinite(difference) and difference <= tol, difference)

end_day = 31 if start_day == 0 else 365
days = end_day-start_day
expected_dates = pd.date_range('2025-01-01', periods=365)[start_day:end_day].strftime('%Y-%m-%d')
check('row_count', len(frame) == days*144, int(len(frame)))
if len(frame) != days*144:
    return dict(pass_check=False, errors=errors, metrics=metrics)
check('date_order', frame.date.tolist() == np.repeat(expected_dates, 144).tolist())
check('slot_order', frame.slot.tolist() == np.tile(np.arange(144), days).tolist())
labels = [f'{t//6:02d}:{t%6*10:02d}' for t in range(145)]
check('interval_start', frame.start.tolist() == labels[:144]*days)
check('interval_end', frame.end.tolist() == labels[1:]*days)
check('daily_date_order', daily.date.tolist() == expected_dates.tolist())
g, c, d, eb, ea, p, n = (frame[k].to_numpy(float) for k in
    ('g_kwh', 'c_kwh', 'd_kwh', 'energy_before_kwh', 'energy_after_kwh',
    'price_yuan_per_kwh', 'actual_net_kwh'))
check('finite_values', all(np.isfinite(a).all() for a in (g,c,d,eb,ea,p,n)))
near('raw_price_max_error', p, price[start_day:end_day].ravel(), 1e-12)
near('raw_net_load_max_error', n, net[start_day:end_day].ravel())
near('soc_balance_max_error', ea, eb+.9*c-d/.9)
near('soc_continuity_max_error', eb[1:], ea[:-1])
near('initial_soc_error', eb[0], initial)
near('terminal_soc_error', ea[-1], terminal)
check('soc_lower_bound', min(eb.min(), ea.min()) >= 1200-1e-5, float(min(eb.min(), ea.min())))
check('soc_upper_bound', max(eb.max(), ea.max()) <= 10800+1e-5, float(max(eb.max(), ea.max())))
check('nonnegative_flows', min(g.min(), c.min(), d.min()) >= -1e-6)
check('charge_power', c.max()*6 <= 5000+1e-5, float(c.max()*6))
check('discharge_power', d.max()*6 <= 5000+1e-5, float(d.max()*6))
overlap = int(((c > 1e-6) & (d > 1e-6)).sum())
check('simultaneous_charge_discharge', overlap == 0, overlap)
z = g+d-c
emergency = np.maximum(n-z, 0.)
dump = np.maximum(z-n, 0.)
near('net_supply_error', frame.z_kwh, z)
near('emergency_error', frame.emergency_kwh, emergency)
near('surplus_error', frame.dump_kwh, dump)
near('actual_balance_error', z+frame.emergency_kwh, n+frame.dump_kwh)
check('point_forecast_floor', np.min(z-frame.forecast_net_kwh) >= -1e-5,
    float(np.min(z-frame.forecast_net_kwh)))
near('planned_cost_error', frame.planned_cost_yuan, p*g)
near('emergency_cost_error', frame.emergency_cost_yuan, 5*p*emergency)
near('total_cost_error', frame.total_cost_yuan, p*(g+5*emergency))

```

```

aggregate = frame.groupby('date', sort=False)
for col in ('planned_cost_yuan', 'emergency_cost_yuan', 'total_cost_yuan',
           'g_kwh', 'c_kwh', 'd_kwh', 'emergency_kwh', 'dump_kwh'):
    near('daily_'+col, daily[col], aggregate[col].sum().to_numpy(), 1e-4)
near('daily_initial_soc', daily.energy_start_kwh, aggregate.energy_before_kwh.first().to_numpy())
near('daily_final_soc', daily.energy_end_kwh, aggregate.energy_after_kwh.last().to_numpy())
if 'max_history_day' in daily:
    check('history_strictly_past', bool((daily.max_history_day < daily.date).all()))
    expected_count = np.minimum(56, np.arange(start_day, end_day)-7)
    near('scenario_count', daily.sample_count, expected_count, 0.)
    check('all_solvers_optimal', bool((daily.solver_status == 0).all()))
metrics['recomputed_total_cost_yuan'] = float(np.sum(p*(g+5*emergency)))
metrics['recomputed_emergency_kwh'] = float(emergency.sum())
return dict(pass_check=not errors, errors=errors, metrics=metrics)

def main():
    net, price = raw_arrays()
    metadata = json.loads((HERE/'run_metadata.json').read_text(encoding='utf-8'))
    initial = metadata['initial_energy_kwh']
    january = pd.read_csv(HERE/'january_warmup.csv.gz')
    january_days = pd.read_csv(HERE/'january_summary.csv')
    reports = dict(january=check_schedule(january, january_days, net, price, 0, 6000., initial), va
    for folder in sorted((HERE/'variants').iterdir()):
        if not folder.is_dir():
            continue
        detail = pd.read_csv(folder/'schedule_detail.csv.gz')
        daily = pd.read_csv(folder/'daily_summary.csv')
        report = dict(schedule=check_schedule(detail, daily, net, price, initial=initial),
                      workbook=check_workbook(folder/'result4-2.xlsx', detail))
        summary = json.loads((folder/'summary.json').read_text(encoding='utf-8'))
        report['summary_total_error'] = abs(summary['total_cost_yuan']-report['schedule']['metrics']
        report['pass_check'] = (report['schedule']['pass_check'] and report['workbook']['pass']
                               and report['summary_total_error'] <= 1e-4)
        reports['variants'][folder.name] = report
        print(folder.name, 'PASS' if report['pass_check'] else 'FAIL', flush=True)
    selected = HERE/'variants'/metadata['primary_variant']
    detail = pd.read_csv(selected/'schedule_detail.csv.gz')
    daily = pd.read_csv(selected/'daily_summary.csv')
    injections = {}
    for column, amount in [('price_yuan_per_kwh', .1), ('energy_after_kwh', 10.), ('emergency_kwh',
        broken = detail.copy()
        broken.loc[0, column] += amount
        injections[column] = not check_schedule(broken, daily, net, price, initial=initial)['pass_c
    reports['injected_errors_rejected'] = injections
    fingerprint_path = HERE/'source_fingerprints.migrated.json'
    if not fingerprint_path.exists():
        fingerprint_path = HERE/'source_fingerprints.json'
    fingerprints = json.loads(fingerprint_path.read_text(encoding='utf-8'))
    reports['source_files_unchanged'] = all(hashlib.sha256((REP0/p).read_bytes()).hexdigest() == h
    reports['pass_check'] = (reports['january']['pass_check'] and bool(reports['variants'])
                           and all(r['pass_check'] for r in reports['variants'].values()))

```

```

        and all(injections.values()) and reports['source_files_unchanged'])
    (HERE/'verification.json').write_text(json.dumps(reports, ensure_ascii=False, indent=2), encoding='utf-8')
    print('Overall', reports['pass_check'])
    if not reports['pass_check']:
        raise SystemExit(1)

if __name__ == '__main__':
    main()

check_information.py
"""Analytical, Q2-reduction and future-information tests for Q4-2."""
from __future__ import annotations

import json
import numpy as np

from q42_model import HERE, Inputs, solve_horizon, base, T

def main():
    inputs = Inputs()
    reports = {}
    # The costly-scarcity scenario changes which net-load quantile to cover.
    # Identical prices and loads across time rule out useful storage arbitrage.
    analytical = []
    weights = np.array([.5, .5])
    net = np.array([np.full(T, 100.), np.full(T, 200.)])
    for scenario_prices, expected in [([2., .1], 18720.), ([.1, 2.], 30240.)]:
        prices = np.repeat(np.array(scenario_prices)[: , None], T, axis=1)
        plan = solve_horizon(np.full(T, 100.), net, weights, weights@prices, prices, 6000., 6000.)
        error = abs(plan['objective']-expected)
        analytical.append(dict(scenario_prices=scenario_prices, expected_cost=expected,
                               computed_cost=plan['objective'], error=error, pass_check=error < 1e-6))
    reports['analytical_price_net_dependence'] = analytical
    data = inputs.planning_data(151, 'joint_price')
    common_price = np.tile(inputs.q2.price, 2)
    repeated_price = np.broadcast_to(inputs.q2.price, data['net_scenarios'].shape).copy()
    ours = solve_horizon(data['net_plan'], data['net_scenarios'], data['weights'],
                          common_price, repeated_price, 8550., 8550.)
    original = base.solve_horizon(data['net_plan'], data['net_scenarios'], data['weights'],
                                   common_price, 8550., 8550.)
    original_cost = common_price@original['g'] + np.sum(5*data['weights'][:,None]*repeated_price*
                                                       np.maximum(data['net_scenarios']-original['z'][:T], 0))
    error = float(abs(ours['objective']-original_cost))
    reports['constant_price_reduces_to_q2'] = dict(objective_difference=error, pass_check=error < 1e-6)
    perturbations = []
    for day in (31, 151, 333):
        changed_price = inputs.price.copy(); changed_price[day:] = changed_price[day:]*1.35+.07
        changed_load = inputs.q2.load.copy(); changed_load[day:] += 1234.
        changed_pv = inputs.q2.pv.copy(); changed_pv[day:] *= .2
        changed = Inputs(price=changed_price, load=changed_load, pv=changed_pv)

```

```

a = inputs.planning_data(day, 'joint_price')
b = changed.planning_data(day, 'joint_price')
changes = {key: float(np.max(np.abs(a[key]-b[key]))) for key in
            ('net_plan', 'net_scenarios', 'weights', 'plan_price', 'scenario_price', 'indices')}
first, _ = inputs.solve_day(day, 8550., 'joint_price')
second, _ = changed.solve_day(day, 8550., 'joint_price')
changes['decisions'] = max(float(np.max(np.abs(first[k]-second[k]))) for k in ('g','c','d',
passed = inputs.method == changed.method and max(changes.values()) < 1e-9
perturbations.append(dict(day=day, date=inputs.dates[day].strftime('%Y-%m-%d'),
                           max_changes=changes, forecaster_unchanged=inputs.method==changed.method,
                           pass_check=passed))
print('Future perturbation', day, passed, flush=True)
reports['future_perturbations'] = perturbations
# Even the known-today contrast cannot read tomorrow's actual prices.
day = 151
changed_price = inputs.price.copy(); changed_price[day+1:] *= 2
changed = Inputs(price=changed_price)
a, _ = inputs.solve_day(day, 8550., 'known_today_price')
b, _ = changed.solve_day(day, 8550., 'known_today_price')
difference = max(float(np.max(np.abs(a[k]-b[k]))) for k in ('g','c','d','E'))
reports['known_today_cannot_read_tomorrow'] = dict(max_decision_change=difference, pass_check=passed)
reports['pass_check'] = (all(r['pass_check'] for r in analytical+perturbations)
                        and reports['constant_price_reduces_to_q2']['pass_check']
                        and reports['known_today_cannot_read_tomorrow']['pass_check'])
(HERE/'information_verification.json').write_text(json.dumps(reports, ensure_ascii=False, indent=2))
print(json.dumps(reports, indent=2))
if not reports['pass_check']:
    raise SystemExit(1)

if __name__ == '__main__':
    main()

```

report_q42.py

"""Publish verified Q4-2 outputs, paper tables and inspected plot artifacts."""

from __future__ import annotations

import hashlib

import json

from pathlib import Path

import shutil

import sys

import numpy as np

import pandas as pd

HERE = Path(__file__).resolve().parents[1] / "results"

LABELS = dict(q2_repriced='原Q2按新电价结算', mean_price='历史预测: 平均电价',

joint_price='历史预测: 联合场景', known_today_price='当日电价已知')

TARGET_DATES = ('2025-03-20', '2025-06-21', '2025-09-23', '2025-12-21')

COLORS = dict(q2_repriced='#969696', mean_price='#C48436', joint_price='#277A8A',

known_today_price='#7960A1')

```

def time_label(slot):
    return f'{slot//6:02d}:{slot%6*10:02d}'

def target_tables(detail, daily, primary):
    dest = HERE/'target_days'; dest.mkdir(exist_ok=True)
    purchases, storage, emergencies = [], [], []
    lines = ['# 第4问4-2指定日期结果', '',
              f'当前发布方案: {LABELS[primary]}. 电量单位kWh, 费用单位元; 购电费按附件4实际电价结算。',
              '', '表内显示值四舍五入, 汇总由未舍入明细计算。', '']
    for date in TARGET_DATES:
        frame = detail.loc[detail.date == date].reset_index(drop=True)
        day = daily.loc[daily.date == date].iloc[0]
        lines += [f'## {date}', '', '表1 计划购电量', '',
                  '| 时间段 | 购电量 | 时间段 | 购电量 | 时间段 | 购电量 |',
                  '|---|---:|---|---:|---|---:|']
        for slots in ((60, 72, 84), (96, 108, 120)):
            cells = []
            for slot in slots:
                span = time_label(slot)+'—'+time_label(slot+1)
                value = float(frame.g_kwh.iloc[slot])
                cells += [span, f'{value:,.4f}']
                purchases.append(dict(date=date, interval=span, planned_grid_kwh=value))
            lines.append('| '+' | '.join(cells)+' |')
        lines += ['', f'全天计划购电量: {day.g_kwh:,.4f} kWh; 计划购电费: {day.planned_cost_yuan:,.4f}元; 紧急购电费: {day.emergency_cost_yuan:,.2f}元; 总购电费: {day.total_cost_yuan:,.2f}元;',
                  '', '表2 储能充放电电量', '',
                  '| 时间段 | 充电量 | 放电量 | 时间段 | 充电量 | 放电量 |',
                  '|---|---:|---:|---:|---:|---:|']
        for pair in ((0,1), (2,3), (4,5)):
            cells = []
            for block in pair:
                part = frame.iloc[block*24:(block+1)*24]
                span = time_label(block*24)+'—'+time_label((block+1)*24)
                charge, discharge = float(part.c_kwh.sum()), float(part.d_kwh.sum())
                cells += [span, f'{charge:,.4f}', f'{discharge:,.4f}']
                storage.append(dict(date=date, interval=span, charge_kwh=charge, discharge_kwh=discharge))
            lines.append('| '+' | '.join(cells)+' |')
        lines += ['', f'0:00储电量: {day.energy_start_kwh:,.4f} kWh; 24:00储电量: {day.energy_end_kwh:,.4f} kWh;',
                  '', '表3 全部紧急购电区间', '', '| 时间段 | 紧急购电量 |', '|---|---:|']
    e = frame.emergency_kwh.to_numpy()
    cursor = 0
    while cursor < 144:
        if e[cursor] <= 1e-6:
            cursor += 1; continue
        start = cursor
        while cursor < 144 and e[cursor] > 1e-6:
            cursor += 1
        amount = float(e[start:cursor].sum())
        span = time_label(start)+'—'+time_label(cursor)

```

```

        emergencies.append(dict(date=date, interval=span, emergency_kwh=amount))
        lines.append(f'| {span} | {amount:,.4f} |')
    lines += ['|', f'当日紧急购电量合计: {day.emergency_kwh:,.4f} kWh。', '|']
pd.DataFrame(purchases).to_csv(dest/'table1_grid.csv', index=False, encoding='utf-8-sig')
pd.DataFrame(storage).to_csv(dest/'table2_storage.csv', index=False, encoding='utf-8-sig')
pd.DataFrame(emergencies).to_csv(dest/'table3_emergency.csv', index=False, encoding='utf-8-sig')
daily.loc[daily.date.isin(TARGET_DATES)].to_csv(dest/'daily_summary.csv', index=False, encoding='utf-8-sig')
(HERE/'指定日期结果.md').write_text('\n'.join(lines), encoding='utf-8')

def main():
    verification = json.loads((HERE/'verification.json').read_text(encoding='utf-8'))
    information = json.loads((HERE/'information_verification.json').read_text(encoding='utf-8'))
    if not verification['pass_check'] or not information['pass_check']:
        raise RuntimeError('Physical, workbook and information checks must pass before publication')
    metadata = json.loads((HERE/'run_metadata.json').read_text(encoding='utf-8'))
    primary = metadata['primary_variant']
    folder = HERE/'variants'/primary
    comparison = pd.read_csv(HERE/'comparison.csv')
    costs = comparison.set_index('name')
    summary = json.loads((folder/'summary.json').read_text(encoding='utf-8'))
    for name in ('result4-2.xlsx', 'daily_summary.csv', 'schedule_detail.csv.gz'):
        shutil.copy2(folder/name, HERE/name)
    summary.update(primary_variant=primary,
        baseline_repriced_cost_yuan=float(costs.loc['q2_repriced'].total_cost_yuan),
        saving_vs_repriced_q2_yuan=float(costs.loc[primary].saving_vs_repriced_q2_yuan),
        saving_percent=float(costs.loc[primary].saving_percent),
        note='Primary follows the declared price-information model; mean-price contrast is slightly')
    (HERE/'summary.json').write_text(json.dumps(summary,ensure_ascii=False,indent=2),encoding='utf-8')
    detail = pd.read_csv(HERE/'schedule_detail.csv.gz')
    daily = pd.read_csv(HERE/'daily_summary.csv')
    baseline_detail = pd.read_csv(HERE/'variants'/q2_repriced/'schedule_detail.csv.gz')
    baseline_days = pd.read_csv(HERE/'variants'/q2_repriced/'daily_summary.csv')
    monthly_rows = []
    for name in comparison.name:
        frame = pd.read_csv(HERE/'variants'/name/'daily_summary.csv')
        changes = pd.DataFrame(dict(month=pd.to_datetime(frame.date).dt.month,
            cost_yuan=frame.total_cost_yuan,saving_yuan=baseline_days.total_cost_yuan-frame.total_cost_yuan))
        for month,row in changes.groupby('month').sum().iterrows():
            monthly_rows.append(dict(name=name,month=int(month),cost_yuan=row.cost_yuan,saving_yuan=row.saving_yuan))
    monthly = pd.DataFrame(monthly_rows)
    monthly.to_csv(HERE/'monthly_comparison.csv',index=False,encoding='utf-8-sig')
    target_tables(detail,daily,primary)
    data_summary = json.loads((HERE/'price_data_summary.json').read_text(encoding='utf-8'))
    joint_daily = pd.read_csv(HERE/'variants'/joint_price/'daily_summary.csv')
    clipped = int(joint_daily.clipped_scenario_price_count.sum())
    price_count = int(joint_daily.sample_count.sum()*144)
    joint, mean, baseline, known = [costs.loc[n] for n in ('joint_price','mean_price','q2_repriced')]
    lines = ['# 第4问4-2计算结果','',
        f'根目录发布方案: **{LABELS[primary]**。完整策略见 [result4-2.xlsx](result4-2.xlsx)。',
        '', '**统计期为2025年2月1日至12月31日334天**，所有方案2月初储电量8550 kWh、年末6000 kWh。1月',
        '', '## 1. 相同实际电价下的比较','',

```

```

| 方案 | 计划购电费 (元) | 紧急购电费 (元) | 总费用 (元) | 较原Q2重新结算节省 (元) |',
'|---|---:|---:|---:|---:|']
for name,row in costs.iterrows():
    lines.append(f'| {LABELS[name]} | {row.planned_cost_yuan:,.2f} | {row.emergency_cost_yuan:,.2f} | {row.total_cost_yuan:,.2f} | {row.saving_vs_repriced_q2_yuan:,.2f} |')
lines += ['','联合场景模型比重新结算的Q2节省 **{joint.saving_vs_repriced_q2_yuan:,.2f}元 ({joint.saving_vs_repriced_q2_yuan:,.2f}元) 计划费增加 {joint.planned_cost_yuan-baseline.planned_cost_yuan:,.2f} 元, 紧急费减少 {baseline.emergency_cost_yuan-joint.emergency_cost_yuan:,.2f} 元, 合计费用下降。',
'', f'紧急电量从 {baseline.emergency_kwh:,.2f} kWh降至 {joint.emergency_kwh:,.2f} kWh, 减少 {baseline.emergency_kwh-joint.emergency_kwh:,.2f} kWh; 剩余弃置电量从 {baseline.dump_kwh:,.2f} kWh增至 {joint.dump_kwh:,.2f} kWh。它体现更保守的高价缺电覆盖, 不能只看紧急电量而忽略多买和弃电。',
'', '原优化Q2在附件1固定电价下为14,389,135.08元。附件4改变了计费条件, 不能直接把本问与该数字对比。',
'', '## 2. 哪些结论成立, 哪些不能过度解释','',
f'- 平均电价简化模型费用 **{mean.total_cost_yuan:,.2f}元**, 比联合场景模型低 **{joint.total_cost_yuan-known.total_cost_yuan:,.2f}元**, 联合场景在本年度没有显示稳定优于均价近似的证据; 若优先考虑实施简单, 平均电价方案是完整可用。',
f'- 默认主文件采用联合场景, 是因为它对应所声明的电价与净负载同时不确定的模型, 并非宣称它是这。',
f'- 当日电价已知对照费用 **{known.total_cost_yuan:,.2f}元**。它比联合场景低 {joint.total_cost_yuan-known.total_cost_yuan:,.2f} 元, 体现本次回测中更充分价格信息的价值。',
f'- 已知电价对照仍使用预测负载和光伏, 不读取未来真实净负载; 48小时窗口中次日价格也只能预测。',
f'- 月度节省有正有负, 结果不证明任意月份、其他年份均能改善, 也不证明全局最优策略。',
'', '## 3. 电价模型及数据检查','',
f'四个候选仅用1月8—31日的24个历史预测日比较RMSE, 入选“最近4次同星期几均价”。2月起固定规则附件4为365×144个完整正电价, 范围 {data_summary["min"]:.4f}—{data_summary["max"]:.4f}元/kWh, 2—12月点预测RMSE为 {data_summary["report_rmse"]:.6f}元/kWh, MAE为 {data_summary["report_mae"]:.6f}元/kWh, 加性误差场景中的正数下限为0.001元/kWh, 共有 {clipped} 个场景价格被截断, 占 {price_count:.4f}%。原始实际价格没有截断。该下限是模型假设, 未据回测费用调参。',
'', '## 4. 指定日期结果','',
f'完整表1、表2、表3见 [指定日期结果.md](指定日期结果.md), 机器可读表格见 target_days/。',
'', '| 日期 | 计划购电量 (kWh) | 紧急购电量 (kWh) | 计划费 (元) | 紧急费 (元) | 总费 (元) |',
'|---|---:|---:|---:|---:|---:|']
for row in daily.loc[daily.date.isin(TARGET_DATES)].itertuples():
    lines.append(f'| {row.date} | {row.g_kwh:,.4f} | {row.emergency_kwh:,.4f} | {row.planned_cost_yuan:,.2f} | {row.emergency_cost_yuan:,.2f} | {row.total_cost_yuan:,.2f} |')
lines += ['','Excel“全天购电费”填写计划购电费, 与计划购电表一致; 紧急费及总费另在逐日CSV、汇总表中。',
'', '## 5. 已完成核验','',
f'- 四套方案各48,096个正式时段全部通过独立附件核验: 实际电价、净负载、能量守恒、0.9效率、功率因数、',
f'- 1月4,464个预热时段独立复算, 从6000 kWh连续到8550 kWh; 四套Excel均反读全部三个工作表及时间戳。',
f'- 人工篡改电价、储电量和紧急电量, 核验器均能拒绝。',
f'- 固定电价退化例与原Q2目标值之差为0; 两组价格—净负载配对比解析例费用18,720元、30,240元, 均与Q2一致。',
f'- 对2月1日、6月1日、11月30日同时改动当天及未来价格、负载、光伏, 历史预测主模型的当前预测、',
f'- 当日电价已知方案中, 改变次日及以后实际价格, 当前决策不变。源Q2代码、输出及原附件指纹未变。',
f'建模公式、信息口径、为什么仍可用LP以及分位数解释见 [建模说明.md](建模说明.md)。']
(HERE/'结果说明.md').write_text('\n'.join(lines),encoding='utf-8')
files = ('result4-2.xlsx','daily_summary.csv','schedule_detail.csv.gz')
publication = dict(primary_variant=primary, physical_verification_passed=verification['pass_check'],
information_verification_passed=information['pass_check'],
published_files_identical=all((HERE/n).read_bytes()==(folder/n).read_bytes() for n in files),
sha256={n:hashlib.sha256((HERE/n).read_bytes()).hexdigest() for n in files})
publication['pass_check'] = all(publication[k] for k in
('physical_verification_passed','information_verification_passed','published_files_identical'))
(HERE/'publication_verification.json').write_text(json.dumps(publication,ensure_ascii=False,indent=4))
if not publication['pass_check']:

```

```

        raise RuntimeError('Published files do not match the verified variant')
    print(json.dumps(summary,ensure_ascii=False,indent=2))

if __name__ == '__main__':
    main()

```

附录 E.5 问题四-3：波动电价下的多时点调整

q3_data.py

```

"""Read the original attachments; all power samples refer to interval ends."""
from pathlib import Path
import os
import hashlib
import numpy as np
import openpyxl

HERE = Path(__file__).resolve().parents[1] / "results"
T, DT = 144, 1 / 6
HOURS = (0, 6, 12, 18)

def attachment_dir():
    """直接定位 C 题附件，避免依赖 C_yang/raw 的项目结构。"""
    configured = os.environ.get("CUMCM_C_ATTACHMENT_DIR")
    candidates = []
    if configured:
        candidates.append(Path(configured))
    candidates.extend([
        HERE / "附件",
        HERE / "data" / "附件",
        HERE.parents[2] / "data" / "附件",
    ])
    for candidate in candidates:
        if all((candidate / f"附件{i}.xlsx").is_file() for i in (1, 2, 3, 4)):
            return candidate
    raise FileNotFoundError("找不到附件1.xlsx 至附件4.xlsx; 请设置 CUMCM_C_ATTACHMENT_DIR。")

ATTACHMENTS = attachment_dir()

def load_data():
    files = [ATTACHMENTS / f"附件{i}.xlsx" for i in (1, 2, 3, 4)]
    w = openpyxl.load_workbook(files[0], read_only=True, data_only=True)
    a = np.array([r[1:4] for r in w.active.iter_rows(min_row=2, values_only=True)], float)
    w.close()
    w = openpyxl.load_workbook(files[1], read_only=True, data_only=True)
    rows = [list(w[s].iter_rows(min_row=2, values_only=True))
             for s in ("小区负载", "光伏发电实际功率")]
    dates = [str(r[0])[:10] for r in rows[0]]
    assert [r[0] for r in rows[0]] == [r[0] for r in rows[1]]
    load, pv = (np.array([r[1:145] for r in rr], float) for rr in rows)

```

```

w.close()
w = openpyxl.load_workbook(files[2], read_only=True, data_only=True)
rr = list(w.active.iter_rows(min_row=2, values_only=True))
assert [str(r[1]) for r in rr[:4]] == ["0:00", "6:00", "12:00", "18:00"]
external = np.array([r[2:26] for r in rr], float).reshape(365, 4, 24)
w.close()
w = openpyxl.load_workbook(files[3], read_only=True, data_only=True)
price_rt = np.array([r[1:145] for r in w.active.iter_rows(min_row=2, values_only=True)], float)
w.close()
assert load.shape == pv.shape == (365, T)
assert price_rt.shape == (365, T)
assert all(np.isfinite(x).all() for x in (a, load, pv, external, price_rt))
assert min(load.min(), pv.min(), external.min(), price_rt.min()) >= 0
return dict(price=a[:, 0], price_rt=price_rt, load=load, pv=pv, external=external,
            dates=dates, source_sha256={p.name: hashlib.sha256(p.read_bytes()).hexdigest()
                                         for p in files})

def label(t):
    return "24:00" if t == T else f"{t // 6:02d}:{t % 6 * 10:02d}"

def intervals(values, tol=1e-7):
    start = None
    for t in range(len(values) + 1):
        active = t < len(values) and values[t] > tol
        if active and start is None:
            start = t
        elif not active and start is not None:
            yield label(start) + "-" + label(t), float(values[start:t].sum())
            start = None

q3_forecast.py
"""Scheme B kernels plus causal, online fusion with the released PV forecasts.

Forecast arrays may be precomputed, but decision code only reads today's current
issue and complete historical rows. Actual future values are used for settlement.
"""
import numpy as np
from q3_data import T, HOURS

WINDOW = 56
FIRST_RESIDUAL = 7

def kernel(x, target, known, same_week=False, window=7):
    if same_week:
        ix = [target - 7 * j for j in range(1, 5) if 0 <= target - 7 * j < known]
        return x[ix].mean(0) if ix else None
    hi = min(target, known)
    return x[max(0, hi - window):hi].mean(0) if hi else None

```

```

def channel(x, target, known, same_week, drift):
    raw = kernel(x, target, known, same_week)
    if raw is None:
        raw = x[:known].mean(0) if known else np.zeros(T)
    errors = []
    for d in range(max(0, known - drift), known):
        f = kernel(x, d, d, same_week)
        if f is not None:
            errors.append(x[d] - f)
    return raw + np.mean(errors, axis=0) if errors else raw

def history_day(data, d):
    l = np.concatenate([channel(data['load'], j, d, True, 5) for j in (d, d + 1)])
    v = np.concatenate([channel(data['pv'], j, d, False, 28) for j in (d, d + 1)])
    return np.maximum(l, 0), np.maximum(v, 0)

def raw_issue(data, d, stage, history_pv):
    """Linear interpolation of +1..+24 h forecasts. Anchor is observed issue-time PV."""
    start = stage * 36
    flat = data['pv'].ravel()
    anchor = float(flat[d * T + start - 1]) if d * T + start else 0.0
    result = history_pv.copy()
    ends = np.arange(start + 1, start + T + 1)
    result[start:start + T] = np.interp(
        ends, np.arange(25) * 6 + start,
        np.r_[anchor, data['external'][d, stage]])
    return result

class Forecaster:
    def __init__(self, data):
        self.data = data
        n = len(data['load'])
        self.load = np.zeros((n, 2 * T))
        self.hist = np.zeros_like(self.load)
        self.raw = np.zeros((n, 4, 2 * T))
        self.fused = np.zeros_like(self.raw)
        self.alpha = np.ones((n, 4, 4))
        self.bias = np.zeros((n, 4, T))
        flat = data['pv'].ravel()
        for d in range(n):
            self.load[d], self.hist[d] = history_day(data, d)
            for stage in range(4):
                start = stage * 36
                self.raw[d, stage] = raw_issue(data, d, stage, self.hist[d])
                out = self.raw[d, stage].copy()
                # d-1 rows may contain next-day outcomes: train only fully mature rows <= d-2.
                past = np.arange(max(FIRST_RESIDUAL, d - WINDOW), max(FIRST_RESIDUAL, d - 1))
                if len(past) >= 7:

```

```

        y = np.array([flat[r * T + start:r * T + start + T] for r in past])
        ext = self.raw[past, stage, start:start + T]
        hist = self.hist[past, start:start + T]
        for b in range(4):
            sl = slice(b * 36, (b + 1) * 36)
            delta = ext[:, sl] - hist[:, sl]
            den = np.sum(delta * delta)
            a = np.clip(np.sum(delta * (y[:, sl] - hist[:, sl])) / den, 0, 1) if den > 0 else 0
            self.alpha[d, stage, b] = a
            old = a * ext[:, sl] + (1 - a) * hist[:, sl]
            bias = (y[:, sl] - old).mean(0) * len(past) / (len(past) + 14)
            self.bias[d, stage, sl] = bias
            dst = slice(start + b * 36, start + (b + 1) * 36)
            out[dst] = a * self.raw[d, stage, dst] + (1 - a) * self.hist[d, dst] + bias
        self.fused[d, stage] = np.maximum(out, 0)

def point(self, d, stage, source='fusion'):
    if source == 'history':
        v = self.hist[d]
    elif source == 'external':
        v = self.raw[d, stage]
    else:
        v = self.fused[d, stage]
    return (self.load[d] - v) / 6

def scenarios(self, d, stage, source='fusion'):
    past = np.arange(max(FIRST_RESIDUAL, d - WINDOW), d)
    if not len(past):
        return self.point(d, stage, source)[None, :T], np.array([], dtype=int)
    residual = np.array([(self.data['load'][r] - self.data['pv'][r]) / 6
                        - self.point(r, stage, source)[:T] for r in past])
    return self.point(d, stage, source)[None, :T] + residual, past

def price_point(self, d, stage):
    """Causal real-time-price forecast; released prices up to the issue are known."""
    start = stage * 36
    past = self.data['price_rt'][max(0, d - 7):d]
    base = past.mean(0) if len(past) else np.full(T, self.data['price'].mean())
    result = base.copy()
    if start:
        result[:start] = self.data['price_rt'][d, :start]
    return result

def price_scenarios(self, d, stage):
    point = self.price_point(d, stage)
    past = np.arange(max(FIRST_RESIDUAL, d - WINDOW), d)
    if not len(past):
        return point[None], past
    residual = np.array([self.data['price_rt'][r] - self.price_point(r, stage) for r in past])
    values = np.maximum(point[None] + residual, 1e-5)
    values[:, :stage * 36] = point[:stage * 36]
    return values, past

```

```

def diagnostics(self, start=31, end=365):
    import pandas as pd
    rows = []
    for d in range(start, end):
        for s, hour in enumerate(HOURS):
            sl = slice(s * 36, (s + 1) * 36)
            actual = self.data['pv'][d, sl]
            rows.append(dict(date=self.data['dates'][d], issue_hour=hour,
                            alpha_next_6h=self.alpha[d, s, 0],
                            history_mae_kw=np.mean(abs(self.hist[d, sl] - actual)),
                            external_mae_kw=np.mean(abs(self.raw[d, s, sl] - actual)),
                            fusion_mae_kw=np.mean(abs(self.fused[d, s, sl] - actual))))
    return pd.DataFrame(rows)

```

q3_model.py

"""Finite, nonanticipative scenario-tree LP, solved again at each allowed issue.

Only the root block is executed. Future nodes group historical forecast revisions, never future realized loads. Stage controls are literally shared variables.

"""

```

from dataclasses import dataclass
import numpy as np
from scipy.optimize import linprog
from scipy.sparse import coo_matrix
from q3_data import T

```

```

ETA = 0.9
E_MIN, E_MAX, E_INIT = 1200., 10800., 6000.
CAP = 5000 / 6

```

@dataclass

```

class Node:
    stage: int
    stop: int
    members: np.ndarray
    parent: int | None
    q: np.ndarray | None = None
    c: np.ndarray | None = None
    d: np.ndarray | None = None
    e: np.ndarray | None = None

```

```

def split_observable(features, members):
    """Deterministic binary k-means on newly observable forecast revisions."""
    if len(members) < 6:
        return [members]
    x = features[members]
    j = int(np.argmax(x.var(0)))
    if x[:, j].std() < 1e-5:
        return [members]

```

```

centers = x[[np.argmin(x[:, j]), np.argmax(x[:, j])]].copy()
prev = None
for _ in range(12):
    group = ((x - centers[1]) ** 2).sum(1) < ((x - centers[0]) ** 2).sum(1)
    if min(group.sum(), (~group).sum()) < 3:
        order = np.argsort(x[:, j], kind='stable')
        return [members[order[:len(order)//2]], members[order[len(order)//2:]]]
    if prev is not None and np.array_equal(group, prev):
        break
    centers = np.array([x[~group].mean(0), x[group].mean(0)])
    prev = group
return [members[~group], members[group]]

def make_tree(forecaster, past, stage, stages, source):
    remaining = [s for s in stages if s >= stage]
    nodes = [Node(stage, (remaining[1] * 36 if len(remaining) > 1 else T),
                  np.arange(max(len(past), 1)), None)]
    parents = [0]
    for k, s in enumerate(remaining[1:], 1):
        stop = remaining[k + 1] * 36 if k + 1 < len(remaining) else T
        previous = remaining[k - 1]
        features = np.array([forecaster.point(r, s, source)[s * 36:]
                             - forecaster.point(r, previous, source)[s * 36:] for r in past])
        children = []
        for parent in parents:
            groups = split_observable(features, nodes[parent].members) if len(past) else [nodes[parent].members]
            for members in groups:
                children.append(len(nodes))
                nodes.append(Node(s, stop, members, parent))
        parents = children
    return nodes

class LP:
    def __init__(self):
        self.cost, self.bounds = [], []
        self.er, self.ec, self.ev, self.eb = [], [], [], []
        self.ur, self.uc, self.uv, self.ub = [], [], [], []

    def var(self, n, cost=0., lo=0., hi=None):
        ids = np.arange(len(self.cost), len(self.cost) + n)
        self.cost.extend(np.broadcast_to(cost, n).tolist())
        self.bounds.extend([(lo, hi)] * n)
        return ids

    def eq(self, terms, rhs=0.):
        row = len(self.eb)
        for col, val in terms:
            self.er.append(row); self.ec.append(int(col)); self.ev.append(val)
        self.eb.append(rhs)

```

```

def le(self, terms, rhs):
    row = len(self.ub)
    for col, val in terms:
        self.ur.append(row); self.uc.append(int(col)); self.uv.append(val)
    self.ub.append(rhs)

def solve(self):
    n = len(self.cost)
    eq = coo_matrix((self.ev, (self.er, self.ec)), shape=(len(self.eb), n)).tocsr()
    ub = coo_matrix((self.uv, (self.ur, self.uc)), shape=(len(self.ub), n)).tocsr()
    res = linprog(self.cost, A_ub=ub, b_ub=self.ub, A_eq=eq, b_eq=self.eb,
                  bounds=self.bounds, method='highs')
    if not res.success:
        raise RuntimeError(res.message)
    return res, eq, ub

def solve(forecaster, day, stage, stages, energy, day_start_energy,
         original=None, source='fusion', deterministic=False, branch=True):
    price = forecaster.price_point(day, stage) if hasattr(forecaster, 'price_point') else forecaster
    point = forecaster.point(day, stage, source)
    scenarios, past = forecaster.scenarios(day, stage, source)
    if deterministic:
        scenarios, past = point[None, :T], np.array([], dtype=int)
    nodes = make_tree(forecaster, past, stage, stages, source)
    if not branch:
        # Preserve update times but assume no future information gain.
        remaining = [s for s in stages if s >= stage]
        nodes = [Node(s, remaining[k + 1] * 36 if k + 1 < len(remaining) else T,
                     np.arange(len(scenarios)), k - 1 if k else None)
                 for k, s in enumerate(remaining)]
    S = len(scenarios)
    price_scenarios, price_past = (forecaster.price_scenarios(day, stage) if hasattr(forecaster, 'price_scenarios')
                                   else (np.broadcast_to(price, (S, T)), past))
    if len(price_past) != len(past):
        raise RuntimeError('Price and net-load scenario samples must use the same historical days.')
    lp = LP()
    base = lp.var(T, price) if original is None else None
    raw_overlap = 0
    for ni, node in enumerate(nodes):
        start, stop = node.stage * 36, node.stop
        n = stop - start
        prob = len(node.members) / S
        node_price = np.mean(price_scenarios[node.members, start:stop], axis=0)
        node.q = lp.var(n, prob * node_price)
        # Tiny throughput tie-breaker only; yuan scale is < 0.1 per year.
        node.c = lp.var(n, prob * 1e-9, hi=CAP)
        node.d = lp.var(n, prob * 1e-9, hi=CAP)
        node.e = lp.var(n, lo=E_MIN, hi=E_MAX)
        if node.stage > 0:
            up = lp.var(n, prob * 1.5 * node_price)
            down = lp.var(n, -prob * .5 * node_price)

```

```

old_change = np.zeros(2 * T)
if node.stage != stage and len(past):
    old_change = np.mean([forecaster.point(past[s], node.stage, source)
                          - forecaster.point(past[s], stage, source)
                          for s in node.members], axis=0)
node_point = point + old_change
for j, t in enumerate(range(start, stop)):
    terms = [(node.q[j], 1)]
    if node.stage > 0:
        terms += [(up[j], -1), (down[j], 1)]
    if base is not None:
        terms.append((base[t], -1))
    lp.eq(terms, 0 if base is not None else original[t])
    terms = [(node.e[j], 1), (node.c[j], -ETA), (node.d[j], 1 / ETA)]
    if j:
        terms.append((node.e[j - 1], -1))
    elif node.parent is not None:
        terms.append((nodes[node.parent].e[-1], -1))
    lp.eq(terms, energy if j == 0 and node.parent is None else 0)
    lp.le([(node.q[j], -1), (node.d[j], -1), (node.c[j], 1)], -node_point[t])
    emergency = lp.var(len(node.members) * n, (5 * price_scenarios[node.members, start:stop] /
    for k, s in enumerate(node.members):
        for j, t in enumerate(range(start, stop)):
            lp.le([(node.q[j], -1), (node.d[j], -1), (node.c[j], 1),
                    (emergency[k * n + j], -1)], -scenarios[s, t])
    if stop == T:
        if day == len(forecaster.data['load']) - 1:
            lp.eq([(node.e[-1], 1)], E_INIT)
        else:
            # Tomorrow is a deterministic value approximation, conditional on the node's forecast
            q = lp.var(T, prob * price)
            c = lp.var(T, prob * 1e-9, hi=CAP)
            d = lp.var(T, prob * 1e-9, hi=CAP)
            e = lp.var(T, lo=E_MIN, hi=E_MAX)
            for t in range(T):
                lp.eq([(e[t], 1), (c[t], -ETA), (d[t], 1 / ETA),
                        (e[t - 1] if t else node.e[-1], -1)])
                lp.le([(q[t], -1), (d[t], -1), (c[t], 1)], -node_point[T + t])
                lp.eq([(e[-1], 1)], day_start_energy)
    res, eq, ub = lp.solve()
    root = nodes[0]
    q, c, d, e = (res.x[ids].copy() for ids in (root.q, root.c, root.d, root.e))
    raw_overlap = int(np.sum((c > 1e-7) & (d > 1e-7)))
    # This preserves internal SOC and increases useful supply; no disposal upper bound.
    remove = np.minimum(c, d / ETA**2)
    c -= remove; d -= ETA**2 * remove
    return dict(original=res.x[base].copy() if base is not None else original.copy(),
                q=q, c=c, d=d, E=e, stop=root.stop,
                objective=float(res.fun), nodes=len(nodes), scenarios=S,
                raw_overlap=raw_overlap,
                lp_eq_max=float(np.max(np.abs(eq @ res.x - lp.ub))),
                lp_ineq_max=float(max(0., np.max(ub @ res.x - lp.ub))),

```

```

        tree=[dict(stage=n.stage, stop=n.stop, parent=n.parent,
                    historical_days=past[n.members].tolist() if len(past) else [],
                    probability=len(n.members) / S) for n in nodes])

run_q3.py
"""Run Q3 and comparable ablations. Outputs stay in this directory.

python run_q3.py                # full-year solution and all comparisons
python run_q3.py --mode all --days 4 --out smoke
"""

from pathlib import Path
from datetime import datetime, timedelta
import argparse
import json
import os
import platform
import time
import numpy as np
import pandas as pd
from q3_data import HERE, T, load_data, label, intervals
from q3_forecast import Forecaster
from q3_model import solve, E_INIT

# 附件5 result4-3.xlsx 模板（用于取原表头，保证导出与模板逐列一致）
_ETA_DIR = Path(__file__).resolve().parents[3] / "common" / "efficiency"
if str(_ETA_DIR) not in sys.path:
    sys.path.insert(0, str(_ETA_DIR))
from template_layout import TPL43 as TPL # noqa: E402

MODES = {
    'B_aligned': ('history', (0,)),
    'only_0': ('fusion', (0,)),
    'at_0_6': ('fusion', (0, 1)),
    'at_0_6_12': ('fusion', (0, 1, 2)),
    'all': ('fusion', (0, 1, 2, 3)),
}

def simulate(f, mode, end=365):
    source, stages = MODES[mode]
    energy = E_INIT
    records, daily, trees = [], [], {}
    started = time.time()
    for day in range(31, end):
        price = f.data['price_rt'][day]
        date = (datetime(2025, 1, 1) + timedelta(days=day)).strftime('%Y-%m-%d')
        day_start = energy
        original = None
        q = np.zeros(T); c = np.zeros(T); d = np.zeros(T); E = np.zeros(T)
        issues = np.zeros(T, dtype=int); forecast = np.zeros(T)
        worst_eq = worst_ineq = 0.
        max_nodes = 0

```

```

overlap = 0
for stage in stages:
    r = solve(f, day, stage, stages, energy, day_start, original, source)
    original = r['original']
    start, stop = stage * 36, r['stop']
    q[start:stop], c[start:stop], d[start:stop], E[start:stop] = r['q'], r['c'], r['d'], r['E']
    energy = E[stop - 1]
    issues[start:stop] = stage * 6
    forecast[start:stop] = f.point(day, stage, source)[start:stop]
    worst_eq = max(worst_eq, r['lp_eq_max'])
    worst_ineq = max(worst_ineq, r['lp_ineq_max'])
    max_nodes = max(max_nodes, r['nodes'])
    overlap += r['raw_overlap']
    if day in (31, 78, 171, 265, 354) and stage == 0:
        trees[date] = r['tree']
actual = (f.data['load'][day] - f.data['pv'][day]) / 6
z = q + d - c
emergency = np.maximum(actual - z, 0)
dumped = np.maximum(z - actual, 0)
up, down = np.maximum(q - original, 0), np.maximum(original - q, 0)
planned_cost = price * original
added_cost, cancellation_refund, breach_cost = 1.5 * price * up, price * down, .5 * price * up
adjustment_cost = added_cost - cancellation_refund + breach_cost
emergency_cost = 5 * price * emergency
total = planned_cost + adjustment_cost + emergency_cost
before = np.r_[day_start, E[:-1]]
for t in range(T):
    records.append(dict(date=date, slot=t, start=label(t), end=label(t + 1),
                        issue_hour=int(issues[t]), price_yuan_per_kwh=price[t],
                        original_kwh=original[t], adjusted_kwh=q[t], increase_kwh=up[t], decrease_kwh=down[t],
                        charge_kwh=c[t], discharge_kwh=d[t], energy_before_kwh=before[t], energy_kwh=E[t],
                        load_kwh=f.data['load'][day,t]/6, pv_kwh=f.data['pv'][day,t]/6,
                        forecast_net_kwh=forecast[t], net_supply_kwh=z[t], emergency_kwh=emergency[t],
                        planned_cost_yuan=planned_cost[t], increase_cost_yuan=added_cost[t],
                        cancellation_refund_yuan=cancellation_refund[t], breach_cost_yuan=breach_cost[t],
                        adjustment_net_yuan=adjustment_cost[t], emergency_cost_yuan=emergency_cost[t]))
daily.append(dict(date=date, planned_cost_yuan=planned_cost.sum(), increase_cost_yuan=added_cost.sum(),
                  cancellation_refund_yuan=cancellation_refund.sum(), breach_cost_yuan=breach_cost.sum(),
                  adjustment_net_yuan=adjustment_cost.sum(), emergency_cost_yuan=emergency_cost.sum(),
                  total_cost_yuan=total.sum(), original_kwh=original.sum(), adjusted_kwh=q.sum(),
                  increase_kwh=up.sum(), decrease_kwh=down.sum(), emergency_kwh=emergency.sum(),
                  dump_kwh=dumped.sum(), charge_kwh=c.sum(), discharge_kwh=d.sum(),
                  energy_start_kwh=day_start, energy_end_kwh=energy[stop],
                  lp_eq_max=worst_eq, lp_ineq_max=worst_ineq, max_tree_nodes=max_nodes,
                  raw_simultaneous_slots=overlap))
if (day - 31) % 30 == 0 or day == end - 1:
    print(f'{mode:12s} {date} cost={sum(x["total_cost_yuan"] for x in daily)/1e4:.2f} wan; ')
detail, days = pd.DataFrame(records), pd.DataFrame(daily)
sums = days.select_dtypes(include='number').sum().to_dict()
summary = {k:float(sums[k]) for k in ('planned_cost_yuan','increase_cost_yuan','cancellation_refund_yuan',
                                     'breach_cost_yuan','adjustment_net_yuan','emergency_cost_yuan','total_cost_yuan',
                                     'original_kwh','adjusted_kwh','increase_kwh','decrease_kwh','emergency_kwh','dump_kwh')}

```

```

summary.update(mode=mode, source=source, update_hours=[s*6 for s in stages], days=len(days),
               initial_energy_kwh=float(days.iloc[0].energy_start_kwh),
               final_energy_kwh=float(days.iloc[-1].energy_end_kwh), seconds=time.time()-starte
return detail, days, summary, trees

```

```

def export_excel(detail, days, path):
    import openpyxl
    from openpyxl.styles import Font, Alignment
    from openpyxl.comments import Comment
    import sys as _sys
    _sys.path.insert(0, str(Path(__file__).resolve().parents[3]
                             / "common" / "efficiency"))
    from template_layout import block_labels, circular, template_labels
    book = openpyxl.Workbook()
    book.remove(book.active)
    # 模板表头 (左端点, 首格 0:10-0:20) + 环形映射 out[i]=data[(i+1)%144]
    headers = template_labels(TPL, '计划购电量')
    blocks6 = block_labels(TPL, '充放电量')
    for name, col, cost in [('计划购电量', 'original_kwh', 'planned_cost_yuan'),
                           ('调整购电量', 'adjusted_kwh', 'adjustment_net_yuan')]:
        ws = book.create_sheet(name)
        ws.append(['日期\\时间']+headers+['全天购电量', '全天购电费'])
        if col == 'adjusted_kwh':
            ws.cell(1, 147).comment = Comment('此列为相对0点计划的净调整费用: 1.5p×增购-0.5p×减购; 表
        for date, group in detail.groupby('date', sort=False):
            row = days.loc[days.date == date].iloc[0]
            vals = circular(group[col].to_numpy(float))
            ws.append([datetime.fromisoformat(date)]+[round(v, 6) for v in vals]
                      +[group[col].sum(), float(row[cost])])
        ws.freeze_panes='B2'
        ws.column_dimensions['A'].width=13
    ws=book.create_sheet('充放电量')
    ws.append(['日期', '时间段', '充电量', '放电量', '时刻', '储电量'])
    for date, group in detail.groupby('date', sort=False):
        for b in range(6):
            g=group.iloc[b*24:(b+1)*24]
            ws.append([datetime.fromisoformat(date) if b==0 else None,
                      blocks6[b], g.charge_kwh.sum(), g.discharge_kwh.sum(),
                      '0:00' if b==0 else ('24:00' if b==1 else None),
                      group.iloc[0].energy_before_kwh if b==0 else (group.iloc[-1].energy_after_kwh
    ws=book.create_sheet('紧急购电量')
    ws.append(['日期', '购电时间段', '购电量'])
    for date, group in detail.groupby('date', sort=False):
        blocks=list(intervals(group.emergency_kwh.to_numpy())) or [('-', 0.0)]
        for i, (span, amount) in enumerate(blocks):
            ws.append([datetime.fromisoformat(date) if i==0 else None, span, amount])
    for ws in book:
        for cell in ws[1]:
            cell.font=Font(bold=True); cell.alignment=Alignment(horizontal='center')
        for row in ws.iter_rows(min_row=2):
            for cell in row:

```

```

        if isinstance(cell.value,datetime): cell.number_format='yyyy-mm-dd'
        elif isinstance(cell.value,(float,int)): cell.number_format='0.0000'
    if ws.max_column<10:
        for col in 'ABCDEF': ws.column_dimensions[col].width=18
book.save(path)

def target_tables(detail, days, out):
    targets=['2025-03-20','2025-06-21','2025-09-23','2025-12-21']
    rows1, rows2, rows3=[],[],[]
    for date in targets:
        g=detail[detail.date==date]
        if not len(g): continue
        for hour in (10,12,14,16,18,20):
            r=g.iloc[hour*6]
            rows1.append(dict(date=date,span=r['start']+'-'+r['end'],original_kwh=r.original_kwh,ad=
        for b in range(6):
            block=g.iloc[b*24:(b+1)*24]
            rows2.append(dict(date=date,span=label(b*24)+'-'+label((b+1)*24),charge_kwh=block.charg
        for span,amount in intervals(g.emergency_kwh.to_numpy()): rows3.append(dict(date=date,span=
pd.DataFrame(rows1).to_csv(out/'target_table1.csv',index=False,encoding='utf-8-sig')
pd.DataFrame(rows2).to_csv(out/'target_table2.csv',index=False,encoding='utf-8-sig')
pd.DataFrame(rows3).to_csv(out/'target_table3.csv',index=False,encoding='utf-8-sig')
days[days.date.isin(targets)].to_csv(out/'target_daily.csv',index=False,encoding='utf-8-sig')

def write_report(out):
    """Keep the readable result notes tied to the actual exported numbers."""
    comp=pd.read_csv(out/'comparison.csv')
    report_days=int(comp.iloc[0]['days'])
    last_date=(datetime(2025,2,1)+timedelta(days=report_days-1)).strftime('%Y年%m月%d日')
    model_doc=Path(os.path.relpath(HERE.parent/'建模说明.md',out)).as_posix()
    names={'B_aligned':'方案B框架同口径复算','only_0':'仅使用0点融合预报',
        'at_0_6':'0、6点','at_0_6_12':'0、6、12点','all':'0、6、12、18点（主方案）'}
    lines=['# 第三问结果说明','',
        f'结果期为2025年2月1日至{last_date}，共{report_days}天。采用在线预报融合、经验场景树LP和
        '', '**费用口径**：取消部分退原费、另付50%违约金；费用按供电时段电价计算。原计划费、增购
        '', '## 全年对照','',
        '| 方案 | 总费用（元） | 紧急购电量（kWh） |','|---|---:|---:|']
    for _,r in comp.iterrows():
        lines.append(f'| {names[r["mode"]]} | {r.total_cost_yuan:,.2f} | {r.emergency_kwh:,.2f} |')
    idx=comp.set_index('mode')
    if 'all' in idx.index:
        a=idx.loc['all']
        lines+=['','## 主方案费用分解','',
            '| 项目 | 金额（元） |','|---|---:|',
            f'| 原计划购电费 | {a.planned_cost_yuan:,.2f} |',
            f'| 增购费（1.5倍） | {a.increase_cost_yuan:,.2f} |',
            f'| 取消原费退款 | -{a.cancellation_refund_yuan:,.2f} |',
            f'| 取消违约金（50%） | {a.breach_cost_yuan:,.2f} |',
            f'| 净调整费 | {a.adjustment_net_yuan:,.2f} |',
            f'| 紧急购电费（5倍） | {a.emergency_cost_yuan:,.2f} |',

```

```

        f'| **总费用** | **{a.total_cost_yuan:,.2f}** |',' ',
        '净调整费已经包含增购费、退款和违约金，计算总额时不能重复叠加这三项。']
if 'B_aligned' in idx.index:
    b=idx.loc['B_aligned']
    lines += [' ',f'相对同口径方案B，主方案节省 **{b.total_cost_yuan-a.total_cost_yuan:,.2f}元**',
             ' ', '## 是否需要每次都调整',' ']
for old,new in [('B_aligned','only_0'),('only_0','at_0_6'),('at_0_6','at_0_6_12')]:
    if old in idx.index and new in idx.index:
        delta=idx.loc[old].total_cost_yuan-idx.loc[new].total_cost_yuan
        lines.append(f'- 从“{names[old]}”改为“{names[new]}”，全年减少费用 {delta:,.2f}元')
if 'at_0_6_12' in idx.index:
    delta=a.total_cost_yuan-idx.loc['at_0_6_12'].total_cost_yuan
    lines += [' ',f'加入18点更新后，全年费用变化为 **{delta:,.2f}元**。此差值很小，数据未显示',
             ' ', '该结论限于本数据和算法：18点仍可能通过次日预报影响跨日储能，不能断言其永远',
             ' ', '主结果 [result3.xlsx](result3.xlsx) 保留预设的四次更新方案；[0、6、12点结果]'],
if (out/'target_daily.csv').exists():
    target=pd.read_csv(out/'target_daily.csv')
    lines += [' ', '## 题目指定日期',' ',
             ' | 日期 | 原计划费（元） | 净调整费（元） | 紧急电量（kWh） | 紧急费（元） | 总费用（元） |',
             ' |---|---:|---:|---:|---:|---:|---:|'],
    for _,r in target.iterrows():
        lines.append(f'| {r.date} | {r.planned_cost_yuan:,.2f} | {r.adjustment_net_yuan:,.2f} |')
    lines += [' ', '表1的指定十分钟购电量、表2的六个四小时充放电量、表3的全部紧急时间区间，分别参见',
             ' [result1.xlsx](result1.xlsx)、[result2.xlsx](result2.xlsx)和[result3.xlsx](result3.xlsx)'],
lines += [' ', '## 核验与解释边界',' ',
         '- 主方案及对照的初末SOC统一为6000 kWh，能量递推、上下限、功率、充放电互斥、跨日连续和',
         '- `verify_q3.py` 另对三天四个发布时刻做未来信息扰动，反读Excel并独立核算费用；应以该',
         '- 方案B的历史内核参数由既有方案继承，不能据本次因果执行检查声称原参数选择过程一定没有',
         '- 取消量若不退还原费，同一主调度的费用会增加退款总额；该敏感性写入核验JSON。若更换结果',
         '- 主方案为有限经验场景上的LP加滚动执行，不宣称全年真实随机问题全局最优。',
         ' ', '## 图表',' ',
         ' ![费用与紧急电量对照](figures/cost_comparison.png)',' ',
         ' ![四个指定日购电与储能](figures/target_dispatch.png)',' ',
         ' ![发布预报与历史信息融合](figures/forecast_fusion.png)',' ']
(out/'结果说明.md').write_text('\n'.join(lines),encoding='utf-8')

def main():
    ap=argparse.ArgumentParser()
    ap.add_argument('--mode',choices=list(MODES)+['suite'],default='suite')
    ap.add_argument('--days',type=int,default=334)
    ap.add_argument('--out',type=Path,default=HERE)
    args=ap.parse_args()
    out=args.out if args.out.is_absolute() else HERE/args.out
    out.mkdir(parents=True,exist_ok=True)
    data=load_data(); f=Forecaster(data)
    end=min(365,31+args.days)
    summaries=[]
    modes=MODES if args.mode=='suite' else [args.mode]
    for mode in modes:
        detail,days,summary,trees=simulate(f,mode,end)
        dest=out if mode=='all' else out/'comparisons'/mode
        dest.mkdir(parents=True,exist_ok=True)

```

```

detail.to_csv(dest/'schedule_detail.csv',index=False,encoding='utf-8-sig')
days.to_csv(dest/'daily_summary.csv',index=False,encoding='utf-8-sig')
(dest/'summary.json').write_text(json.dumps(summary,ensure_ascii=False,indent=2),encoding='utf-8-sig')
summaries.append(summary)
from verify_q3 import check_schedule
checks=check_schedule(detail,days,data,require_year_end=end==365)
(dest/'verification.json').write_text(json.dumps(checks,ensure_ascii=False,indent=2),encoding='utf-8-sig')
if not checks['pass']: raise RuntimeError(f'Independent verification failed: {mode}')
if mode=='at_0_6_12':
    # A ready-to-review lower-frequency alternative, kept separate from the preset main policy
    export_excel(detail,days,dest/'result4-3.xlsx')
if mode=='all':
    export_excel(detail,days,out/'result4-3.xlsx')
    target_tables(detail,days,out)
    (out/'scenario_trees.json').write_text(json.dumps(trees,ensure_ascii=False,indent=2),encoding='utf-8-sig')
pd.DataFrame(summaries).to_csv(out/'comparison.csv',index=False,encoding='utf-8-sig')
f.diagnostics(end=end).to_csv(out/'forecast_diagnostics.csv',index=False,encoding='utf-8-sig')
import scipy
metadata=dict(source_sha256=data['source_sha256'],python=platform.python_version(),numpy=np.__version__,
              residual_window_days=56,first_residual_day='2025-01-08',history_load='sw4/drift5',
              initialization='January battery idle at 6000 kWh; all reports start February 1 at 6000 kWh',
              end_condition='48h end equals current day start; Dec 31 ends at 6000',
              settlement='attachment-4 actual real-time price p: p*g0 + 1.5*p*max(q-g0,0) - 0.5*p*max(g-q,0)',
              execution='Only next issue block is committed; each delivery slot adjusted at most 100 kWh',
              method='Joint historical net-load and real-time-price scenario LP; causal seven-day forecast')
(out/'run_metadata.json').write_text(json.dumps(metadata,ensure_ascii=False,indent=2),encoding='utf-8-sig')
write_report(out)
print(pd.DataFrame(summaries)[['mode','total_cost_yuan','emergency_kwh']].to_string(index=False))

```

```

if __name__=='__main__': main()

```

verify_q3.py

```

"""Independent physical, accounting, workbook and information-boundary checks."""

```

```

from pathlib import Path

```

```

import json

```

```

import numpy as np

```

```

import pandas as pd

```

```

from q3_data import HERE, T, load_data

```

```

def check_schedule(detail, daily, data, require_year_end=True):

```

```

    x=detail

```

```

    p=data['price_rt'][31:31+len(daily)].reshape(-1)

```

```

    g,q,c,d=(x[k].to_numpy() for k in ('original_kwh','adjusted_kwh','charge_kwh','discharge_kwh'))

```

```

    day_indices=(pd.to_datetime(x.date)-pd.Timestamp('2025-01-01')).dt.days.to_numpy()

```

```

    slots=x.slot.to_numpy(int)

```

```

    net=(data['load'][day_indices,slots]-data['pv'][day_indices,slots])/6

```

```

    emergency=np.maximum(net+c-d-q,0)

```

```

    down=np.maximum(g-q,0); up=np.maximum(q-g,0)

```

```

    # Independently use paid delivered base + breach + new purchases.

```

```

    cost=p*np.minimum(g,q)+.5*p*down+1.5*p*up+5*p*emergency

```

```

before=x.energy_before_kwh.to_numpy(); after=x.energy_after_kwh.to_numpy()
balance=q+d+x.emergency_kwh.to_numpy()-net-c-x.dump_kwh.to_numpy()
sumcols=['planned_cost_yuan','increase_cost_yuan','cancellation_refund_yuan','breach_cost_yuan',
         'adjustment_net_yuan','emergency_cost_yuan','total_cost_yuan','original_kwh','adjusted',
         'increase_kwh','decrease_kwh','emergency_kwh','dump_kwh','charge_kwh','discharge_kwh']
summed=x.groupby('date',sort=False)[sumcols].sum().to_numpy()
errs=dict(
    periods=int(len(x)), duplicate_date_slots=int(x.duplicated(['date','slot']).sum()),
    energy_min_kwh=float(min(after.min(),before.min())), energy_max_kwh=float(max(after.max(),before.max())),
    energy_recursion_max_error=float(np.max(abs(after-before-.9*c+d/.9))),
    chronological_continuity_max_error=float(np.max(abs(after[:-1]-before[1:]))) ,
    initial_energy_kwh=float(before[0]), final_energy_kwh=float(after[-1]),
    max_charge_kwh=float(c.max()), max_discharge_kwh=float(d.max()),
    min_flow_kwh=float(min(c.min(),d.min(),q.min(),g.min())),
    simultaneous_periods=int(((c>1e-7)&(d>1e-7)).sum()),
    energy_balance_max_error=float(abs(balance).max()),
    emergency_max_error=float(np.max(abs(emergency-x.emergency_kwh.to_numpy()))),
    settlement_per_slot_max_error=float(np.max(abs(cost-x.total_cost_yuan.to_numpy()))),
    daily_sum_max_error=float(np.max(abs(summed-daily[sumcols].to_numpy()))),
    cost_recomputed_yuan=float(cost.sum()),
    altered_past_periods=int(np.sum(x.issue_hour.to_numpy()>slots/6)),
    midnight_block_plan_max_error=float(np.max(abs((q-g)[x.issue_hour.to_numpy()==0]))),
    lp_eq_max=float(daily.lp_eq_max.max()),lp_ineq_max=float(daily.lp_ineq_max.max()),
    alternative_no_refund_cost_same_schedule_yuan=float((cost+p*down).sum()))
errs['pass']=bool(
    len(x)==len(daily)*T and errs['duplicate_date_slots']==0 and
    errs['energy_min_kwh']>=1200-1e-6 and errs['energy_max_kwh']<=10800+1e-6 and
    errs['energy_recursion_max_error']<1e-6 and errs['chronological_continuity_max_error']<1e-6 and
    abs(before[0]-6000)<1e-6 and (not require_year_end or abs(after[-1]-6000)<1e-6) and
    max(c.max(),d.max())<=5000/6+1e-6 and errs['min_flow_kwh']>=-1e-6 and
    errs['simultaneous_periods']==0 and errs['energy_balance_max_error']<1e-6 and
    errs['emergency_max_error']<1e-6 and errs['settlement_per_slot_max_error']<1e-6 and
    errs['daily_sum_max_error']<1e-5 and errs['altered_past_periods']==0 and
    errs['midnight_block_plan_max_error']<1e-6 and errs['lp_eq_max']<1e-6 and errs['lp_ineq_max']<1e-6
)
return errs

def information_checks(data):
    from q3_forecast import Forecaster
    from q3_model import solve, make_tree
    base=Forecaster(data)
    tests=[]
    for day in (31,171,354):
        original=None; energy=6000.
        for stage in range(4):
            a=solve(base,day,stage,(0,1,2,3),energy,6000.,original)
            changed={k:(v.copy() if isinstance(v,np.ndarray) else v) for k,v in data.items()}
            # Includes all still-unrealized current-day values, all later days,
            # and all forecasts released strictly after the current issue.
            cut=day*T+stage*36
            changed['load'].ravel()[cut:]*=1.7
            changed['pv'].ravel()[cut:]+=321.

```

```

        changed['external'][day,stage+1:] += 777.
        changed['external'][day+1:] += 555.
        other=Forecaster(changed)
        b=solve(other,day,stage,(0,1,2,3),energy,6000.,original)
        prediction_error=float(np.max(abs(base.point(day,stage)[stage*36:]-other.point(day,stage*36:))))
        control_error=max(float(np.max(abs(a[k]-b[k]))) for k in ('original','q','c','d','E'))
        tests.append(dict(day=day,issue_hour=stage*6,prediction_max_change=prediction_error,
                           decision_max_change=control_error,pass_check=control_error<1e-7 and prediction_max_change<1e-7))
        original=a['original']; energy=a['E'][-1]
    # Test zero-update identity: same optimization returns no adjustment.
    r=solve(base,171,0,(0,),6000.,6000.)
    identity=float(np.max(abs(r['q']-r['original'])))
    result=dict(future_perturbation_tests=tests, no_update_identity_max_error=identity,
                pass_check=all(t['pass_check'] for t in tests) and identity<1e-7)
    return result

def _workbook_slot_order(headers):
    """Map result-table headers to internal slots, including template wraparound."""
    slots=[]
    for label in headers:
        try:
            start=str(label).split('-',1)[0].replace('+1','')
            hour,minute=map(int,start.split(':'))
            slots.append((hour*6+minute//10)%T)
        except (AttributeError,TypeError,ValueError):
            return []
    return slots if len(slots)==T and sorted(slots)==list(range(T)) else []

def workbook_checks(path,detail):
    import openpyxl
    w=openpyxl.load_workbook(path,read_only=True,data_only=True)
    errors=[]
    for name,col in [('计划购电量','original_kwh'),('调整购电量','adjusted_kwh')]:
        s=w[name]
        slot_order=_workbook_slot_order([s.cell(1,col).value for col in range(2,146)])
        if not slot_order:
            w.close()
            return dict(max_roundtrip_error=float('inf'),pass_check=False,
                        errors=[f'{name}: invalid interval headers'])
        actual=np.array([r[1:145] for r in s.iter_rows(min_row=2,values_only=True)],float)
        expected=detail[col].to_numpy().reshape(-1,T)[: ,slot_order]
        errors.append(float(np.max(abs(actual-expected))))
    rows=list(w['充放电量'].iter_rows(min_row=2,values_only=True))
    charge=np.array([r[2] for r in rows]); discharge=np.array([r[3] for r in rows])
    errors.extend([float(abs(charge-detail.charge_kwh.to_numpy().reshape(-1,24).sum(1)).max()),
                  float(abs(discharge-detail.discharge_kwh.to_numpy().reshape(-1,24).sum(1)).max())])
    total_emergency=sum(float(r[2] or 0) for r in w['紧急购电量'].iter_rows(min_row=2,values_only=True))
    errors.append(abs(total_emergency-detail.emergency_kwh.sum()))
    w.close()
    # Workbook values are exported to six decimals; summing rounded blocks can

```

```

# accumulate a few 1e-6 units, so use a rounding-aware tolerance.
return dict(max_roundtrip_error=float(max(errors)),pass_check=bool(max(errors)<5.1e-5))

def main():
    data=load_data()
    d=pd.read_csv(HERE/'schedule_detail.csv'); days=pd.read_csv(HERE/'daily_summary.csv')
    result=check_schedule(d,days,data)
    result['information_boundary']=information_checks(data)
    result['workbook']=workbook_checks(HERE/'result4-3.xlsx',d)
    result['pass']=bool(result['pass'] and result['information_boundary']['pass_check'] and result[
    alternative=HERE/'comparisons'/'at_0_6_12'
    if (alternative/'result4-3.xlsx').exists():
        result['alternative_workbook']=workbook_checks(alternative/'result4-3.xlsx',pd.read_csv(alt
        result['pass']=bool(result['pass'] and result['alternative_workbook']['pass_check'])
    (HERE/'verification.json').write_text(json.dumps(result,ensure_ascii=False,indent=2),encoding='
    print(json.dumps(result,ensure_ascii=False,indent=2))
    if not result['pass']: raise SystemExit(1)

if __name__=='__main__': main()

```

附录 E.6 公共模块：第二问基础数据与 LP (q2_base)

q2_data.py

"""问题2 数据层：读取附件1（电价/典型日）与附件2（全年实际负载与光伏）。

单位口径

- 附件1/附件2 的负载与光伏均为功率 kW，时间分辨率 10 分钟
- 每段电量(kWh) = 功率(kW) × Δt, Δt = 1/6 h
- 附件1 的行标签是"区间末" (0:10 表示 00:00-00:10)，共 144 段

问题2 的数据口径（依据题面）

- 电价：只用附件1 的电价列，且"每天的电价相同" -> 全年复用同一条 144 段电价曲线
- 负载/光伏：用附件2 的实际值（2025.1.1-12.31，365 天）
- 净负载 N = 负载 - 光伏 (kW 或 kWh，同比例)

附件5/result2.xlsx 的表头时间标签与附件1 的行标签存在一格错位

（模板首列为 '0:10-0:20'，末列为 '0:00+1-0:10+1'），二者是一一对应的，本模块直接从模板读取标签，保证导出文件与模板逐列对齐。

"""

```
from __future__ import annotations
```

```
from pathlib import Path
```

```
import numpy as np
```

```
import openpyxl
```

```

DT = 1.0 / 6.0          # 每段小时数
T = 144                 # 每天时段数
TAU = 10                # 每段分钟数

```

```

ROOT = Path(__file__).resolve().parents[2]
ATT1 = ROOT / "data" / "附件" / "附件1.xlsx"
ATT2 = ROOT / "data" / "附件" / "附件2.xlsx"
TPL2 = ROOT / "data" / "附件" / "附件5" / "result2.xlsx"

def _series_from_row(row):
    """把附件2 的一行（日期 + 144 个数值）转成 float 数组。"""
    vals = np.array([float(v) for v in row[1:1 + T]], dtype=float)
    assert vals.size == T, f"期望 {T} 个时段，实际 {vals.size}"
    return vals

def load_attachment1():
    """附件1: 返回 (labels, price, load_kW, pv_kW)。labels 为区间末标签。"""
    ws = openpyxl.load_workbook(ATT1, data_only=True).worksheets[0]
    labels, price, load, pv = [], [], [], []
    for t, row in enumerate(ws.iter_rows(min_row=2, values_only=True), start=1):
        labels.append(_label(t))
        price.append(float(row[1]))
        load.append(float(row[2]))
        pv.append(float(row[3]))
    assert len(price) == T, f"附件1 期望 {T} 段，实际 {len(price)} 段"
    return labels, np.array(price), np.array(load), np.array(pv)

def _label(t):
    """第 t 段（1-based）的区间末标签，t=144 -> '24:00'。"""
    m = t * TAU
    return f"{m // 60:02d}:{m % 60:02d}"

def load_attachment2():
    """附件2: 返回 (dates, load_kW, pv_kW)，形状均为 (365, 144)。

    日期取自"小区负载"工作表第一列，已核对与"光伏发电实际功率"工作表一致。
    """
    wb = openpyxl.load_workbook(ATT2, data_only=True)
    ws_l = wb["小区负载"]
    ws_v = wb["光伏发电实际功率"]

    dates, load, pv = [], [], []
    for rl, rv in zip(ws_l.iter_rows(min_row=2, values_only=True),
                      ws_v.iter_rows(min_row=2, values_only=True)):
        if rl[0] is None:
            continue
        dates.append(rl[0])
        load.append(_series_from_row(rl))
        pv.append(_series_from_row(rv))
        assert rl[0] == rv[0], "两个工作表的日期不一致"

    load = np.vstack(load)

```

```

pv = np.vstack(pv)
assert load.shape == (365, T), f"附件2 形状异常: {load.shape}"
return dates, load, pv

def template_labels():
    """从附件5/result2.xlsx 读取 144 个时间标签, 保证导出与模板对齐。"""
    ws = openpyxl.load_workbook(TPL2, data_only=True)["计划购电量"]
    head = next(ws.iter_rows(min_row=1, max_row=1, values_only=True))
    labs = [str(x) for x in head[1:1 + T]]
    assert len(labs) == T, f"模板标签数 {len(labs)}"
    return labs

def net_load(load_kW, pv_kW):
    """净负载 (kW), 等于负载减光伏。"""
    return load_kW - pv_kW

def to_energy(power_kW):
    """功率 -> 每段电量(kWh)。"""
    return np.asarray(power_kW, dtype=float) * DT

def weekday_index(dates):
    """返回每天的星期序号 (周一=0)。"""
    return np.array([d.weekday() for d in dates], dtype=int)

if __name__ == "__main__":
    labs1, price, l1, v1 = load_attachment1()
    dates, L, V = load_attachment2()
    t1 = template_labels()

    print("=" * 66)
    print("附件1 (典型日) ")
    print("=" * 66)
    print(f" 时段数      : {len(price)}")
    print(f" 电价        : {price.min():.4f} ~ {price.max():.4f} 元/kWh 均值 {price.mean():.4f}")
    print(f" 负载        : 全天 {l1.sum() * DT:,.2f} kWh 峰值 {l1.max():,.2f} kW")
    print(f" 光伏        : 全天 {v1.sum() * DT:,.2f} kWh 峰值 {v1.max():,.2f} kW")

    print("\n" + "=" * 66)
    print("附件2 (2025 全年实际) ")
    print("=" * 66)
    print(f" 天数        : {L.shape[0]} ({dates[0].date()} ~ {dates[-1].date()})")
    print(f" 负载        : 全年 {L.sum() * DT / 1000:,.1f} MWh 日均 {L.sum() * DT / 365:,.1f} kWh")
    print(f" 光伏        : 全年 {V.sum() * DT / 1000:,.1f} MWh 日均 {V.sum() * DT / 365:,.1f} kWh")
    N = net_load(L, V)
    print(f" 净负载      : 日均 {N.sum() * DT / 365:,.1f} kWh "
          f"最小 {N.min() * DT:.1f} kWh/段 最大 {N.max() * DT:.1f} kWh/段")

```

```

print("\n附件1 是否为'典型日'（与附件2 日均比对）")
print(f"  全天负载  附件1 {l1.sum() * DT:,.2f}  vs  附件2 日均 {L.sum() * DT / 365:,.2f} kWh"
      f"  偏差 {(l1.sum() - L.sum() / 365) / (L.sum() / 365) * 100:+.3f}%")
print(f"  全天光伏  附件1 {v1.sum() * DT:,.2f}  vs  附件2 日均 {V.sum() * DT / 365:,.2f} kWh"
      f"  偏差 {(v1.sum() - V.sum() / 365) / (V.sum() / 365) * 100:+.3f}%")

print("\n模板时间标签（前3/后3）:", t1[:3], "...", t1[-3:])
print("附件1 行标签（前3/后3）:", labs1[:3], "...", labs1[-3:])

run_1439.py
"""问题2 最优方案（1438.9 万）独立复现脚本。

方案构成（五件套，缺一不可）
    预测器      : L[sw4/d5] + V[t7/d28]  —— 同星期几均值 + 漂移校正
    计划平衡    : 不等式  $g + d - c \geq N$ （保留报童自由空间）
    时域终端    : 48h 滚动前瞻 + 周期终端（末段电量 = 起点电量）
    场景        : 最近 56 天整日联合残差，等权不削减
    结构        : 两阶段随机规划（计划非前瞻，紧急购电场景补救）

理论支撑：报童临界分位  $F^* = 1 - 1/5 = 0.80$ 

依赖
- 同目录 q2_data.py（数据层）
- 本文件内置 forecast_net（预测器，无外部目录依赖）

输出（本脚本所在目录）
    result2.xlsx          三个工作表：计划购电量 / 充放电量 / 紧急购电量
    daily_summary.csv     逐日结算
    schedule_detail.csv   逐时段明细
    summary.json          汇总指标
    verification.json     可行性校验
    figures/fig1..fig5    论文图表
"""

from __future__ import annotations

import json
import sys
from datetime import datetime, time
from pathlib import Path

import numpy as np

HERE = Path(__file__).resolve().parent
# --- 依赖路径（相对本仓库；若目录移动请修改） ---
REPO = HERE.parents[1] # mathematical-modeling/
CY = HERE
sys.path.insert(0, str(CY))

from q2_data import DT, T, load_attachment1, load_attachment2, net_load, template_labels # noqa: E

# ===== 配置 =====
HORIZON = 2 # 前瞻天数

```

```

WINDOW = 56                # 场景窗口 (天)
REPORT_FROM = 31           # 2025-02-01 (0-based)
E_INIT, E_MIN, E_MAX = 6000.0, 1200.0, 10800.0
ETA_C = ETA_D = 0.90
P_MAX = 5000.0
FLOW_CAP = P_MAX * DT
EMERGENCY_MULT = 5.0
YEAR_END_E = E_INIT
# 预测器: 负载用最近 4 次同星期几均值 + 近 5 天误差校正; 光伏用近 7 天均值 + 近 28 天误差校正
LOAD_K, LOAD_DRIFT = 4, 5
PV_M, PV_DRIFT = 7, 28

# ===== 预测器 =====
def _sw(X, d, k, upto=None):
    """最近 k 次同星期几的均值。"""
    idx = [d - 7 * j for j in range(1, k + 1)
            if d - 7 * j >= 0 and (upto is None or d - 7 * j < upto)]
    return X[idx].mean(axis=0) if idx else None

def _tr(X, d, m, upto=None):
    """最近 m 天均值。"""
    hi = d if upto is None else min(d, upto)
    return X[max(0, hi - m):hi].mean(axis=0) if hi > 0 else None

def _channel(X, d, kernel, drift, upto=None):
    """带漂移校正的预测通道: raw + 近 drift 天平均误差。"""
    raw = kernel(X, d, upto)
    if raw is None:
        return None
    if drift == 0:
        return raw
    hi = d if upto is None else min(d, upto)
    errs = []
    for dd in range(max(0, hi - drift), hi):
        r = kernel(X, dd, upto)
        if r is not None:
            errs.append(X[dd] - r)
    return raw if not errs else raw + np.mean(errs, axis=0)

def forecast_net(L, V, d, upto=None):
    """净负载点预测 (kW): 负载通道 - 光伏通道。"""
    fl = _channel(L, d, lambda X, dd, up: _sw(X, dd, LOAD_K, up), LOAD_DRIFT, upto)
    fv = _channel(V, d, lambda X, dd, up: _tr(X, dd, PV_M, up), PV_DRIFT, upto)
    _, _, l1, v1 = load_attachment1()
    fl = l1 if fl is None else fl
    fv = v1 if fv is None else fv
    return fl - fv

```

```

def residual_matrix(L, V):
    """历史整日残差 (kW)，预热期为 NaN:  $R[d] = N[d] * DT - \text{预测}(d)$ 。"""
    N = net_load(L, V)
    R = np.full((L.shape[0], T), np.nan)
    for d in range(L.shape[0]):
        pred = forecast_net(L, V, d)
        R[d] = N[d] - pred
    return R

def point_matrix(L, V):
    """逐日点预测矩阵 (kWh)。"""
    N = net_load(L, V)
    FC = np.zeros((L.shape[0], T))
    for d in range(L.shape[0]):
        FC[d] = forecast_net(L, V, d) * DT
    return FC, N

# ===== 单时段模型 =====
def solve_horizon(N_plan, N_scen, weights, price, E_init, E_term):
    """滚动前瞻两阶段随机规划 (稀疏 LP)。”

    N_plan (K,) 计划用净负载点预测 kWh;  $K = H * T$ 
    N_scen (S,T) 决策日场景净负载 kWh
    price (K,) 电价
    E_term 终端电量 (周期终端传 E_init; 年末传 6000)
    """

    from scipy.optimize import linprog
    from scipy.sparse import coo_matrix

    K = N_plan.shape[0]
    S, TD = N_scen.shape
    n = 4 * K + S * TD
    G, C, D, E = slice(0, K), slice(K, 2 * K), slice(2 * K, 3 * K), slice(3 * K, 4 * K)
    EE = slice(4 * K, 4 * K + S * TD)

    # 目标
    c_obj = np.zeros(n)
    c_obj[G] = price
    px = price[:T]
    for s in range(S):
        c_obj[EE.start + s * T: EE.start + (s + 1) * T] = EMERGENCY_MULT * weights[s] * px

    # 等式: 电量递推 + 终端
    rows, cols, vals, b_eq = [], [], [], []
    n_eq = K + 1
    b_eq = np.zeros(n_eq)
    for k in range(K):
        pairs = [(E.start + k, 1.0), (C.start + k, -ETA_C), (D.start + k, 1.0 / ETA_D)]
        if k > 0:

```

```

        pairs.append((E.start + k - 1, -1.0))
    else:
        b_eq[k] = E_init
    for j, v in pairs:
        rows.append(k); cols.append(j); vals.append(v)
rows.append(K); cols.append(E.start + K - 1); vals.append(1.0); b_eq[K] = E_term
A_eq = coo_matrix((vals, (rows, cols)), shape=(n_eq, n)).tocsr()

# 不等式：计划下界（不等式）+ 场景平衡
r2, c2, v2, b_list = [], [], [], []
nrow = 0
for k in range(K):
    r2 += [nrow] * 3; c2 += [G.start + k, D.start + k, C.start + k]
    v2 += [-1.0, -1.0, 1.0]; b_list.append(-N_plan[k]); nrow += 1
for s in range(S):
    base = EE.start + s * T
    for t in range(T):
        r2 += [nrow] * 4; c2 += [base + t, G.start + t, D.start + t, C.start + t]
        v2 += [-1.0, -1.0, -1.0, 1.0]; b_list.append(-N_scen[s, t]); nrow += 1
A_ub = coo_matrix((v2, (r2, c2)), shape=(nrow, n)).tocsr()
b_ub = np.array(b_list)

lb = np.zeros(n); ub = np.full(n, np.inf)
lb[E], ub[E] = E_MIN, E_MAX
ub[C] = FLOW_CAP; ub[D] = FLOW_CAP

res = linprog(c_obj, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq,
              bounds=list(zip(lb, ub)), method="highs")
if res.status != 0:
    raise RuntimeError(f"LP 失败: {res.message}")
x = res.x
g, c, d = x[G], x[C], x[D]
# 精确消除同时充放电
eff = ETA_C * ETA_D
eps = np.minimum(c, d / eff)
c = c - eps; d = d - eff * eps
c[np.abs(c) < 1e-9] = 0.0; d[np.abs(d) < 1e-9] = 0.0
return dict(g=g, c=c, d=d, z=g + d - c, E=x[E])

def settle_day(g, c, d, N_actual_kwh, price):
    """按题面结算：计划购电按 p 计费，缺口按 5p 紧急购电。"""
    z = g + d - c
    e = np.maximum(N_actual_kwh - z, 0.0)
    return dict(e=e,
                plan_cost=float(price @ g),
                emergency_cost=float(EMERGENCY_MULT * (price @ e)))

# ===== 主流程 =====
def main():
    _, L, V = load_attachment2()

```

```

_, price, _, _ = load_attachment1()
N = net_load(L, V)
FC, _ = point_matrix(L, V)
R = residual_matrix(L, V)

ND = L.shape[0]
g_arr = np.zeros((ND, T)); c_arr = np.zeros((ND, T)); d_arr = np.zeros((ND, T))
z_arr = np.zeros((ND, T)); e_arr = np.zeros((ND, T)); Etraj = np.zeros((ND, T))
E_start = np.zeros(ND); E_end = np.zeros(ND)
plan_cost = np.zeros(ND); emg_cost = np.zeros(ND)

E_now = E_INIT
for d in range(ND):
    H = min(HORIZON, ND - d)
    # 前瞻点预测: 只用第 0..d-1 天信息 (upto=d)
    N_plan = np.concatenate([forecast_net(L, V, j, upto=d) * DT for j in range(d, d + H)])
    # 决策日场景: 最近 WINDOW 天的有效残差
    lo = max(0, d - WINDOW)
    Rw = R[lo:d]
    ok = ~np.isnan(Rw).any(axis=1)
    Rw = Rw[ok]
    if Rw.shape[0] == 0:
        Rw = np.zeros((1, T))
    N_scen = FC[d][None, :] + Rw * DT
    w = np.full(Rw.shape[0], 1.0 / Rw.shape[0])
    # 终端: 年末回 6000, 其余周期终端
    E_term = YEAR_END_E if d == ND - 1 else E_now
    plan = solve_horizon(N_plan, N_scen, w, np.tile(price, H), E_now, E_term)

    g0, c0, d0 = plan["g"][:T], plan["c"][:T], plan["d"][:T]
    g_arr[d], c_arr[d], d_arr[d], z_arr[d] = g0, c0, d0, g0 + d0 - c0
    E_start[d] = E_now
    Etraj[d] = plan["E"][:T]
    E_now = float(plan["E"][T - 1])
    E_end[d] = E_now

    st = settle_day(g0, c0, d0, N[d] * DT, price)
    e_arr[d] = st["e"]
    plan_cost[d] = st["plan_cost"]
    emg_cost[d] = st["emergency_cost"]
    if d % 30 == 0 or d == ND - 1:
        print(f" 第 {d+1:3d}/{ND} 天 E={E_now:8.2f} 累计费用="
              f"{(plan_cost[REPORT_FROM:d+1].sum()+emg_cost[REPORT_FROM:d+1].sum())/1e4:8.2f} 万"
              flush=True)

i0 = REPORT_FROM
s1 = slice(i0, ND)
tot_plan = float(plan_cost[s1].sum())
tot_emg = float(emg_cost[s1].sum())
total = tot_plan + tot_emg
dates = [datetime(2025, 1, 1).replace(year=2025) for _ in range(ND)]
from datetime import timedelta

```

```

dates = [datetime(2025, 1, 1) + timedelta(days=i) for i in range(ND)]

# ----- 校验 -----
E_traj_rep = E_traj[s1]
checks = {
    "report_days": int(ND - i0),
    "energy_min_kwh": float(E_traj_rep.min()),
    "energy_max_kwh": float(E_traj_rep.max()),
    "charge_max_kwh_per_slot": float(c_arr[s1].max()),
    "discharge_max_kwh_per_slot": float(d_arr[s1].max()),
    "flow_cap_kwh_per_slot": FLOW_CAP,
    "grid_nonneg": bool((g_arr[s1] >= -1e-7).all()),
    "simultaneous_charge_discharge_periods": int(
        ((c_arr[s1] > 1e-6) & (d_arr[s1] > 1e-6)).sum()),
    "plan_supply_ge_forecast_min_margin": float(
        np.min(z_arr[s1] - np.array([forecast_net(L, V, d) * DT for d in range(i0, ND)]))),
    "emergency_equals_deficit_max_dev": float(
        np.abs(e_arr[s1] - np.maximum(N[s1] * DT - z_arr[s1], 0)).max()),
    "cross_day_continuity_max_dev": float(np.abs(E_end[i0:ND-1] - E_start[i0+1:ND]).max()),
    "year_end_energy_kwh": float(E_end[-1]),
}
checks["pass"] = bool(
    checks["energy_min_kwh"] >= E_MIN - 1e-6
    and checks["energy_max_kwh"] <= E_MAX + 1e-6
    and checks["charge_max_kwh_per_slot"] <= FLOW_CAP + 1e-6
    and checks["discharge_max_kwh_per_slot"] <= FLOW_CAP + 1e-6
    and checks["grid_nonneg"]
    and checks["simultaneous_charge_discharge_periods"] == 0
    and checks["cross_day_continuity_max_dev"] <= 1e-6
    and abs(checks["year_end_energy_kwh"] - YEAR_END_E) <= 1e-6
)

# ----- 汇总 -----
emg_periods = int((e_arr[s1] > 1e-6).sum())
dump = float(np.maximum(z_arr[s1] - N[s1] * DT, 0).sum())
summary = {
    "scheme": "1439万方案（48h前瞻 + 不等式 + 周期终端 + 漂移校正预测）",
    "report_period": [dates[i0].strftime("%Y-%m-%d"), dates[-1].strftime("%Y-%m-%d")],
    "days": int(ND - i0),
    "total_cost_yuan": total,
    "total_cost_wan": total / 1e4,
    "planned_cost_yuan": tot_plan,
    "emergency_cost_yuan": tot_emg,
    "emergency_kwh": float(e_arr[s1].sum()),
    "planned_grid_kwh": float(g_arr[s1].sum()),
    "charge_kwh": float(c_arr[s1].sum()),
    "discharge_kwh": float(d_arr[s1].sum()),
    "dump_kwh": dump,
    "dump_ratio_of_plan": float(dump / max(g_arr[s1].sum(), 1e-9)),
    "emergency_periods": emg_periods,
    "emergency_period_fraction": emg_periods / float((ND - i0) * T),
    "emergency_days": int((e_arr[s1].sum(axis=1) > 1e-6).sum()),
}

```

```

    "energy_initial_kwh": float(E_start[i0]),
    "energy_final_kwh": float(E_end[-1]),
    "config": {"horizon": HORIZON, "window": WINDOW,
               "load_kernel": f"sw{LOAD_K}/drift{LOAD_DRIFT}",
               "pv_kernel": f"t{PV_M}/drift{PV_DRIFT}",
               "emergency_multiplier": EMERGENCY_MULT,
               "newsboy_fractile": 1 - 1 / EMERGENCY_MULT},
}

print()
print("=" * 64)
print(f"结果期 {summary['report_period'][0]} ~ {summary['report_period'][1]}")
print(f"  计划购电费 : {tot_plan:,.2f} 元")
print(f"  紧急购电费 : {tot_emg:,.2f} 元")
print(f"  总购电费   : {total:,.2f} 元   ({total/1e4:.1f} 万)")
print(f"  紧急购电量 : {summary['emergency_kwh']:,.2f} kWh")
print(f"  弃置电量   : {dump:,.2f} kWh")
print(f"  校验通过   : {checks['pass']}")

# ----- 导出 result2.xlsx -----
export_result2(g_arr[s1], c_arr[s1], d_arr[s1], E_start[s1], E_end[s1],
               e_arr[s1], dates[i0:], HERE / "result2.xlsx", price, plan_cost[s1])
export_daily_csv(g_arr[s1], e_arr[s1], plan_cost[s1], emg_cost[s1],
                 E_start[s1], E_end[s1], c_arr[s1], d_arr[s1],
                 z_arr[s1], N[s1], dates[i0:], HERE / "daily_summary.csv")
export_detail_csv(g_arr[s1], c_arr[s1], d_arr[s1], z_arr[s1], FC[s1],
                  e_arr[s1], E_end[s1], dates[i0:], HERE / "schedule_detail.csv")
(HERE / "summary.json").write_text(
    json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8")
(HERE / "verification.json").write_text(
    json.dumps(checks, ensure_ascii=False, indent=2), encoding="utf-8")

# ----- 图表 -----
try:
    from make_figures_B import make_all_figures
    make_all_figures(HERE, g_arr[s1], e_arr[s1], c_arr[s1], d_arr[s1],
                     z_arr[s1], Etraj[s1], E_start[s1], E_end[s1],
                     plan_cost[s1], emg_cost[s1], N[s1], FC[s1],
                     price, dates[i0:], summary, R, L, V)
    print("  图表已生成: figures/")
except Exception as ex:
    # 绘图失败不影响数值结果
    print(f"  [警告] 绘图失败: {ex}")

print(f"\n输出目录: {HERE}")

# ----- 导出辅助 -----
def export_result2(g, c, d, E_start, E_end, e, dates, path, price, plan_cost):
    """result2.xlsx (修复版: 充放电时段按 24 段/4 小时切分) 。"""
    import openpyxl
    from openpyxl.styles import Font, Alignment

```

```

labels = template_labels()
wb = openpyxl.Workbook()
bold = Font(bold=True)

ws = wb.active; ws.title = "计划购电量"
ws.append(["日期\\时间"] + labels + ["全天购电量", "全天购电费"])
for cc in ws[1]:
    cc.font = bold; cc.alignment = Alignment(horizontal="center")
for i, dt in enumerate(dates):
    ws.append([dt] + [round(float(v), 4) for v in g[i]] +
                [round(float(g[i].sum()), 4), round(float(plan_cost[i]), 2)])
ws.column_dimensions["A"].width = 12
ws.freeze_panes = "B2"

ws2 = wb.create_sheet("充放电电量")
ws2.append(["日期", "时间段", "充电量", "放电量", "时刻", "储电量"])
for cc in ws2[1]:
    cc.font = bold
blocks = ["0:00-4:00", "4:00-8:00", "8:00-12:00",
          "12:00-16:00", "16:00-20:00", "20:00-24:00"]
for i, dt in enumerate(dates):
    for b, nm in enumerate(blocks):
        t0, t1 = b * 24, (b + 1) * 24          # ← 修复: 原为 b*36
        ws2.append([dt if b == 0 else None, nm,
                    round(float(c[i, t0:t1].sum()), 4),
                    round(float(d[i, t0:t1].sum()), 4),
                    time(0, 0) if b == 0 else ("24:00" if b == 1 else None),
                    round(float(E_start[i]), 4) if b == 0
                    else (round(float(E_end[i]), 4) if b == 1 else None)])
for col, wd in zip("ABCDEF", (12, 14, 12, 12, 10, 12)):
    ws2.column_dimensions[col].width = wd

ws3 = wb.create_sheet("紧急购电量")
ws3.append(["日期", "购电时间段", "购电量"])
for cc in ws3[1]:
    cc.font = bold
for i, dt in enumerate(dates):
    ivs = merge_intervals(e[i])
    if not ivs:
        ws3.append([dt, "—", 0.0]); continue
    for j, (lab0, lab1, q) in enumerate(ivs):
        ws3.append([dt if j == 0 else None, f"{lab0}-{lab1}", round(q, 4)])
for col, wd in zip("ABC", (12, 20, 12)):
    ws3.column_dimensions[col].width = wd
wb.save(path)

def _lbl(t):
    m = t * 10
    return "0:00+1" if m == 1440 else f"{m // 60:02d}:{m % 60:02d}"

```

```

def merge_intervals(e, tol=1e-6):
    out, t = [], 0
    while t < T:
        if e[t] > tol:
            t0, q = t, 0.0
            while t < T and e[t] > tol:
                q += e[t]; t += 1
            out.append((t0, t, q))
        else:
            t += 1
    # 区间标签用"起点-终点"
    return [(_lbl_hm(a), _lbl_hm(b), q) for a, b, q in out]

def _lbl_hm(t):
    m = t * 10
    return "0:00+1" if m >= 1440 else f"{m // 60}:{m % 60:02d}"

def export_daily_csv(g, e, plan_cost, emg_cost, E_start, E_end, c, d, z, N,
                    dates, path):
    import csv
    with open(path, "w", newline="", encoding="utf-8-sig") as f:
        w = csv.writer(f)
        w.writerow(["日期", "计划购电量kWh", "紧急购电量kWh", "计划购电费元",
                    "紧急购电费元", "合计购电费元", "紧急时段数",
                    "0:00储电量kWh", "24:00储电量kWh", "充电量kWh", "放电电量kWh",
                    "计划净供给kWh", "弃置电量kWh"])
        for i, dt in enumerate(dates):
            w.writerow([dt.strftime("%Y-%m-%d"),
                        f"{g[i].sum():.4f}", f"{e[i].sum():.4f}",
                        f"{plan_cost[i]:.2f}", f"{emg_cost[i]:.2f}",
                        f"{plan_cost[i]+emg_cost[i]:.2f}",
                        int(np.sum(e[i] > 1e-6)),
                        f"{E_start[i]:.4f}", f"{E_end[i]:.4f}",
                        f"{c[i].sum():.4f}", f"{d[i].sum():.4f}",
                        f"{z[i].sum():.4f}",
                        f"{max(z[i].sum() - N[i].sum()*DT, 0):.4f}"])

def export_detail_csv(g, c, d, z, fc, e, E_end, dates, path):
    import csv
    with open(path, "w", newline="", encoding="utf-8-sig") as f:
        w = csv.writer(f)
        w.writerow(["日期", "时段", "计划购电量kWh", "充电量kWh", "放电电量kWh",
                    "计划净供给kWh", "净负载预测kWh", "紧急购电量kWh"])
        for i, dt in enumerate(dates):
            for t in range(T):
                w.writerow([dt.strftime("%Y-%m-%d"), _lbl_hm(t + 1),
                            f"{g[i,t]:.4f}", f"{c[i,t]:.4f}", f"{d[i,t]:.4f}",
                            f"{z[i,t]:.4f}", f"{fc[i,t]:.4f}", f"{e[i,t]:.4f}"])

```

```
if __name__ == "__main__":
    main()
```

附录 E.7 公共模块：附件 5 结果模板表头（主程序依赖）

template_layout.py

"""附件5 模板的时间标签与「环形映射」规则（各问导出统一调用）。

问题

附件1/2/4 的行标签是**区间末**时刻（`0:10` 表示 00:00 - 00:10，`0:00+1` 表示 24:00），而附件5 模板的表头是**左端点**且首格为 `0:10-0:20`、末格为 `0:00+1-0:10+1`。二者相差一格。

规则（依据官方参考实现 CUMCM2026Problems/C题/问题一/make_result1.py）

模板第 i 格（0-based）对应内部时段

$idx = (i + 1) \% 144$

即：

模板 `0:10-0:20` ← 内部 slot 1（区间 00:10 - 00:20）

...

模板 `23:50-0:00+1` ← 内部 slot 143（区间 23:50 - 24:00）

模板 `0:00+1-0:10+1` ← 内部 slot 0（环形环绕）

这样既与模板逐格对齐，又保证每个值落在标签所指的时段上。

验证

以官方 `问题一/result1.xlsx` 为准，本规则 142/144 格精确吻合（余 2 格为官方样本自身的陈旧值，差额相等且相反，不影响规则）；而「位置保持」仅 54/144 吻合。

"""

```
from __future__ import annotations
```

```
from pathlib import Path
```

```
import openpyxl
```

```
# 模板表头（左端点）与内部时段（区间末）相差一格
```

```
SHIFT = 1
```

```
# 各问结果文件的模板（供导出器直接引用）
```

```
REPO = Path(__file__).resolve().parents[2]
```

```
TPL_DIR = REPO / "data" / "附件" / "附件5"
```

```
TPL1 = TPL_DIR / "result1.xlsx"
```

```
TPL2 = TPL_DIR / "result2.xlsx"
```

```
TPL3 = TPL_DIR / "result3.xlsx"
```

```
TPL42 = TPL_DIR / "result4-2.xlsx"
```

```
TPL43 = TPL_DIR / "result4-3.xlsx"
```

```
def template_path(repo: Path, name: str) -> Path:
```

```
    return repo / "data" / "附件" / "附件5" / name
```

```

def template_labels(path: Path, sheet: str = "计划购电量") -> list[str]:
    """从模板读取 144 个时间标签（原样，保留模板自身的写法如 7:0-7:10）。"""
    wb = openpyxl.load_workbook(path, read_only=True)
    ws = wb[sheet]
    rows = list(ws.iter_rows(max_row=2, values_only=True))
    hdr = list(rows[0]) if rows else []
    wb.close()

    # 行式 (result1): 标签在 A 列第 2..145 行
    if len(hdr) <= 3:
        wb = openpyxl.load_workbook(path, read_only=True)
        ws = wb[sheet]
        labels = [str(r[0]) for r in ws.iter_rows(min_row=2, max_row=145,
                                                    values_only=True)]
        wb.close()
    else:
        labels = [str(x) for x in hdr[1:145]]
    if len(labels) != 144:
        raise ValueError(f"{path.name}[{sheet}] 标签数 {len(labels)} != 144")
    return labels


def circular(values, shift: int = SHIFT):
    """把 144 个内部时段值排成模板格顺序: out[i] = values[(i + shift) % 144]。"""
    n = len(values)
    return [values[(i + shift) % n] for i in range(n)]


def sheet_names(path: Path) -> list[str]:
    wb = openpyxl.load_workbook(path, read_only=True)
    names = wb.sheetnames
    wb.close()
    return names


def header_of(path: Path, sheet: str) -> list:
    wb = openpyxl.load_workbook(path, read_only=True)
    ws = wb[sheet]
    hdr = list(next(ws.iter_rows(max_row=1, values_only=True)))
    wb.close()
    return hdr


def block_labels(path: Path, sheet: str = "充放电量", n: int = 6) -> list[str]:
    """模板「充放电量」的 6 个 4 小时时段标签（如 0:00-4:00）。

    两种模板列布局不同:
        result1          5 列: 时间段 | 充电量 | 放电量 | 时刻 | 储电量
        result2/3/4-*    6 列: 日期 | 时间段 | 充电量 | 放电量 | 时刻 | 储电量
    故按表头定位「时间段」列，避免写死列号。
    """
    wb = openpyxl.load_workbook(path, read_only=True)

```

```

ws = wb[sheet]
rows = list(ws.iter_rows(min_row=1, max_row=1 + n, values_only=True))
wb.close()
hdr = [str(x).strip() if x is not None else "" for x in rows[0]]
col = hdr.index("时间段") if "时间段" in hdr else 0
return [str(r[col]) for r in rows[1:1 + n]]

def lapse_label(minutes: int) -> str:
    """内部时段序号 (0-based) → 区间末标签 ('0:00+1' 表示 24:00) 。”"""
    m = minutes * 10
    return "0:00+1" if m >= 1440 else f"{m // 60}:{m % 60:02d}"

```